

Dirección Xeral de Educación, Formación Profesional e Innovación Educativa

Índice

1. A1. Menús e xanelas 3

1.1. Introducción.....	3
1.2. Actividade.....	3
1.2.1. Menús 3	
1.2.1.1. Menús despregables	4
Configuración dun menú despregable empregando NetBeans.....	5
1.2.1.2. Menús contextuais	16
Configuración dun menú contextual empregando NetBeans.....	16
1.2.2. Aplicacións multixanela	22
1.2.2.1. Aplicacións multixanela Dialog	22
Configuración dunha aplicación multixanela Dialog empregando NetBeans.....	24
Paso de información entre xanelas.....	42
1.2.2.2. Aplicacións multixanela MDI	53
Configuración dunha aplicación multixanela MDI empregando NetBeans.....	55
1.2.3. Diálogos predefinidos	72
1.2.3.1. Diálogo showMessageDialog	72
1.2.3.2. Diálogo showConfirmDialog	76
1.2.3.3. Diálogo showInputDialog	81
1.2.3.4. Diálogo showOptionDialog	88
1.2.3.5. Consideracións finais	95

1. A1. Menús e xanelas

1.1. Introducción

Na actividade que nos ocupa aprenderanse os seguintes conceptos e manexo de destrezas:

- Crear e xestionar nas nosas aplicacións gráficas de usuario os diferentes tipos de menús proporcionados pola linguaxe de programación java.
- Implementar aplicacións multixanela empregando diversas técnicas para a xestionar o entendemento entre as diferentes xanelas que poden compoñer as nosas aplicacións.
- Creación e emprego dos diálogos predefinidos proporcionados pola linguaxe co fin de axilizar a realización de tarefas habituais.

1.2. Actividade

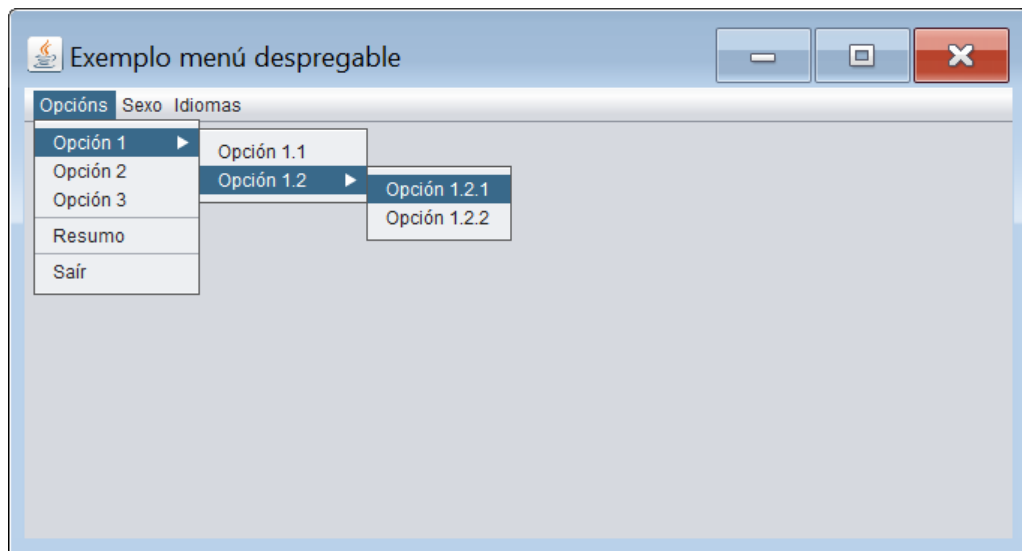
1.2.1. Menús

Un menú é unha representación gráfica ordenada dunha serie de opcións que nos proporciona unha aplicación. En función da opción seleccionada realizarase algunha acción asociada á selección. A linguaxe java proporcionáanos dous tipos de menús, menús despregables e menús contextuais.

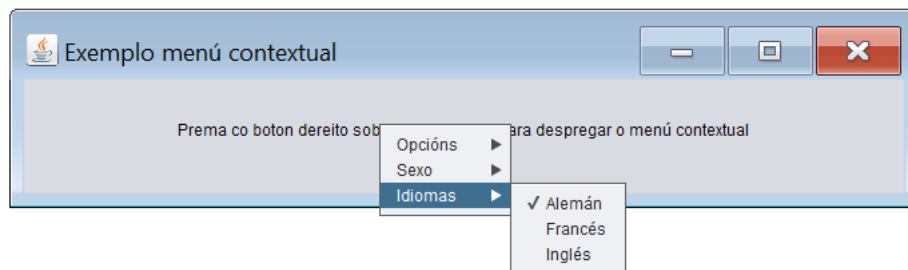
Os menús empregados habitualmente son de tipo menú despregable. Constan dunha serie de opcións organizadas nunha estrutura de árbore que colga dunha barra de menú. Á vez, a barra de menú colga dunha xanela (tipicamente do borde superior da xanela). Inicialmente só será visible a barra de menú, a cal conterá as opcións principais do menú. Ao premer sobre algunha destas opcións principais despregaranse os distintos submenús que colgan da barra de menú.

Pola outra banda, os menús contextuais inicialmente non son visibles. Constan dunha serie de opcións organizadas nunha estrutura de árbore que se visualizará unicamente ao realizar algunha acción (tipicamente clic dereito) sobre o compoñente que ten asociado o menú contextual.

A continuación amósase o aspecto visual dun menú despregable:



Nas seguinte imaxe pódese visualizar o aspecto dun menú contextual:



1.2.1.1. Menús despregables

Como acabamos de explicar un menú despregable consta basicamente dunha barra de menú da que colgan as opcións principais e de cada opción á vez poden colgar outros submenús (esta estrutura poderíase repetir recursivamente aínda que por motivos de claridade recoméndase non descender demasiados niveis na creación de submenús). A organización das distintas opcións que compoñen o menú ten unha estrutura de árbore na cal as ramas son accesos a submenús mentres que as follas son as opcións finais.

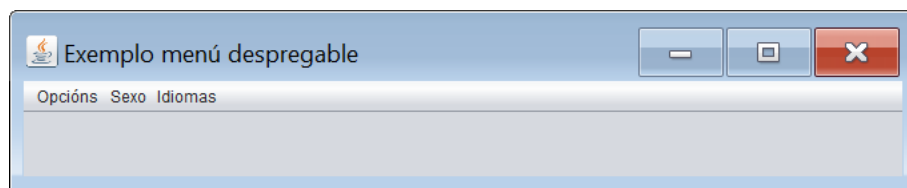
A creación de menús despregables baséase no emprego de obxectos das seguintes clases:

- `javax.swing.JMenuBar`: esta clase representa unha barra de menú
- `javax.swing.JMenu`: esta clase representa un menú. Dun menú pódense colgar máis `JMenu` (para a creación de submenús) o pódense colgar obxectos das clases `JMenuItem`, `JRadioButtonMenuItem`, `JCheckBoxMenuItem` e `JSeparator`. Os obxectos `JMenu` compoñen as ramas do árbore do menú, mentres que os obxectos das clases que imos comentar de seguido son as súas follas.
- `javax.swing.JMenuItem`: esta clase representa unha folla da árbore de opcións de menú. É unha opción á que se lle asocia a realización dunha acción ao ser seleccionada.
- `javax.swing.JRadioButtonMenuItem`: esta clase representa unha folla da árbore de opcións do menú. Compórtase como un botón de opción podendo estar ou non seleccionada entre un grupo de `JRadioButtonMenuItem`s. É unha opción á que pódesele asociar a realización dunha acción ao ser seleccionada ou deseleccionada, aínda que normalmente emprégase para indicar un valor de varios posibles, de xeito que a nosa aplicación funcione dun modo determinado en función desa selección.

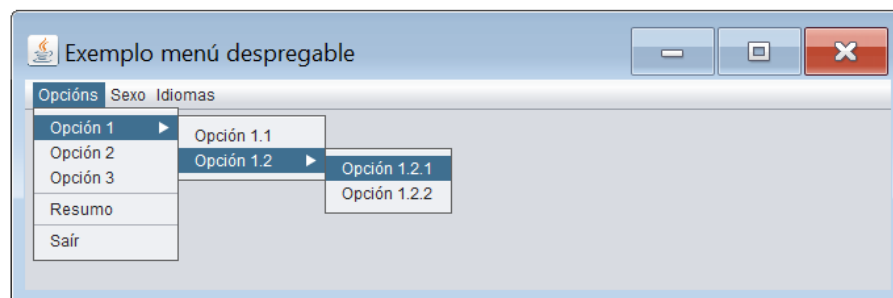
- `javax.swing.JCheckBoxMenuItem`: esta clase representa unha folla da árbore de opcións do menú. Comportase como unha casilla de verificación podendo estar ou non seleccionada. É unha opción á que pódese asociar a realización dunha acción ao ser seleccionada ou deseleccionada, aínda que normalmente emprégase para indicar se un valor está activo ou non, de xeito que a nosa aplicación funcione dun modo determinado en función desa situación.
- `javax.swing.JSeparator`: esta clase representa unha liña e emprégase para separar visualmente dous opcións de menú entre si.

Configuración dun menú despregable empregando NetBeans

Co fin de explicar como crear un menú despregable empregando NetBeans imos desenvolver a seguinte aplicación:



Da opción principal Opcións colgarán os seguintes elementos:



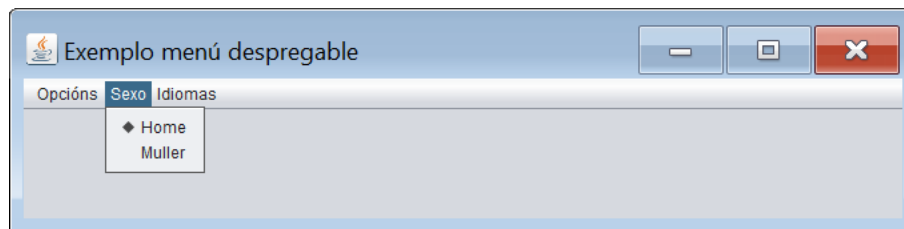
As opcións cunha frecha indican que son contedores de submenús (son obxectos de clase `JMenu`) de xeito que ao situarnos sobre elas co rato, ábrese o submenú asociado. Son ramas da árbore de opcións do menú.

O resto de opcións son follas da árbore de opcións do menú (son obxectos da clase `JMenuItem`):

- As opcións Opción 1.1, Opción 1.2.1, Opción 1.2.2, Opción 2 e Opción 3 ao ser premidas amosarán unha mensaxe indicando cal foi a opción seleccionada.
- A opción Resumo amosará por pantalla o sexo seleccionado no menú sexo e os idiomas seleccionados no menú Idiomas.
- A opción Saír finalizará a execución da aplicación.

Adicionalmente hai dous obxectos de clase `JSeparator` que empregaremos para separar entre si algunhas das opcións de menú.

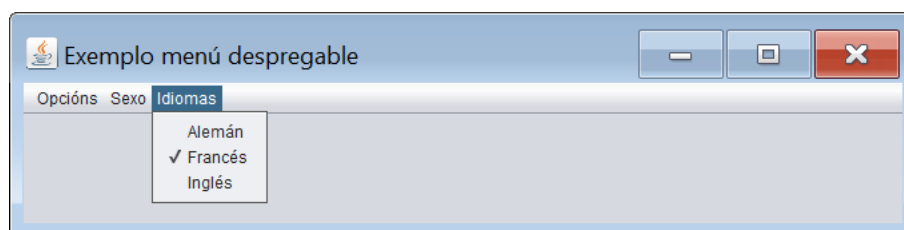
Da opción principal Sexo colgarán os seguintes elementos:



As dous opcións presentes neste menú son follas da árbore de opcións de menú (son obxectos da clase `JRadioButtonMenuItem`). Só unha delas pode estar seleccionada simultaneamente.

Cando elixamos a opción Home ademais de seleccionar esa opción, amosaremos unha mensaxe por pantalla indicando a selección realizada. Para a opción Muller faremos o mesmo.

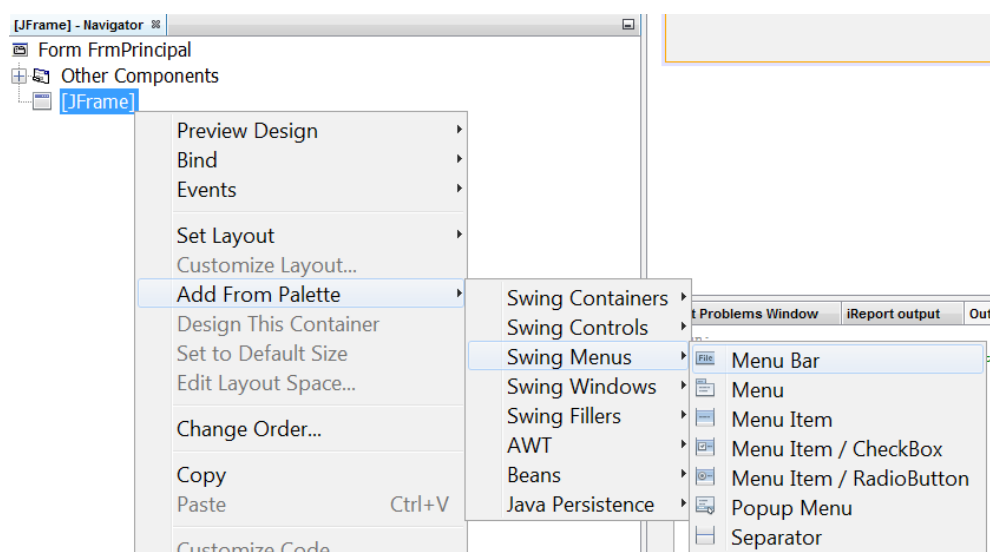
Por último da opción principal Idiomas colgarán os seguintes elementos:



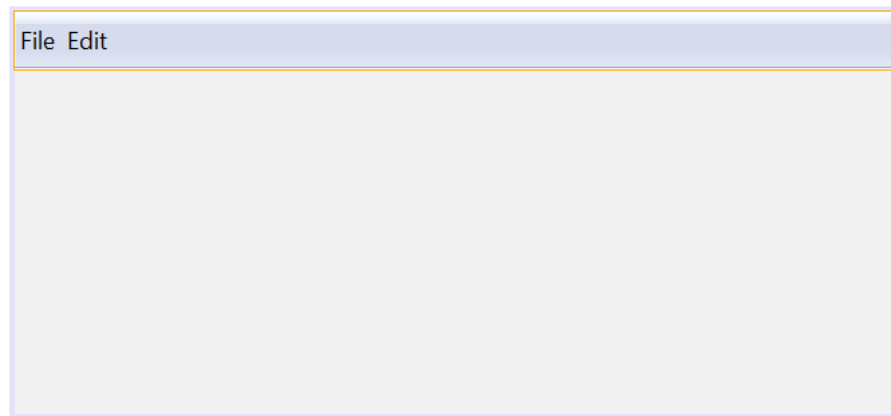
As tres opcións presentes neste menú son follas da árbore de opcións do menú (son obxectos da clase `JCheckBoxMenuItem`). Cada unha delas pode estar marcada ou desmarcada. Cando marquemos ou desmarquemos calquera delas amosaremos unha mensaxe por pantalla indicando a acción realizada.

Existen diferentes xeitos para xerar un menú empregando NetBeans. Aquí vaise explicar como facelo empregando a xanela de obxectos do formulario (xanela Navigator).

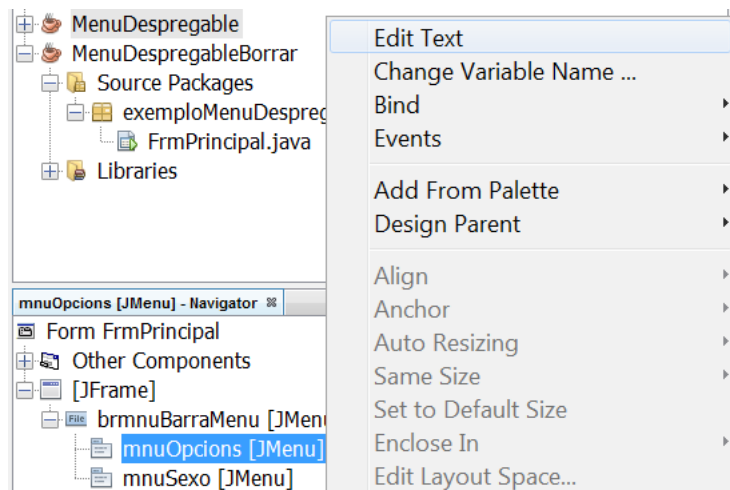
Partindo dun `JFrame` xa creado, imos engadirlle unha barra de menú. Para elo prememos co botón dereito do rato sobre o `JFrame` ao que lle queremos engadir a barra de menú e seleccionamos `Add from Palette / Swing Menus / Menu Bar`:



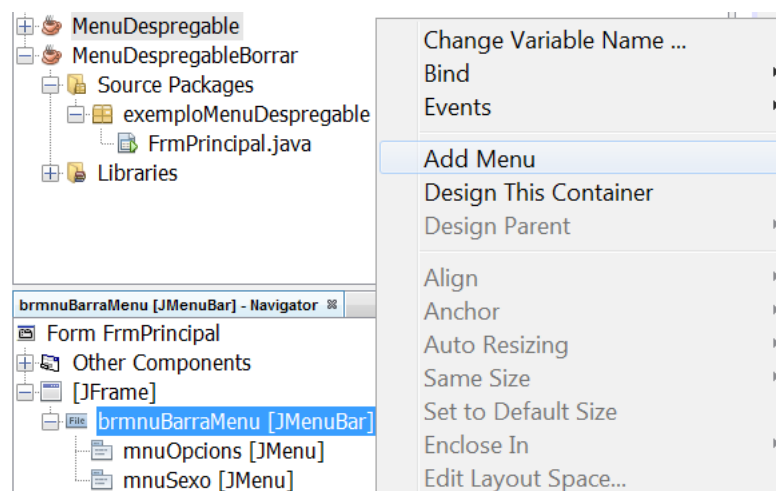
Ao realizar esta acción o entorno de programación engadirá ao formulario unha barra de menú con dúas opcións predefinidas (de clase `JMenu`):



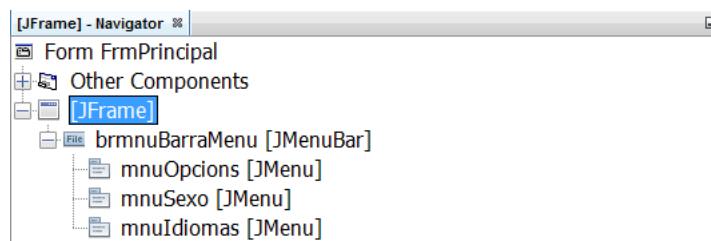
A estas dúas opcións de menú despois de darlles un nome interno máis descritivo, deberémoslle cambiar o texto. Imos ver como cambiar o texto que amosa o primeiro JMenu. Prememos co botón dereito do rato sobre o JMenu ao que queremos modificar o seu texto e seleccionamos a opción Edit Text. Por último modificamos o seu texto para que conteña o texto que desexamos que teña:



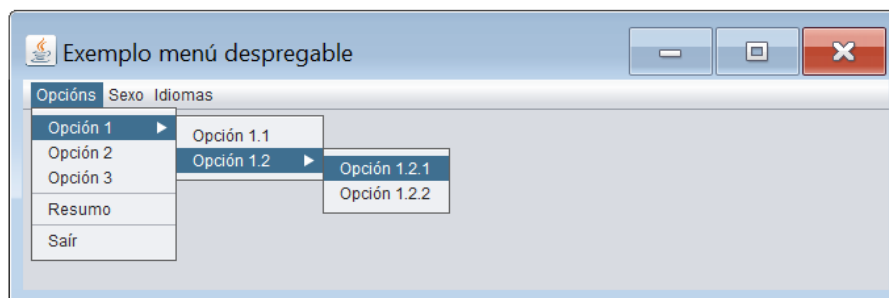
Agora mesmo temos dous opcións de menú pero precisamos de tres, polo tanto imos ver como engadir unha terceira. Como a opción de menú debe colgar da barra de menú, prememos co botón dereito sobre a barra de menú e seleccionamos a opción Add Menu. Unha vez engadido o terceiro menú modificaremos o seu nome interno e o seu texto.



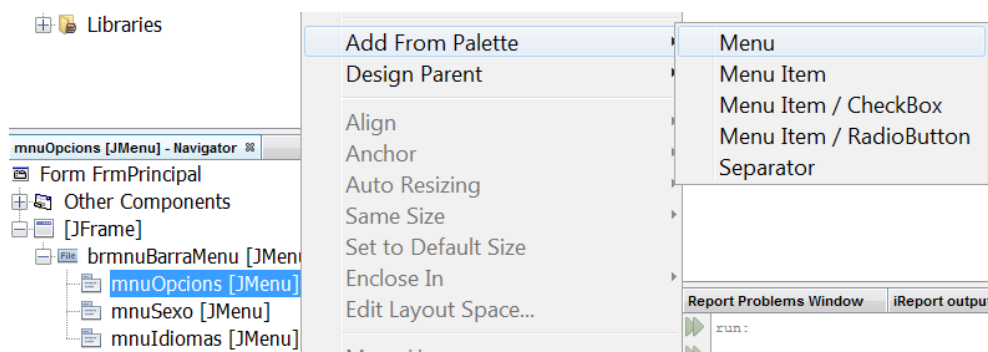
Deste xeito temos construídas tódalas opcións que colgan da barra de menú:



Agora ímonos centrar na rama que colga da opción Opcións.



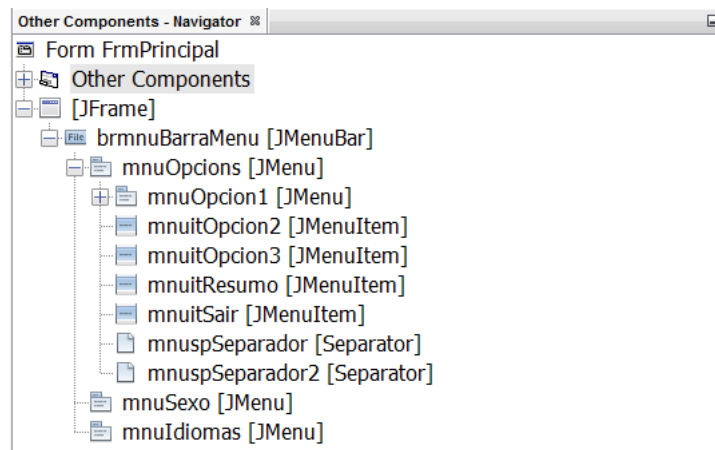
Opción 1 será un obxecto de clase JMenu (xa que vaise empregar para despregar un submenú) e colgará do obxecto mnuOpciones. Para crealo prememos co botón dereito do rato sobre o contedor da opción que imos crear e seleccionamos Add From Palette / Menu para indicar que queremos crear un JMenu.



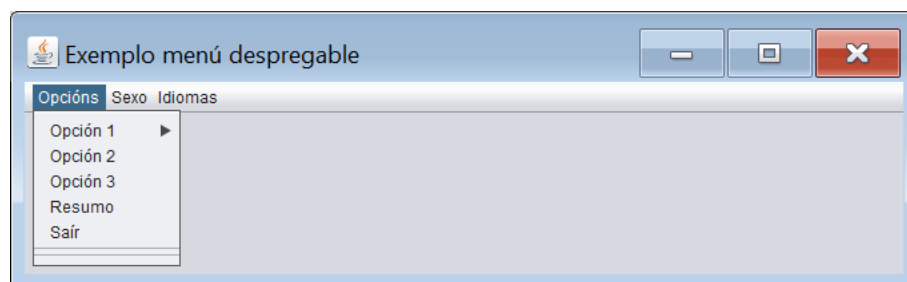
Non obstante, Opción 2 será un obxecto de clase JMenuItem (xa que vai ser unha folla da árbore do menú). Ademais tamén colgará do obxecto mnuOpciones. Para crealo prememos co botón dereito do rato sobre o contedor da opción que imos crear e seleccionamos Add From Palette / Menu Item para indicar que queremos crear un JMenuItem. Repetiremos este proceso para as opcións de menú Opción 3, Resumo e Saír.

Ademais temos que crear dous separadores de opcións de menú (obxectos da clase JSeparator). Para crear un separador prememos co botón dereito sobre o contedor do separador que imos crear e seleccionamos Add From Palette / Separator para indicar que queremos crear un JSeparator.

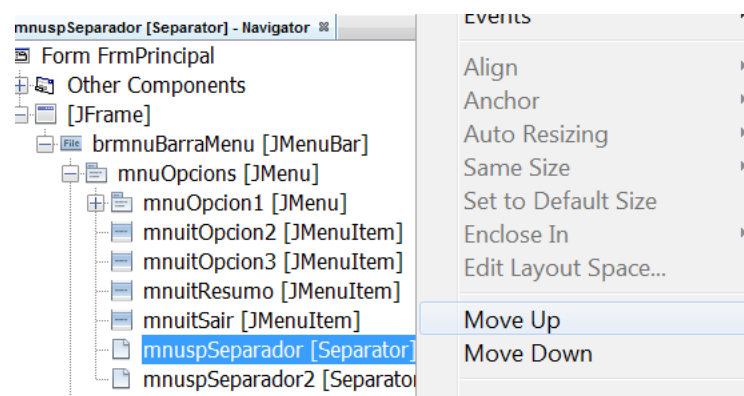
Unha vez creados estes elementos este é o resultado:



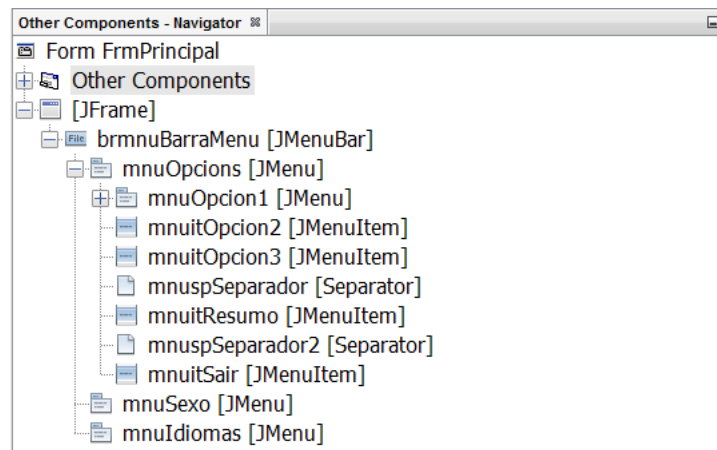
Se executamos a aplicación e prememos sobre o menú opcións observaremos o seguinte:



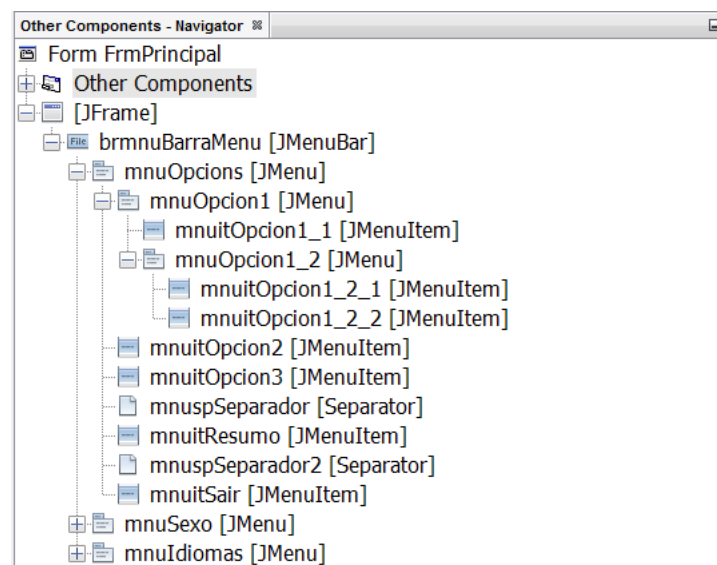
Os separadores aparecen despois das opcións. Isto é debido a que a orden dos elementos no menú é a mesma que a orde que ocupan na árbore de obxectos do proxecto. Neste caso vemos que na árbore do proxecto `mnuspSeparador` e `mnuspSeparador2` aparecen ao final da súa rama e polo tanto ao executar a aplicación tamén aparecen ao final do menú. Para solucionalo deberemos colocar a `mnuspSeparador` e a `mnuspSeparador2` no lugar que queremos que ocupen na árbore. A continuación imos ver como facer isto para `mnuspSeparador`. Prememos co botón dereito do rato sobre o elemento que queremos desprazar e prememos en `Move Up`. Con isto subiremos a `mnuspSeparador` unha posición na árbore de obxectos, pasando a estar entre `mnuitResumo` e `mnuitSair`. Repetiremos este proceso tantas veces como sexa necesario co fin de colocar cada elemento do menú no seu lugar. Se en lugar de subir un elemento quixéramolo baixar teremos que seleccionar a opción `Move Down`:



Finalmente esta é a configuración actual do noso menú:



Agora deberíamos crear os elementos do submenú que é despregado ao pasar sobre mnuOpcion1. Este submenú contén un JMenuItem (Opción 1.1) e un JMenu (Opción 1.2). Para crear o JMenuItem repetimos o proceso visto anteriormente fixándonos unicamente en que faremos clic co botón dereito do rato sobre o elemento que vai ser o seu contedor, é dicir, mnuOpcion1. O mesmo faremos para o JMenu. Unha vez creado este submenú crearemos o submenú que colga de Opción 1.2. Finalmente este será o resultado do menú Opcións:

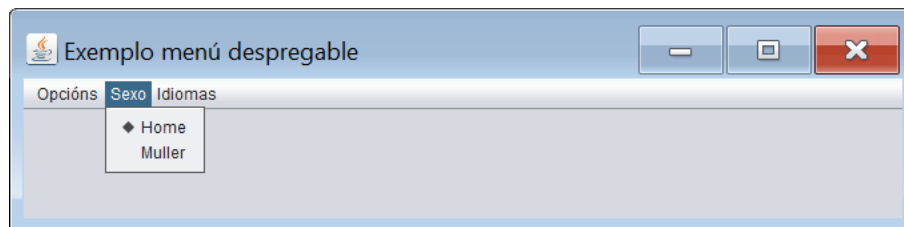


Unha vez resolta a parte visual imos implementar a súa funcionalidade. Os elementos da árbore que realizarán algunha acción son os elementos de clase JMenuItem. Imos ver como implementar a acción para a opción de menú mnuitOpcion1_1. Para elo temos que implementar o seu evento ActionPerformed. Para isto facemos dobre clic sobre o elemento mnuitOpcion1_1 na árbore de obxectos do proxecto e escribimos o seguinte código:

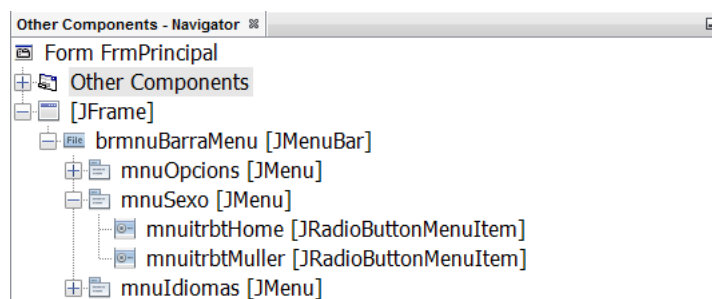
```
private void mnuitOpcion1_1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "Opción 1.1 seleccionada");
}
```

Como pódese observar no bloque de código anterior, no caso de que fagamos clic sobre a opción de menú mnuitOpcion1 amosarase por pantalla a mensaxe Opción 1.1 seleccionada.

De seguido ímonos centrar na rama que colga da opción Sexo.



Neste caso imos construír un menú cuxos compoñentes van ser `JRadioButtonMenuItem`. Procedemos do mesmo xeito que fixemos para o menú anterior, pero coa precaución de engadir elementos de clase `JRadioButtonMenuItem`. Unha vez creados esta será a configuración do menú:

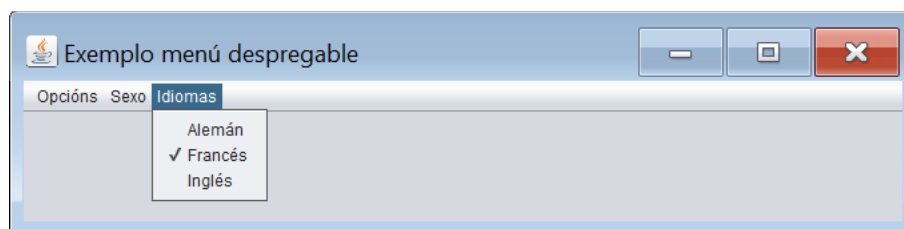


Os obxectos da clase `JRadioButtonMenuItem` compórtanse exactamente igual que os botóns de opción, polo tanto teremos que ter con eles as mesmas precaucións. Primeiramente, como queremos que unicamente un deles estea seleccionado, deberemos crear un `ButtonGroup` e asignarlle á propiedade `buttonGroup` de cada un dos `JRadioButtonMenuItem` o `buttonGroup` creado. Ademais a un deles deberemos de activarlle a súa propiedade `selected` de xeito que apareza marcado ao arrancar a aplicación. Unha vez modificadas as propiedades dos `JRadioButtonMenuItem` imos implementar os requisitos que nos piden no enunciado da aplicación. Neste caso pídenos que cando elixamos a opción Home ademais de seleccionar esa opción, amosemos unha mensaxe por pantalla indicando a selección realizada. Para a opción Muller o mesmo. Ao igual que ocorría cos botóns de opción, os `JRadioButtonMenuItem` lanzan o evento `ItemStateChanged` cando cambia o seu valor. A continuación imos ver a implementación do evento para el `JRadioButtonMenuItem` `mnuitrbtHome`:

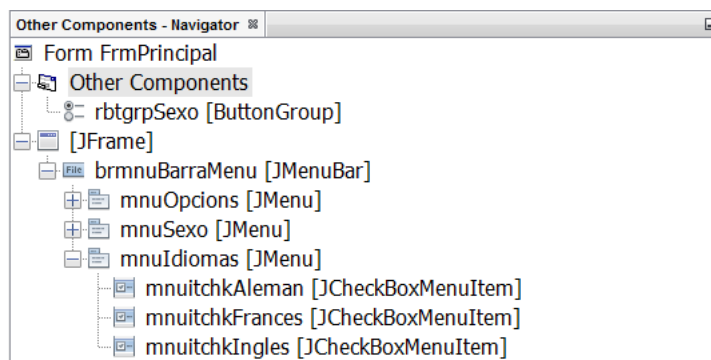
```
private void mnuitrbtHomeItemStateChanged(java.awt.event.ItemEvent evt) {
    // TODO add your handling code here:
    if (evt.getStateChange() == ItemEvent.SELECTED)
    {
        JOptionPane.showMessageDialog(this, "A opción Home foi seleccionada");
    }
}
```

Como pódese observar, o tratamento do evento é similar ao realizado para o caso dos botóns de opción.

Por ultimo, ímonos centrar na rama que colga da opción Idiomas.



Neste caso imos construír un menú cuxos compoñentes van ser JCheckBoxMenuItem. Precedemos do mesmo xeito que fixemos para os menús anteriores, pero coa precaución de engadir elementos de clase JCheckBoxMenuItem. Unha vez creados esta será a configuración do menú:



Os obxectos da clase JCheckBoxMenuItem compórtanse exactamente igual que as casillas de verificación podendo estar marcadas ou non en función do valor da súa propiedade selected. Unha vez creados os JCheckBoxMenuItem imos implementar os requisitos que nos piden no enunciado da aplicación. Neste caso pídenos que cando marquemos ou desmarquemos calquera deles amosaremos unha mensaxe por pantalla indicando a acción realizada. Ao igual que ocorría coas casillas de verificación os JCheckBoxMenuItem lanzan o evento ItemStateChanged cando cambia o seu valor. A continuación imos ver a implementación do evento para o JCheckBoxMenuItem mnuitchkAleman:

```
private void mnuitchkAlemanItemStateChanged(java.awt.event.ItemEvent evt) {
    // TODO add your handling code here:
    if(mnuitchkAleman.isSelected())
    {
        JOptionPane.showMessageDialog(this, "A opción Alemán foi marcada");
    }
    else
    {
        JOptionPane.showMessageDialog(this, "A opción Alemán foi desmarcada");
    }
}
```

Como pódese observar o tratamento do evento é similar ao realizado para o caso das casillas de verificación.

Un pequeno detalle que nos falta é ver como é recollido o elemento de menú marcado entre tódolos JRadioButtonMenuItem ou como e recollido o estado dos diferentes JCheckBoxMenuItem da nosa aplicación. Isto é feito na opción de menú Resumo. Os requisitos da aplicación indicaban que esta opción de menú debía amosar por pantalla o sexo seleccionado no menú sexo e os idiomas seleccionados no menú Idiomas. Para elo implementamos o seu evento actionPerformed do seguinte xeito:

```
private void mnuitResumoActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String sexo="SEXO: ";
    if(mnuitrbtnHome.isSelected()) sexo+="Home";
    else sexo+="Muller";
    String idiomas="IDIOMAS: ";
    if(mnuitchkAleman.isSelected()) idiomas+="Alemán ";
    if(mnuitchkFrances.isSelected()) idiomas+="Francés ";
    if(mnuitchkIngles.isSelected()) idiomas+="Inglés ";
    if(idiomas.compareTo("IDIOMAS: ")==0) sexo+="\nIDIOMAS: Ningún";
    else sexo+="\n"+idiomas;
    JOptionPane.showMessageDialog(this, sexo);
}
```

Como pódese ver, o xeito de acceder aos valores establecidos sobre un JRadioButtonMenuItem é similar ao empregado para tratar cun botón de opción e o mesmo ocorre co JCheckBoxMenuItem respecto á casilla de verificación. En canto aos métodos empregados para modificar o estado do JRadioButtonMenuItem e do JCheckBoxMenuItem,

tamén neste caso son os mesmos que os empregados para os botóns de opción e para as casillas de verificación respectivamente.

A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación:

```
/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploMenuDespregable;

import java.awt.event.ItemEvent;
import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class FrmPrincipal extends javax.swing.JFrame {

    /**
     * Creates new form FrmPrincipal
     */
    public FrmPrincipal() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        rbtgrpSexo = new javax.swing.ButtonGroup();
        brmnvBarraMenu = new javax.swing.JMenuBar();
        mnuOpciones = new javax.swing.JMenu();
        mnuOpcion1 = new javax.swing.JMenu();
        mnuitOpcion1_1 = new javax.swing.JMenuItem();
        mnuitOpcion1_2 = new javax.swing.JMenuItem();
        mnuitOpcion1_2_1 = new javax.swing.JMenuItem();
        mnuitOpcion1_2_2 = new javax.swing.JMenuItem();
        mnuitOpcion2 = new javax.swing.JMenuItem();
        mnuitOpcion3 = new javax.swing.JMenuItem();
        mnuspSeparador = new javax.swing.JPopupMenu.Separator();
        mnuitResumo = new javax.swing.JMenuItem();
        mnuspSeparador2 = new javax.swing.JPopupMenu.Separator();
        mnuitSair = new javax.swing.JMenuItem();
        mnuSexo = new javax.swing.JMenu();
        mnuitrbtHome = new javax.swing.JRadioButtonMenuItem();
        mnuitrbtMuller = new javax.swing.JRadioButtonMenuItem();
        mnuIdiomas = new javax.swing.JMenu();
        mnuitchkAleman = new javax.swing.JCheckBoxMenuItem();
        mnuitchkFrances = new javax.swing.JCheckBoxMenuItem();
        mnuitchkIngles = new javax.swing.JCheckBoxMenuItem();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Exemplo menú despregable");

        mnuOpciones.setText("Opciones");

        mnuOpcion1.setText("Opción 1");

        mnuitOpcion1_1.setText("Opción 1.1");
        mnuOpcion1_1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                mnuitOpcion1_1ActionPerformed(evt);
            }
        });
        mnuOpcion1.add(mnuitOpcion1_1);

        mnuOpcion1_2.setText("Opción 1.2");

        mnuitOpcion1_2_1.setText("Opción 1.2.1");
        mnuitOpcion1_2_1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                mnuitOpcion1_2_1ActionPerformed(evt);
            }
        });
        mnuOpcion1_2.add(mnuitOpcion1_2_1);

        mnuitOpcion1_2_2.setText("Opción 1.2.2");
        mnuitOpcion1_2_2.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                mnuitOpcion1_2_2ActionPerformed(evt);
            }
        });
        mnuOpcion1_2.add(mnuitOpcion1_2_2);

        mnuOpcion1.add(mnuOpcion1_2);

        mnuOpciones.add(mnuOpcion1);

        mnuitOpcion2.setText("Opción 2");
        mnuitOpcion2.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                mnuitOpcion2ActionPerformed(evt);
            }
        });
    }
}
```

```

mnuOpciones.add(mnuitOpcion2);

mnuitOpcion3.setText("Opción 3");
mnuitOpcion3.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        mnuitOpcion3ActionPerformed(evt);
    }
});
mnuOpciones.add(mnuitOpcion3);
mnuOpciones.add(mnuspSeparador);

mnuitResumo.setText("Resumo");
mnuitResumo.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        mnuitResumoActionPerformed(evt);
    }
});
mnuOpciones.add(mnuitResumo);
mnuOpciones.add(mnuspSeparador2);

mnuitSair.setText("Sair");
mnuitSair.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        mnuitSairActionPerformed(evt);
    }
});
mnuOpciones.add(mnuitSair);

brmnubarraMenu.add(mnuOpciones);

mnuSexo.setText("Sexo");

rbtgrpSexo.add(mnuitrbtHome);
mnuitrbtHome.setSelected(true);
mnuitrbtHome.setText("Home");
mnuitrbtHome.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitrbtHomeItemStateChanged(evt);
    }
});
mnuSexo.add(mnuitrbtHome);

rbtgrpSexo.add(mnuitrbtMuller);
mnuitrbtMuller.setText("Muller");
mnuitrbtMuller.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitrbtMullerItemStateChanged(evt);
    }
});
mnuSexo.add(mnuitrbtMuller);

brmnubarraMenu.add(mnuSexo);

mnuIdiomas.setText("Idiomas");

mnuitchkAleman.setText("Alemán");
mnuitchkAleman.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitchkAlemanItemStateChanged(evt);
    }
});
mnuIdiomas.add(mnuitchkAleman);

mnuitchkFrances.setText("Francés");
mnuitchkFrances.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitchkFrancesItemStateChanged(evt);
    }
});
mnuIdiomas.add(mnuitchkFrances);

mnuitchkIngles.setText("Inglés");
mnuitchkIngles.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitchkInglesItemStateChanged(evt);
    }
});
mnuIdiomas.add(mnuitchkIngles);

brmnubarraMenu.add(mnuIdiomas);

setJMenuBar(brmnubarraMenu);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(0, 659, Short.MAX_VALUE)
            >
);
layout.setVerticalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(0, 257, Short.MAX_VALUE)
            >
);

java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
setBounds((screenSize.width-681)/2, (screenSize.height-364)/2, 681, 364);
}

private void mnuitOpcion1_2_1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "Opción 1.2.1 seleccionada");
}

private void mnuitOpcion1_2_2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:

```

```

        JOptionPane.showMessageDialog(this, "Opción 1.2.2 seleccionada");
    }

    private void mnuitOpcion1_1ActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        JOptionPane.showMessageDialog(this, "Opción 1.1 seleccionada");
    }

    private void mnuitOpcion2ActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        JOptionPane.showMessageDialog(this, "Opción 2 seleccionada");
    }

    private void mnuitOpcion3ActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        JOptionPane.showMessageDialog(this, "Opción 3 seleccionada");
    }

    private void mnuitSairActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        System.exit(0);
    }

    private void mnuitrbtHomeItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if(evt.getStateChange()==ItemEvent.SELECTED)
        {
            JOptionPane.showMessageDialog(this, "A opción Home foi seleccionada");
        }
    }

    private void mnuitrbtMullerItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if(evt.getStateChange()==ItemEvent.SELECTED)
        {
            JOptionPane.showMessageDialog(this, "A opción Muller foi seleccionada");
        }
    }

    private void mnuitchkAlemanItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if(mnuitchkAleman.isSelected())
        {
            JOptionPane.showMessageDialog(this, "A opción Alemán foi marcada");
        }
        else
        {
            JOptionPane.showMessageDialog(this, "A opción Alemán foi desmarcada");
        }
    }

    private void mnuitchkFrancesItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if(mnuitchkFrances.isSelected())
        {
            JOptionPane.showMessageDialog(this, "A opción Francés foi marcada");
        }
        else
        {
            JOptionPane.showMessageDialog(this, "A opción Francés foi desmarcada");
        }
    }

    private void mnuitchkInglesItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if(mnuitchkIngles.isSelected())
        {
            JOptionPane.showMessageDialog(this, "A opción Inglés foi marcada");
        }
        else
        {
            JOptionPane.showMessageDialog(this, "A opción Inglés foi desmarcada");
        }
    }

    private void mnuitResumoActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        String sexo="SEXO: ";
        if(mnuitrbtHome.isSelected()) sexo+="Home";
        else sexo+="Muller";
        String idiomas="IDIOMAS: ";
        if(mnuitchkAleman.isSelected()) idiomas+="Alemán ";
        if(mnuitchkFrances.isSelected()) idiomas+="Francés ";
        if(mnuitchkIngles.isSelected()) idiomas+="Inglés ";
        if(idiomas.compareTo("IDIOMAS: ")==0) sexo+="\nIDIOMAS: Ningún";
        else sexo+="\n"+idiomas;
        JOptionPane.showMessageDialog(this, sexo);
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
         * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        }
    }

```

```

    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    }
    //</editor-fold>

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new FrmPrincipal().setVisible(true);
        }
    });
}
// Variables declaration - do not modify
private javax.swing.JMenuBar brmnubarMenu;
private javax.swing.JMenu mnuIdiomas;
private javax.swing.JMenu mnuOpcion1;
private javax.swing.JMenu mnuOpcion1_2;
private javax.swing.JMenu mnuOpciones;
private javax.swing.JMenu mnuSexo;
private javax.swing.JMenuItem mnuitOpcion1_1;
private javax.swing.JMenuItem mnuitOpcion1_2_1;
private javax.swing.JMenuItem mnuitOpcion1_2_2;
private javax.swing.JMenuItem mnuitOpcion2;
private javax.swing.JMenuItem mnuitOpcion3;
private javax.swing.JMenuItem mnuitResumo;
private javax.swing.JMenuItem mnuitSair;
private javax.swing.JCheckBoxMenuItem mnuitchkAleman;
private javax.swing.JCheckBoxMenuItem mnuitchkFrances;
private javax.swing.JCheckBoxMenuItem mnuitchkIngles;
private javax.swing.JRadioButtonMenuItem mnuitrbtHome;
private javax.swing.JRadioButtonMenuItem mnuitrbtMuller;
private javax.swing.JPopupMenu.Separator mnuspSeparador;
private javax.swing.JPopupMenu.Separator mnuspSeparador2;
private javax.swing.ButtonGroup rbtgrpSexo;
// End of variables declaration
}

```

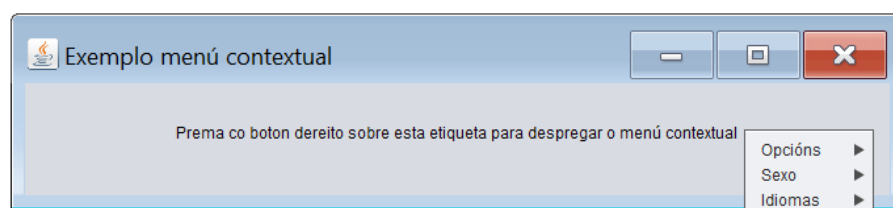
1.2.1.2. Menús contextuais

Un menú contextual é un menú asociado a un compoñente gráfico. Inicialmente non será visualizable. Unicamente cando realicemos algunha acción predeterminada sobre o compoñente gráfico (tipicamente clic dereito) visualizarase o menú e poderemos acceder ás súas opcións. A construción dun menú contextual é moi parecida á dun menú despregable, coa diferenza de que aquí non hai unha barra de menú. A barra de menú é substituída por un obxecto da clase `javax.swing.JPopupMenu`. Este obxecto asocíase ao compoñente gráfico que desprega o menú contextual. As diferentes opcións de menú colgaranse do obxecto `JPopupMenu` asociado ao compoñente gráfico que desprega o menú contextual.

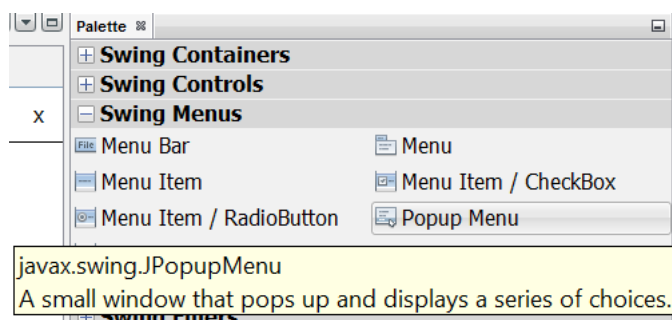
Ademais da clase `JPopupMenu` que substitúe á clase `JMenuBar`, a creación de menús contextuais fai emprego das mesmas clases empregadas para o desenvolvemento dos menús despregables.

Configuración dun menú contextual empregando NetBeans

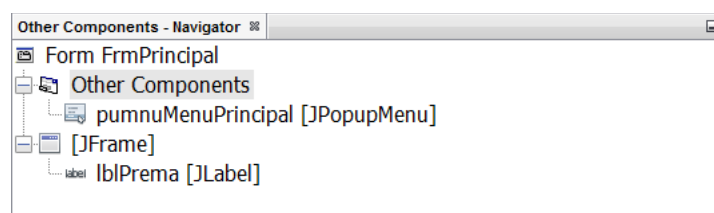
Co fin de explicar como crear un menú contextual empregando NetBeans imos desenvolver un menú similar ao desenvolvido para a explicación dos menús despregables. A única diferenza será que o menú contextual despregarase ao facer clic co botón dereito do rato sobre unha etiqueta existente no formulario:



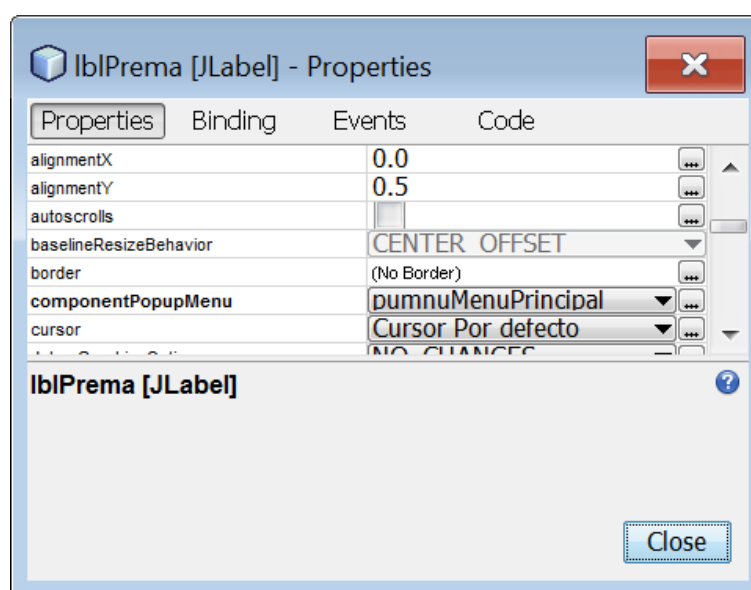
Partindo dun JFrame xa e que contén unha etiqueta, imos a engadir un obxecto de clase JPopupMenu que será o que empregaremos despois para colgar del as diferentes opcións de menú. Para engadir un compoñente de tipo JPopupMenu no noso formulario primeiramente seleccionáremolo da paleta de compoñentes do entorno de programación:



e arrastrámolo ata o formulario. Visualmente non se reflexa nada, pero se miramos atentamente a árbore de obxectos do proxecto, veremos que foi engadido un novo obxecto de clase JPopupMenu. Tras modificar o seu nome interno esta será a configuración da nosa árbore de obxectos:

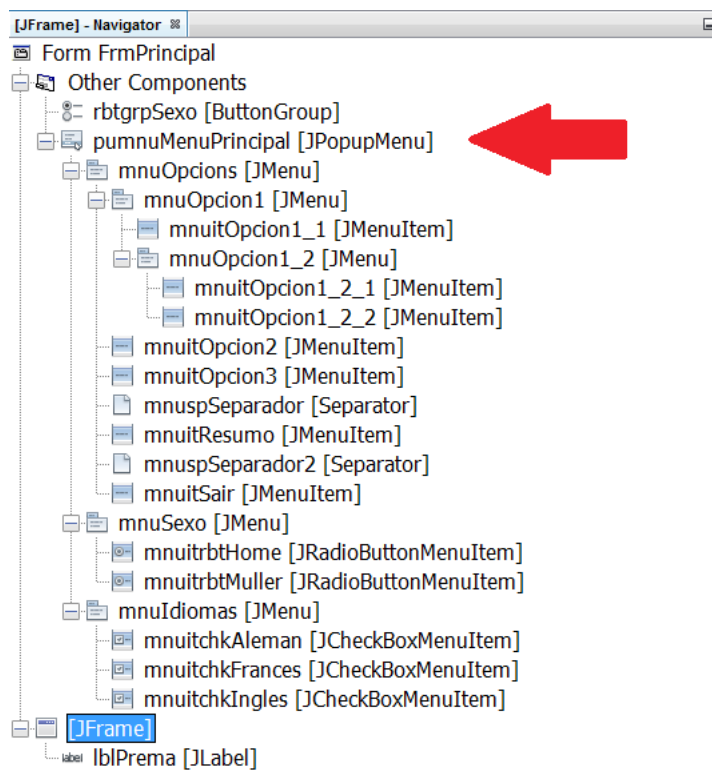


A continuación imos indicar que hai que facer para que cando premamos co botón dereito do rato sobre a etiqueta lblPrema sexa despregado o menú que colga do JPopupMenu pumnuMenuPrincipal. Para elo accedemos ás propiedades da etiqueta e modificamos o valor da súa propiedade componentPopupMenu seleccionando o JPopupMenu do que colga o menú que queremos amosar ao facer clic co botón dereito do rato sobre a etiqueta:



Calquera compoñente que teña a propiedade `componentPopupMenu` ten a capacidade de ter un menú contextual. Nun mesmo formulario non hai limitacións en canto ao número de compoñentes que poden ter un menú contextual.

Agora o único que restaría por facer é construír o menú do mesmo xeito que fixemos para o menú despregable coa diferenza de que os tres primeiros `JMenu`, os que colgaban directamente da barra de menú agora deberán colgar do `JPopupMenu`. Unha vez creados tódolos menús a configuración da árbore de obxectos do proxecto será a seguinte:



Na imaxe se pódese observar que o menú é similar ao creado cando construímos o menú despregable pero coa excepción de que a raíz do menú nesta ocasión é un `JPopupMenu` no lugar dun `JMenuBar`.

Respecto á implementación da funcionalidade, é exactamente igual á desenvolvida para o exemplo do menú despregable.

A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación:

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploMenuContextual;

import java.awt.event.ItemEvent;
import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class FrmPrincipal extends javax.swing.JFrame {

    /**
     * Creates new form FrmPrincipal
     */
    public FrmPrincipal() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
}

```

```

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    rbtgrpSexo = new javax.swing.ButtonGroup();
    pumnuMenuPrincipal = new javax.swing.JPopupMenu();
    mnuOpciones = new javax.swing.JMenu();
    mnuOpcion1 = new javax.swing.JMenu();
    mnuitOpcion1_1 = new javax.swing.JMenuItem();
    mnuOpcion1_2 = new javax.swing.JMenu();
    mnuitOpcion1_2_1 = new javax.swing.JMenuItem();
    mnuitOpcion1_2_2 = new javax.swing.JMenuItem();
    mnuitOpcion2 = new javax.swing.JMenuItem();
    mnuitOpcion3 = new javax.swing.JMenuItem();
    mnuspSeparador = new javax.swing.JPopupMenu.Separator();
    mnuitResumo = new javax.swing.JMenuItem();
    mnuspSeparador2 = new javax.swing.JPopupMenu.Separator();
    mnuitSair = new javax.swing.JMenuItem();
    mnuSexo = new javax.swing.JMenu();
    mnuitrbtHome = new javax.swing.JRadioButtonMenuItem();
    mnuitrbtMuller = new javax.swing.JRadioButtonMenuItem();
    mnuIdiomas = new javax.swing.JMenu();
    mnuitchkAleman = new javax.swing.JCheckBoxMenuItem();
    mnuitchkFrances = new javax.swing.JCheckBoxMenuItem();
    mnuitchkIngles = new javax.swing.JCheckBoxMenuItem();
    lblPrema = new javax.swing.JLabel();

    mnuOpciones.setText("Opciones");

    mnuOpcion1.setText("Opción 1");

    mnuitOpcion1_1.setText("Opción 1.1");
    mnuitOpcion1_1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuitOpcion1_1ActionPerformed(evt);
        }
    });
    mnuOpcion1.add(mnuitOpcion1_1);

    mnuOpcion1_2.setText("Opción 1.2");

    mnuitOpcion1_2_1.setText("Opción 1.2.1");
    mnuitOpcion1_2_1.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuitOpcion1_2_1ActionPerformed(evt);
        }
    });
    mnuOpcion1_2.add(mnuitOpcion1_2_1);

    mnuitOpcion1_2_2.setText("Opción 1.2.2");
    mnuitOpcion1_2_2.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuitOpcion1_2_2ActionPerformed(evt);
        }
    });
    mnuOpcion1_2.add(mnuitOpcion1_2_2);

    mnuOpcion1.add(mnuOpcion1_2);

    mnuOpciones.add(mnuOpcion1);

    mnuitOpcion2.setText("Opción 2");
    mnuitOpcion2.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuitOpcion2ActionPerformed(evt);
        }
    });
    mnuOpciones.add(mnuitOpcion2);

    mnuitOpcion3.setText("Opción 3");
    mnuitOpcion3.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuitOpcion3ActionPerformed(evt);
        }
    });
    mnuOpciones.add(mnuitOpcion3);
    mnuOpciones.add(mnuspSeparador);

    mnuitResumo.setText("Resumo");
    mnuitResumo.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuitResumoActionPerformed(evt);
        }
    });
    mnuOpciones.add(mnuitResumo);
    mnuOpciones.add(mnuspSeparador2);

    mnuitSair.setText("Sair");
    mnuitSair.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuitSairActionPerformed(evt);
        }
    });
    mnuOpciones.add(mnuitSair);

    pumnuMenuPrincipal.add(mnuOpciones);

    mnuSexo.setText("Sexo");

    rbtgrpSexo.add(mnuitrbtHome);
    mnuitrbtHome.setSelected(true);
    mnuitrbtHome.setText("Home");
    mnuitrbtHome.addItemListener(new java.awt.event.ItemListener() {
        public void itemStateChanged(java.awt.event.ItemEvent evt) {
            mnuitrbtHomeItemStateChanged(evt);
        }
    });

```

```

    }
});
mnuSexo.add(mnuitrbtHome);

rbtgrpSexo.add(mnuitrbtMuller);
mnuitrbtMuller.setText("Muller");
mnuitrbtMuller.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitrbtMullerItemStateChanged(evt);
    }
});
mnuSexo.add(mnuitrbtMuller);

pumnuMenuPrincipal.add(mnuSexo);

mnuIdiomas.setText("Idiomas");

mnuitchkAleman.setText("Alemán");
mnuitchkAleman.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitchkAlemanItemStateChanged(evt);
    }
});
mnuIdiomas.add(mnuitchkAleman);

mnuitchkFrances.setText("Francés");
mnuitchkFrances.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitchkFrancesItemStateChanged(evt);
    }
});
mnuIdiomas.add(mnuitchkFrances);

mnuitchkIngles.setText("Inglés");
mnuitchkIngles.addItemListener(new java.awt.event.ItemListener() {
    public void itemStateChanged(java.awt.event.ItemEvent evt) {
        mnuitchkInglesItemStateChanged(evt);
    }
});
mnuIdiomas.add(mnuitchkIngles);

pumnuMenuPrincipal.add(mnuIdiomas);

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
setTitle("Exemplo menú contextual");

lblPrema.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
lblPrema.setText("Prema co boton dereito sobre esta etiqueta para desprezar o menú contextual");
lblPrema.setComponentPopupMenu(pumnuMenuPrincipal);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(lblPrema, javax.swing.GroupLayout.Alignment.TRAILING, javax.swing.GroupLayout.DEFAULT_SIZE,
659, Short.MAX_VALUE)
);
layout.setVerticalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addGap(29, 29, 29)
            .addComponent(lblPrema)
            .addGap(36, 36, Short.MAX_VALUE)
        )
);

java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
setBounds((screenSize.width-681)/2, (screenSize.height-149)/2, 681, 149);
} // </editor-fold>

private void mnuitOpcion1_2_1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "Opción 1.2.1 seleccionada");
}

private void mnuitOpcion1_2_2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "Opción 1.2.2 seleccionada");
}

private void mnuitOpcion1_1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "Opción 1.1 seleccionada");
}

private void mnuitOpcion2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "Opción 2 seleccionada");
}

private void mnuitOpcion3ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "Opción 3 seleccionada");
}

private void mnuitSairActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    System.exit(0);
}

private void mnuitrbtHomeItemStateChanged(java.awt.event.ItemEvent evt) {
    // TODO add your handling code here:
    if (evt.getStateChange() == ItemEvent.SELECTED)
    {
        JOptionPane.showMessageDialog(this, "A opción Home foi seleccionada");
    }
}

```

```

    }

    private void mnuitrbrtMullerItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if (evt.getStateChange() == ItemEvent.SELECTED)
        {
            JOptionPane.showMessageDialog(this, "A opción Muller foi seleccionada");
        }
    }

    private void mnuitchkAlemanItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if (mnuitchkAleman.isSelected())
        {
            JOptionPane.showMessageDialog(this, "A opción Alemán foi marcada");
        }
        else
        {
            JOptionPane.showMessageDialog(this, "A opción Alemán foi desmarcada");
        }
    }

    private void mnuitchkFrancesItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if (mnuitchkFrances.isSelected())
        {
            JOptionPane.showMessageDialog(this, "A opción Francés foi marcada");
        }
        else
        {
            JOptionPane.showMessageDialog(this, "A opción Francés foi desmarcada");
        }
    }

    private void mnuitchkInglesItemStateChanged(java.awt.event.ItemEvent evt) {
        // TODO add your handling code here:
        if (mnuitchkIngles.isSelected())
        {
            JOptionPane.showMessageDialog(this, "A opción Inglés foi marcada");
        }
        else
        {
            JOptionPane.showMessageDialog(this, "A opción Inglés foi desmarcada");
        }
    }

    private void mnuitResumoActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        String sexo="SEXO: ";
        if (mnuitrbrtHome.isSelected()) sexo+="Home";
        else sexo+="Muller";
        String idiomas="IDIOMAS: ";
        if (mnuitchkAleman.isSelected()) idiomas+="Alemán ";
        if (mnuitchkFrances.isSelected()) idiomas+="Francés ";
        if (mnuitchkIngles.isSelected()) idiomas+="Inglés ";
        if (idiomas.compareTo("IDIOMAS: ") == 0) sexo+="\nIDIOMAS: Ningún";
        else sexo+="\n"+idiomas;
        JOptionPane.showMessageDialog(this, sexo);
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
         * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (InstantiationException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (IllegalAccessException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        }
        //</editor-fold>

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new FrmPrincipal().setVisible(true);
            }
        });
    }
    // Variables declaration - do not modify
    private javax.swing.JLabel lblPrema;
    private javax.swing.JMenu mnuIdiomas;
    private javax.swing.JMenu mnuOpcion1;
    private javax.swing.JMenu mnuOpcion1_2;
    private javax.swing.JMenu mnuOpciones;
    private javax.swing.JMenu mnuSexo;

```

```

private javax.swing.JMenuItem mnuitOpcion1_1;
private javax.swing.JMenuItem mnuitOpcion1_2_1;
private javax.swing.JMenuItem mnuitOpcion1_2_2;
private javax.swing.JMenuItem mnuitOpcion2;
private javax.swing.JMenuItem mnuitOpcion3;
private javax.swing.JMenuItem mnuitResumo;
private javax.swing.JMenuItem mnuitSair;
private javax.swing.JCheckBoxMenuItem mnuitchkAleman;
private javax.swing.JCheckBoxMenuItem mnuitchkFrances;
private javax.swing.JCheckBoxMenuItem mnuitchkIngles;
private javax.swing.JRadioButtonMenuItem mnuitrbtHome;
private javax.swing.JRadioButtonMenuItem mnuitrbtMuller;
private javax.swing.JPopupMenu.Separator mnuspSeparador;
private javax.swing.JPopupMenu.Separator mnuspSeparador2;
private javax.swing.JPopupMenu pumnuMenuPrincipal;
private javax.swing.ButtonGroup rbtgrpSexo;
// End of variables declaration
}

```

1.2.2. Aplicacións multixanela

As aplicacións desenvolvidas ata o momento foron sempre aplicacións cuxa acción desenrolábase nunha única xanela. Isto adoita ocorrer en aplicacións de pouca entidade, pero unha vez que unha aplicación comenza a ter certa complexidade o habitual é que o seu contido teña que desdoblarse en varios formularios. Cando o número de formularios da aplicación comenza a ser elevado á recomendable separalos en distintas xanelas co fin de que o manexo da aplicación sexa máis sinxelo. Unha interface de usuario demasiado cargada vólvese lenta e complexa de empregar. Para desenvolver aplicacións multixanela java ofrécenos dúas solucións:

- Aplicacións multixanela Dialog
- Aplicacións multixanela MDI.

1.2.2.1. Aplicacións multixanela Dialog

Para desenvolver aplicacións multixanela de tipo Dialog empregamos dous tipos de obxectos:

- O obxecto JFrame para representar a xanela principal (a primeira xanela que se crea).
- O obxecto JDialog para o resto de xanelas da aplicación.

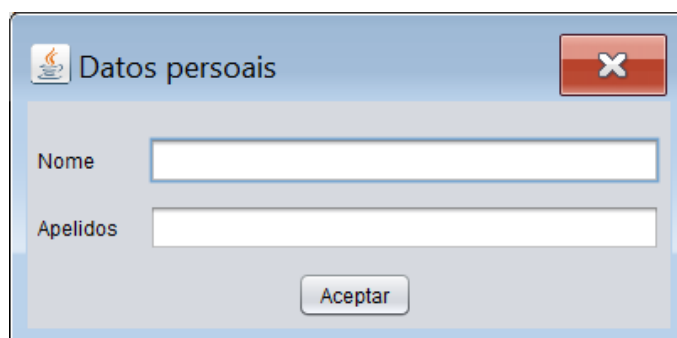
Pero, ¿porqué non desenvolver as aplicacións multixanela empregando unicamente obxectos da clase JFrame?. Ben, tecnicamente pódese e ademais non presenta ningunha complicación engadida a facelo empregando JFrame e JDialogs, pero presenta algúns problemas:

Un JFrame representa a unha xanela “principal”. Cando está aberta, na barra de tarefas do sistema operativo temos unha referencia gráfica (tipicamente unha icona) a esa xanela. Supoñamos que temos unha aplicación multixanela con catro JFrames. Na barra de tarefas do sistema operativo teremos catro referencias gráficas, unha por cada unha das xanelas que temos abertas na nosa aplicación. Pola contra, un JDialog representa a unha xanela “secundaria”. Cando está aberta non aparece na barra de tarefas do sistema operativo ningunha referencia gráfica para poder xestionala. Polo tanto se desenvolvemos as nosas aplicacións multixanela empregando un JFrame para a xanela principal e obxectos JDialog para as xanelas secundarias, o aspecto das nosas aplicacións será o habitual, é dicir, unicamente haberá na barra de tarefas do sistemas operativo unha referencia gráfica cara á nosa aplicación multixanela.

Un JFrame, ao ser unha xanela principal, non permite ter unha xanela pai. O JDialog ao ser unha xanela secundaria si admite pai (o pai dunha xanela é o que a crea. Dise que a xanela aberta é filla de quen a creou. Os pais dun JDialog serán de tipo JFrame ou JDialog). Unha xanela filla graficamente sempre situarase por encima da súa xanela pai. A xanela filla nunca poderá quedar agochada detrás da xanela pai. Conceptualmente, unha xanela pai abre unha xanela filla para que o usuario realice algunha tarefa en ela. O non permitir que esa

xanela quede agochada detrás do seu pai é un xeito de lembrar ao usuario que ten que realizar esa tarefa.

A clase JDialog defínese a través da clase javax.swing.JDialog. Gráficamente o seu aspecto é o seguinte:



Como pódese observar na imaxe anterior, apenas hai diferencias entre un JDialog e un JFrame. Ambas as dúas son xanelas nas cales podemos colocar os compoñentes que consideremos oportunos para desenvolver as nosas aplicacións. Visualmente a diferenza máis notable é que os JDialog non teñen botóns de redimensionamento (maximizar e minimizar).

Propiedades do contedor JDialog empregadas máis habitualmente e que pódense establecer a través do entorno de programación:

Propiedade	Función
DefaultCloseOperation	Mediante esta propiedade indícase cal será o comportamento da xanela ao pechala. Pode tomar 3 valores posibles:DISPOSE, HIDE o DO_NOTHING. Con DISPOSE péchase a xanela liberando os recursos consumidos pola mesma. Con HIDE a xanela unicamente será escondida. Con DO_NOTHING non se fará nada.
Modal	Mediante esta propiedade indícase se o comportamento da xanela será modal ou non.
Resizable	Indica se a xanela será ou non redimensionable.
Title	Título da xanela.
Type	Indica o tipo de xanela que imos crear. Pode tomar os valores NORMAL, UTILITY o POPUP. NORMAL indica unha xanela normal. Se empregamos os valores UTILITY ou POPUP, dependendo do sistema operativo sobre o que corra a nosa aplicación podería variar lixeiramente o aspecto da xanela.

Eventos do contedor JDialog empregados máis habitualmente e que pódense establecer a través do entorno de programación:

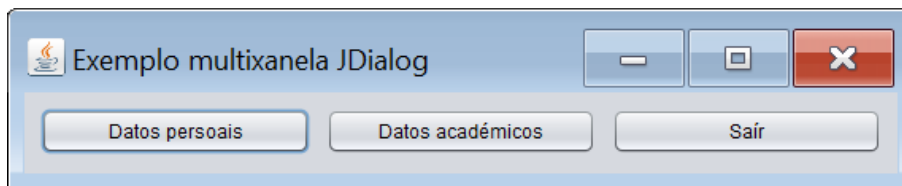
Evento	Lanzamento
WindowClosed	Este evento é disparado cando pechamos a xanela xa sexa ao premer sobre a aspa da xanela ou ao pechala a través de código.

Métodos do contedor JDialog empregados máis habitualmente:

Método	Función
public void dispose()	Mediante este método pódese pechar a xanela liberando os recursos consumidos pola mesma.
public void setVisible(boolean b)	Este método emprégase para facer visible ou invisible a xanela de diálogo.

Configuración dunha aplicación multixanela Dialog empregando NetBeans

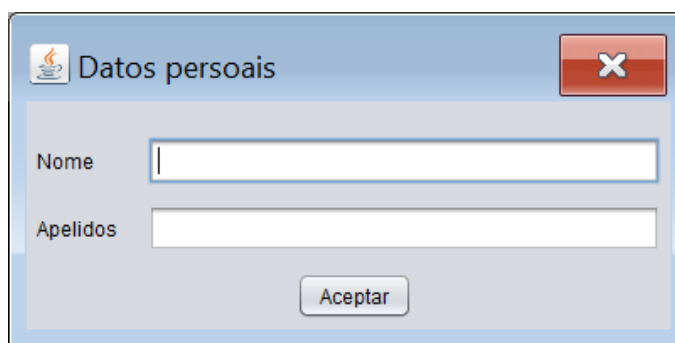
Imos desenvolver unha aplicación cuxa xanela principal (JFrame) será a seguinte:



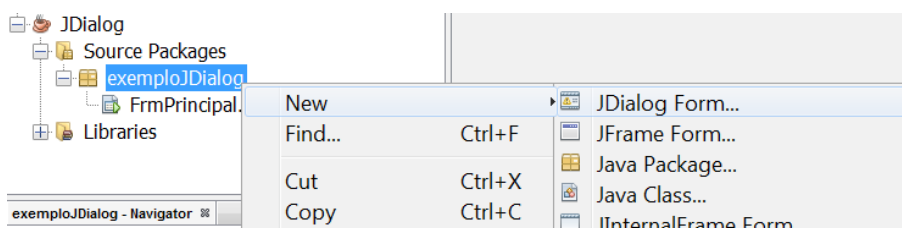
Ao premer sobre o botón Datos persoais abrírase un JDialog para introducir os datos persoais do usuario. Ao pulsar sobre o botón Datos académicos abrírase un JDialog para introducir os datos académicos do usuario. O botón Saír finalizará a execución da aplicación. A medida que avance a explicación imos ir complicando os requisitos do programa para desenvolver novas funcionalidades.

Sobre o deseño da xanela principal non hai nada que dicir. É un formulario con tres botóns.

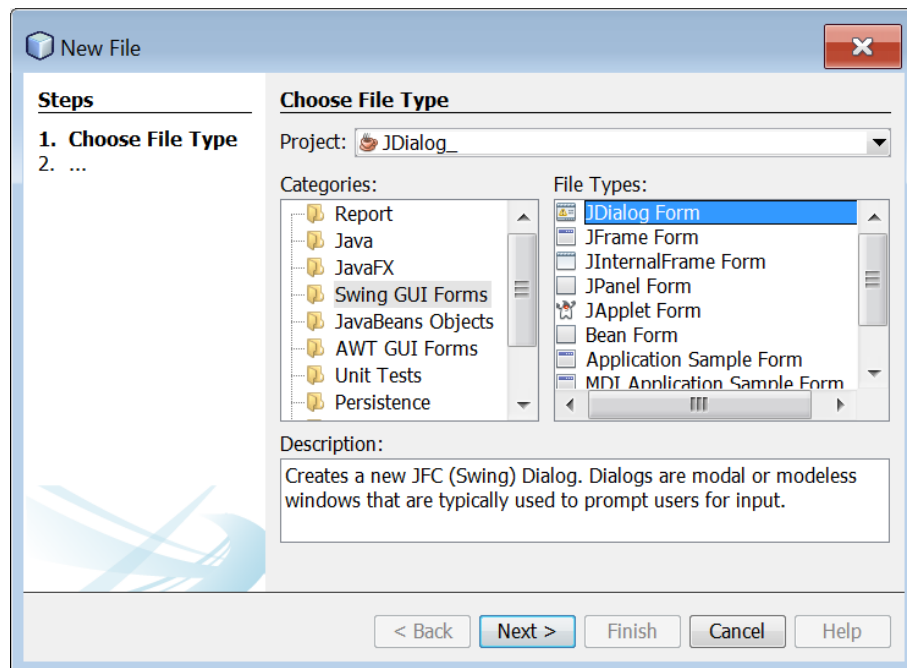
Cando pulsamos sobre o botón Datos persoais débese abrir un JDialog que ten o seguinte aspecto:



Para crear este JDialog debemos facer o seguinte: o primeiro é crear unha clase personalizada que herde de JDialog para definir o noso contedor. Para elo, prememos co botón dereito do rato sobre o paquete no cal queremos crear a nosa clase personalizada. Sobre o menú despregado eliximos New e sobre o novo menú despregado eliximos JDialog Form:

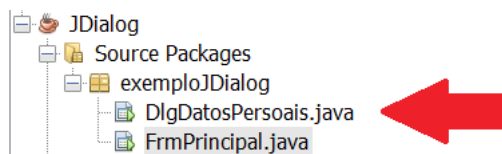


É posible que a primeira vez que creamos un JDialog, ao pulsar na opción New non apareza a opción JDialog Form. Nese caso, no mesmo menú deberemos elixir unha opción etiquetada como Other. Ao seleccionala abríranos a seguinte pantalla:

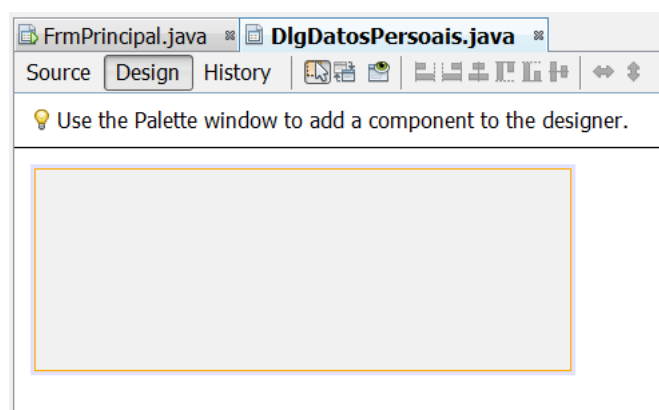


Esta pantalla emprégase para crear clases baseándose nos diferentes modelos que ofrece o entorno de programación. No caso que estamos desenvolvendo elixiremos dentro de Categories a opción Swing GUI Forms e en File Types JDialog Form.

Independentemente de como cheguemos a ela, ao seleccionar a opción JDialog Form saltará un asistente similar ao de creación dun novo formulario para indicar o nome que lle imos dar á nosa nova clase de tipo JDialog (chamarémola DlgDatosPersoais) e opcionalmente para indicar en que paquete queremos almacenala. Unha vez que completamos o asistente, na nosa árbore de proxecto imos ter unha nova clase que representara ao JDialog que acabamos de crear:

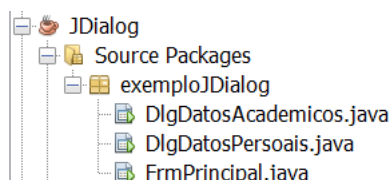


Respecto ao deseño gráfico do JDialog, lévase a cabo do mesmo xeito que para un formulario de tipo JFrame. Se seleccionamos a clase que acabamos de crear (DlgDatosPersoais), na xanela de deseño do contedor veremos o seguinte:



Como pódese observar é un contedor baleiro. Sobre el estableceremos as súas propiedades, inseriremos os compoñentes, estableceremos os layouts necesarios e escribiremos o código necesario para os diferentes eventos que queiramos xestionar sobre os compoñentes do JDialog. Resumindo, unha vez creado o noso JDialog personalizado, xestionarémolo tal e como fixemos ata agora cos nosos formularios. Polo tanto, o deseño visual do dialogo DlgDatosPersoais quedaría configurado do seguinte xeito:

Una vez desenvolvido o diálogo que se amosará cando premamos o botón Datos persoais, facemos o mesmo para o dialogo que se amosará ao premer o botón Datos académicos. O resultado será o seguinte:



Como pódese observar na imaxe anterior, a clase que acabamos de crear como modelo para os obxectos que se amosaran cando premamos sobre o botón Datos académicos chámase DlgDatosAcademicos. O aspecto do seu deseño gráfico é o seguinte:

Non nos imos centrar no que facen os diálogos que acabamos de crear, senón unicamente no código relacionado coa xestión muxixanela da aplicación.

De seguido imos ver que é o que temos que facer para que se amosen os diálogos ao premer sobre os botóns correspondentes. Para amosar o diálogo DlgDatosPersoais implementamos o actionPerformed do botón Datos persoais, sendo o código resultante o seguinte:

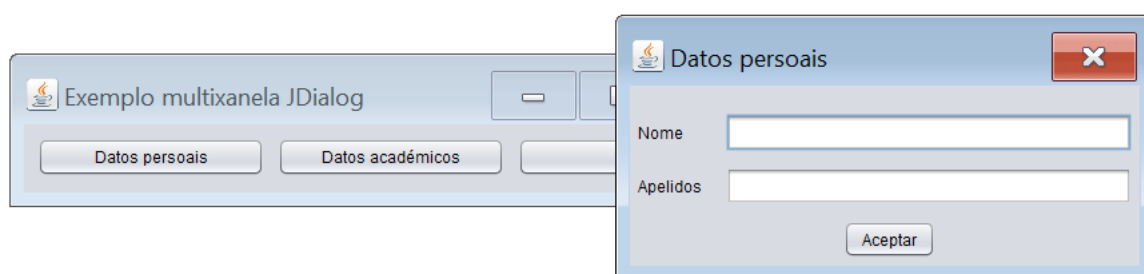
```
private void btnDatosPersoaisActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    DlgDatosPersoais dlgDatosPersoais=new DlgDatosPersoais(this, false);
    dlgDatosPersoais.setVisible(true);
}
```

Na primeira liña o que facemos é instanciar un obxecto da clase DlgDatosPersoais empregando o seu construtor. O construtor ten dous parámetros. O primeiro parámetro emprégase para indicar quen é a xanela pai da que estamos creando (neste caso é o propio chamador, é dicir, this). O segundo parámetro indica se a xanela aberta será amosada en

modo modal (true) ou non modal (false). Unha xanela aberta en modo modal bloquea ao resto da aplicación. Mentres unha xanela está aberta en modo modal non se pode acceder ao resto de xanelas que compoñen a aplicación. Cando sexa pechada será posible acceder a calquera outra xanela das que compoñen a aplicación. Sóense empregar as xanelas en modo modal cando queremos chamar a atención do usuario por algunha razón determinada (datos requiridos, mensaxes de erro, ...). Fixémonos que o obxecto creado (que será unha xanela de tipo diálogo non modal e filla do formulario principal) será xestionada a través da variable local `dlgDatosPersoais`. Neste momento o diálogo está creado en memoria (fíxose o new) pero non é visible.

Na segunda liña facemos visible ao diálogo que acabamos de crear en memoria. Empregando a súa referencia, aplicamos sobre ela o método `setVisible(true)` que o que fai é facer visible ao contedor. Se lle pasamos o valor false o que fariamos sería facer invisible ao contedor.

Se executamos a aplicación e prememos sobre o botón Datos persoais este será o resultado:

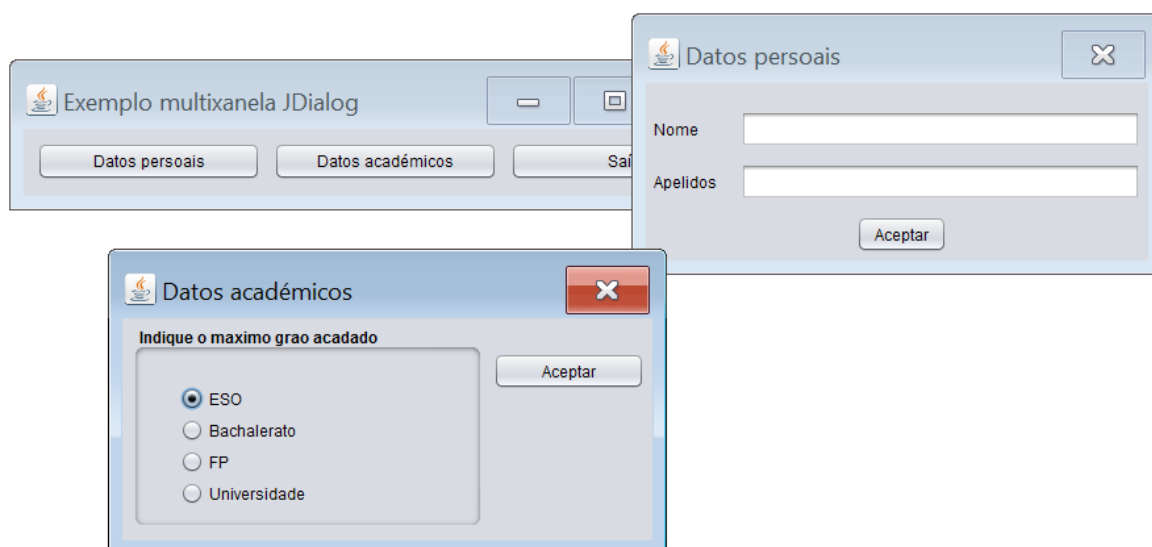


Tal e como dixemos con anterioridade, no caso de superpoñer a xanela filla sobre a pai, esta última sempre estará por debaixo da filla (aínda que a seleccionemos explicitamente).

Para a resolución do botón Datos académicos escribimos un código con una funcionalidade similar:

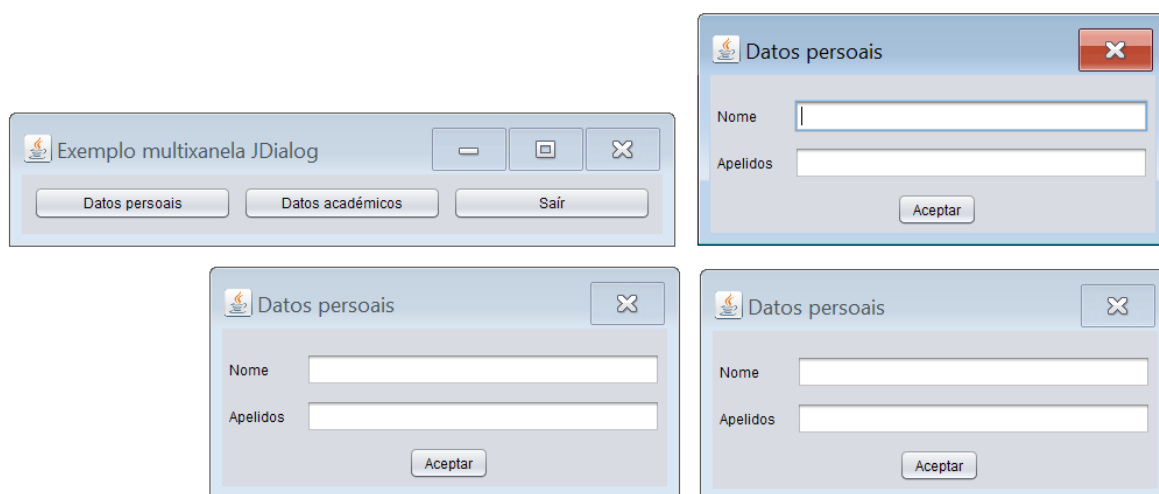
```
private void btnDatosAcademicosActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    DlgDatosAcademicos dlgDatosAcademicos=new DlgDatosAcademicos(this, false);  
    dlgDatosAcademicos.setVisible(true);  
}
```

sendo o resultado de premer ambos botóns o seguinte:



Como podemos ver a xestión básica dunha aplicación multixanela en java e bastante sinxela.

Pero, co código actual, ¿que ocorre se prememos un botón varias veces?, p.e., imaxinemos que prememos tres veces o botón de Datos persoais. Ocorrerá o seguinte:



Como era de esperar, cada vez que prememos o botón ábrese un novo diálogo. Se os requirimentos do noso programa permítennos facelo entón non temos ningún problema, pero supoñamos que os requirimentos do noso programa dinnos que só podemos ter aberta á vez unha xanela de tipo `dlgDatosPersoais` e como moito dous de tipo `DlgDatosAcademicos`. Neste caso necesitaremos algún sistema a modo de xestor de xanelas para coñecer en todo momento que xanelas temos abertas e poder controlar se podemos abrir ou non podemos abrir máis. Solucións para resolver este problema hai moitas. Unha relativamente sinxela é o emprego dunha clase que almacene o estado das distintas xanelas que temos que controlar. Partindo do suposto que acabamos de propoñer (só podemos ter aberta á vez unha xanela de tipo `dlgDatosPersoais` e como moito dous de tipo `DlgDatosAcademicos`) imos desenvolver esta solución. Para elo creamos no noso proxecto unha nova clase que se chamará `XestorDeXanelas`. O seu código será o seguinte:

```
/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

/**
 *
 * @author German DR
 */
public class XestorDeXanelas {
    static private int numXanelasDatosPersoais=0;
    static private int numXanelasDatosAcademicos=0;

    public static void cerrarXanelasDatosPersoais()
    {
        numXanelasDatosPersoais--;
    }
    public static boolean abrirXanelasDatosPersoais()
    {
        if(numXanelasDatosPersoais<1)
        {
            numXanelasDatosPersoais++;
            return true;
        }
        else
        {
            return false;
        }
    }

    public static void cerrarXanelasDatosAcademicos()
    {
        numXanelasDatosAcademicos--;
    }

    public static boolean abrirXanelasDatosAcademicos()
    {
        if(numXanelasDatosAcademicos<2)
        {
            numXanelasDatosAcademicos++;
            return true;
        }
        else
        {
            return false;
        }
    }
}
```

```

        {
            numXanelasDatosAcademicos++;
            return true;
        }
        else
        {
            return false;
        }
    }
}

```

Esta clase ten dous atributos chamados numXanelasDatosPersoais e numXanelasDatosAcademicos, ambas de tipo enteiro que empregamos para levar a conta de cantos diálogos de datos persoais e de datos académicos temos abertos en cada momento. Para o atributo numXanelasDatosPersoais hai un método abrirXanelaDatosPersoais que é chamado cada vez que prememos sobre o botón Datos persoais. Este método devolve true se é posible abrir un diálogo de tipo DlgDatosPersoais (se o contador numXanelasDatosPersoais é menor de 1) e ademais incrementa o número de xanelas abertas nunha unidade. No caso contrario, se xa hai algunha xanela de tipo DlgDatosPersoais aberta devolverá false. Para poder empregar este método debemos modificar o código do botón Datos persoais do seguinte xeito:

```

private void btnDatosPersoaisActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    if (XestorDeXanelas.abrirXanelasDatosPersoais())
    {
        DlgDatosPersoais dlgDatosPersoais=new DlgDatosPersoais(this, false);
        dlgDatosPersoais.setVisible(true);
    }
    else
    {
        JOptionPane.showMessageDialog(this, "Non é posible abrir máis xanelas deste tipo");
        return;
    }
}

```

Como pódese observar, o código de creación do diálogo de clase DlgDatosPersoais é o mesmo de antes, pero antes de crear a xanela de diálogo compróbase se é posible creala (que non hai xa unha aberta) chamando ao método abrirXanelaDatosPersoais e só é creada se este método llo permite. Se non llo permite, emítese unha mensaxe de erro ao usuario da aplicación. É importante fixarse en que este método, ao igual que tódolos métodos da clase XestorDeXanelas é un método estático, de xeito que as chamadas son máis sinxelas ao non ter que instanciar ningún obxecto de tipo XestorDeXanelas (outra opción para facilitar e unificar o código sería introducir a xestión de xanelas no propio código da xanela que abre os diálogos, é dicir, no formulario pai).

Para poder controlar totalmente o número de xanelas abertas de clase DlgDatosPersoais, ao igual que incrementamos o seu contador cada vez que abrimos unha xanela, tamén deberemos decrementar o seu contador cada vez que pechemos unha xanela. Cando péchase un dialogo é disparado o seu evento WindowClosed. Polo tanto, este será o lugar axeitado para informar ao xestor de xanelas de que acabamos de pechar unha xanela de tipo DlgDatosPersoais. Para elo escribimos as seguintes liñas de código:

```

private void formWindowClosed(java.awt.event.WindowEvent evt) {
    // TODO add your handling code here:
    XestorDeXanelas.cerrarXanelasDatosPersoais();
}

```

O que facemos é simplemente chamar ao método cerrarXanelasDatosPersoais, o cal decrementa o número de diálogos de tipo DlgDatosPersoais abertos nunha unidade.

De igual xeito que acabamos de facer co botón Datos persoais, modificamos o botón Datos académicos para xestionar a creación dun novo diálogo de clase DlgDatosAcademicos:

```

private void btnDatosAcademicosActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    if (XestorDeXanelas.abrirXanelasDatosAcademicos())
    {
        DlgDatosAcademicos dlgDatosAcademicos=new DlgDatosAcademicos(this, false);
        dlgDatosAcademicos.setVisible(true);
    }
    else
    {
        JOptionPane.showMessageDialog(this, "Non é posible abrir máis xanelas deste tipo");
    }
}

```

```

    }
    return;
}

```

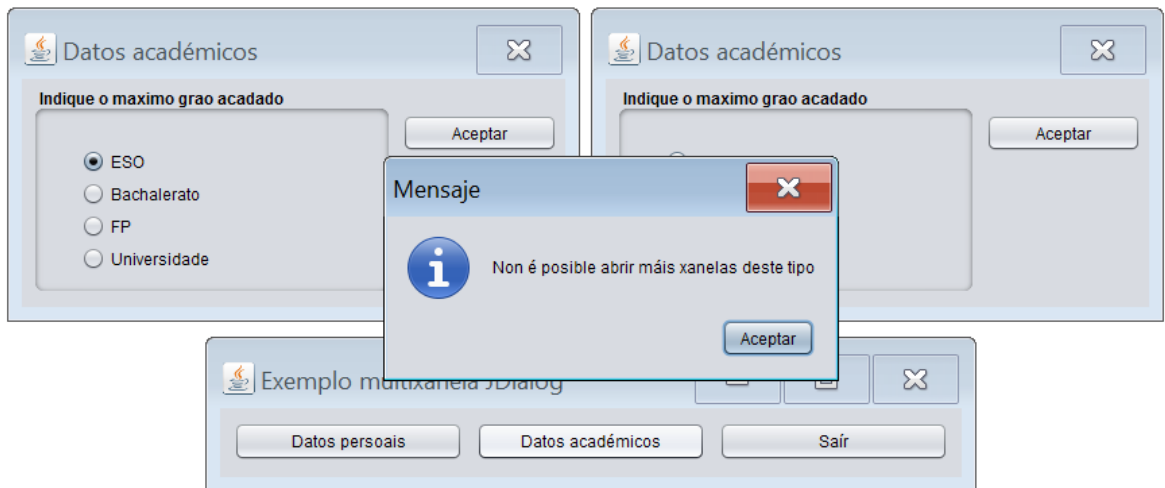
Por outra parte deberemos implementar o evento `WindowClosed` do diálogo `DlgDatosAcademicos` para decrementar o número de diálogos abertos cando un deles é pechado:

```

private void formWindowClosed(java.awt.event.WindowEvent evt) {
    // TODO add your handling code here:
    XestorDeXanelas.cerrarXanelasDatosAcademicos();
}

```

Se agora temos abertos dous diálogos de tipo `DlgDatosAcademicos` e intentamos abrir outro máis ocorre o seguinte:



O xestor de xanelas leva a conta do número de xanelas abertas de cada tipo e impídenos superar o límite máximo para cada uno dos tipos de xanelas implicados na nosa aplicación.

Esta é unha das utilidades que nos proporciona un xestor de xanelas. Outra utilidade que lle poderíamos dar ao xestor de xanelas sería a de ter unha referencia a cada unha das xanelas fillas dunha xanela pai. Unha vez que temos unha referencia desde unha xanela á súas fillas, a xanela pai poderá interactuar con todas elas e facer emprego dos métodos dispoñibles nas xanelas fillas. Imos desenvolver un exemplo sinxelo para practicar esta ultima utilidade que acabamos de comentar. Imos modificar a xanela pai para que o seu aspecto sexa o seguinte:



Como pódese observar hai un novo botón chamado `Cascada`. A utilidade deste botón é poñer en cascada as xanelas de tipo `DlgDatosPersoais` abertas mediante o botón `Datos Persoais`. Tal e como temos configurado agora mesmo o código da aplicación, cada vez que abrimos un diálogo, a variable que empregamos para crealo e unha variable local que se perde:

```

private void btnDatosPersoaisActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    if (XestorDeXanelas.abrirXanelasDatosPersoais())
    {
        DlgDatosPersoais dlgDatosPersoais=new DlgDatosPersoais(this, false);
    }
}

```

```

        dlgDatosPersoais.setVisible(true);
    }
    else
    {
        JOptionPane.showMessageDialog(this, "Non é posible abrir máis xanelas deste tipo");
        return;
    }
}

```

A variable `dlgDatosPersoais` é a referencia ao novo diálogo creado, pero como é unha variable local, perdese ao terminarse de executarse o método e polo tanto o noso código fonte xa non pode interactuar de novo coa xanela que acabamos de crear. Unha solución é crear no noso xestor de xanelas un almacén no cal gardaremos as referencias das xanelas que imos creando. Para o caso que estamos probando engadimos o seguinte atributo:

```

static private Vector xanelasDatosPersoais=new Vector();

```

e os seguintes métodos:

```

public static void engadirXanelaDatosPersoais(DlgDatosPersoais xanela)
{
    xanelasDatosPersoais.add(xanela);
}

public static void eliminarXanelaDatosPersoais(DlgDatosPersoais xanela)
{
    for(int i=0;i<xanelasDatosPersoais.size();i++)
    {
        if(xanelasDatosPersoais.elementAt(i)==xanela)
        {
            xanelasDatosPersoais.removeElementAt(i);
            break;
        }
    }
    System.out.println(xanelasDatosPersoais.size());
}

public static Vector recuperarXanelasDatosPersoais()
{
    return xanelasDatosPersoais;
}

```

- O atributo `xanelasDatosPersoais` é un `Vector` no cal almacenaremos as referencias a tódolos novos diálogos de clase `dlgDatosPersoais` que imos engadindo á nosa aplicación.
- O método `engadirXanelaDatosPersoais` emprégase para insertar no `Vector` `xanelasDatosPersoais` unha referencia a un diálogo de clase `dlgDatosPersoais`. Será invocado cada vez que engadamos un diálogo de clase `dlgDatosPersoais` á nosa aplicación.
- O método `eliminarXanelaDatosPersoais` emprégase para eliminar do `Vector` `xanelasDatosPersoais` unha referencia a un diálogo de clase `dlgDatosPersoais`. Será invocado cada vez que eliminemos un diálogo de clase `dlgDatosPersoais` da nosa aplicación.
- O método `recuperarXanelasDatosPersoais` emprégase para devolver o contido do `Vector` `xanelasDatosPersoais`. Empregando este método pódense acadar tódalas referencias dos diálogos de clase `dlgDatosPersoais` da nosa aplicación.

Con estes tres elementos xestionamos tódalas referencias de calquera elemento de clase `dlgDatosPersoais` que teñamos na nosa aplicación.

Ademais imos modificar o método `abrirXanelasDatosPersoais` para que nos permita abrir ata cinco xanelas de tipo `DlgDatosPersoais`:

```

public static boolean abrirXanelasDatosPersoais()
{
    if(numXanelasDatosPersoais<5)
    {
        numXanelasDatosPersoais++;
        return true;
    }
    else
    {
        return false;
    }
}

```

Agora deberemos modificar o noso código de xeito que cada vez que engadamos un diálogo de tipo `dlgDatosPersoais`, sexa engadida a súa referencia tamén ao `Vector xanelasDatosPersoais` do xestor de xanelas, é dicir, chamaremos ao método `engadirXanelaDatosPersonais` pasándolle a xanela de diálogo que acabamos de crear:

```
private void btnDatosPersoaisActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    if (XestorDeXanelas.abrirXanelasDatosPersoais())
    {
        DlgDatosPersoais dlgDatosPersoais=new DlgDatosPersoais(this, false);
        XestorDeXanelas.engadirXanelaDatosPersoais(dlgDatosPersoais);
        dlgDatosPersoais.setVisible(true);
    }
    else
    {
        JOptionPane.showMessageDialog(this, "Non é posible abrir máis xanelas deste tipo");
        return;
    }
}
```

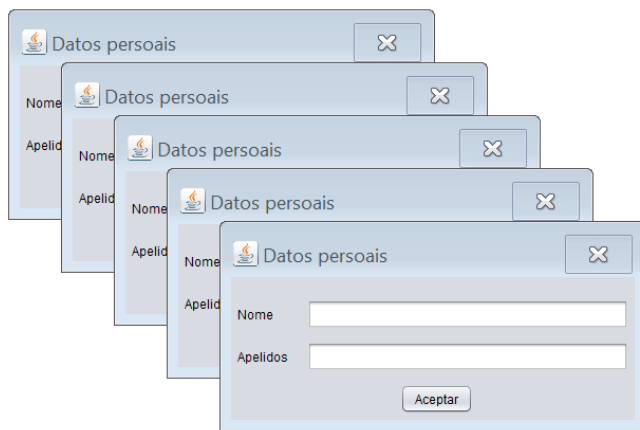
Tamén deberemos modificar o noso código de xeito que cada vez que eliminemos un diálogo de tipo `dlgDatosPersoais` sexa eliminada tamén a súa referencia do `Vector xanelasDatosPersoais` do xestor de xanelas, é dicir, chamaremos ao método `eliminarXanelaDatosPersonais` pasándolle a xanela de diálogo que queremos eliminar do vector:

```
private void formWindowClosed(java.awt.event.WindowEvent evt) {
    // TODO add your handling code here:
    XestorDeXanelas.eliminarXanelaDatosPersoais(this);
    XestorDeXanelas.cerrarXanelasDatosPersoais();
}
```

Unha vez que temos tódalas pezas, unicamente resta por crear o método que sitúa en cascada tódalas xanelas de tipo `dlgDatosPersoais`. Para elo, no formulario pai creamos o seguinte método:

```
public void xanelasDatosPersoaisEnCascada()
{
    Vector xanelasDatosPersoais=XestorDeXanelas.recuperarXanelasDatosPersoais();
    int posX=10, posY=10, incremento=50;
    for(int i=0;i<xanelasDatosPersoais.size();i++)
    {
        DlgDatosPersoais xanela=(DlgDatosPersoais)xanelasDatosPersoais.elementAt(i);
        xanela.setLocation(posX, posY);
        posX+=incremento;
        posY+=incremento;
    }
}
```

Este método será chamado cando fagamos clic sobre o botón Cascada. Este método o primeiro que fai é recuperar do vector `xanelasDatosPersoais` do xestor de xanelas as referencias a tódalas xanelas de clase `dlgDatosPersoais`. Posteriormente recupera un a un os elementos dese vector, é dicir, recupera un a un os diálogos de clase `dlgDatosPersoais` e a cada un deles aplícalle o seu método `setLocation` que serve para fixar a posición dun contedor dando a súa posición superior esquerda. Para cada uno deles recalculáanse esas posicións de xeito que a disposición final dos diálogos sexa unha disposición en cascada. A continuación pódese ver a execución da aplicación sobre cinco diálogos de clase `dlgDatosPersoais`:



Se unha xanela pai ten referenciadas ás súas xanelas fillas, poderá realizar sobre elas calquera acción que estas permitan a través dos seus métodos públicos. O exemplo que acabamos de ver é unha sinxela demostración disto.

Acabamos de ver como facer que unha xanela pai teña referencias a tódalas súas fillas a través do emprego dun xestor de xanelas. Agora imos ver o caso contrario, é dicir, como facer que unha xanela filla teña unha referencia á súa xanela pai. Unha vez que unha xanela filla ten unha referencia á súa xanela pai, poderá invocar calquera método público da mesma. Para ver este concepto imos modificar a aplicación para que faga o seguinte: cando enchemos os datos dun formulario de clase `dlgDatosPersoais` no caso de que cometamos algún erro (non indiquemos o nome ou os apelidos) amosarase un erro ao usuario, pero o encargado de amosar o erro non vai ser o `dlgDatosPersoais` senón que será o formulario pai. Polo tanto, o dialogo `dlgDatosPersoais` terá que ter algunha referencia á xanela pai para acceder ao método desta que amosa as mensaxes de erro. É moi sinxelo. No pai creamos un método público que se encargará de xestionar as mensaxes de erro. Este será o seu código:

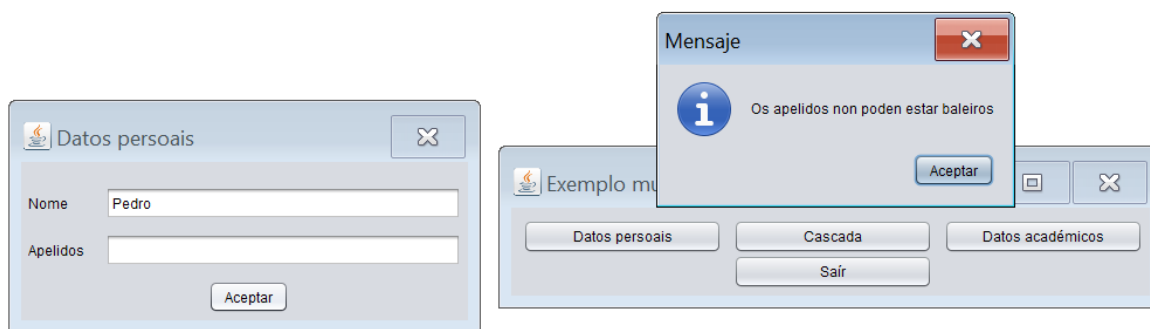
```
public void xestionDeMensaxesDeErro(int numErro)
{
    switch(numErro)
    {
        case 1: JOptionPane.showMessageDialog(this, "O nome non pode estar baleiro");
                break;
        case 2: JOptionPane.showMessageDialog(this, "Os apelidos non poden estar baleiros");
                break;
    }
}
```

Agora no botón Aceptar do diálogo de clase `dlgDatosPersoais` escribimos as seguintes liñas:

```
private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    if(txtNome.getText().trim().compareTo("")==0)
    {
        ((FrmPrincipal)getParent()).xestionDeMensaxesDeErro(1);
        return;
    }
    if(txtApelidos.getText().trim().compareTo("")==0)
    {
        ((FrmPrincipal)getParent()).xestionDeMensaxesDeErro(2);
        return;
    }
}
```

O método que se encarga de obter a referencia ao pai do diálogo é o método `getParent`. Este método devólvenos un obxecto de tipo `java.awt.Container` (un contedor xenérico). O único que temos que facer é castealo á clase que realmente é (un `FrmPrincipal`) e despois empregar os seus métodos públicos.

Na seguinte imaxe podemos ver a chamada ao método público na xanela pai que se encarga de xestionar as mensaxes de erro:



Un xeito de abordar o problema é o que acabamos de ver empregando `getParent`. Outro xeito é empregar os parámetros pasados a través do construtor da xanela filla. Cando o pai crea a xanela filla pásalle a través do construtor desta unha referencia de si mesmo, a cal poderá ser empregada pola xanela filla para acceder aos métodos públicos do seu pai. Imos aplicar esta técnica modificando a aplicación para que faga o seguinte: imos facer que cando premamos no botón aceptar do diálogo de clase `DlgDatosAcademicos` amósese unha mensaxe co tipo de estudos elixidos, pero quen lanzará esa mensaxe non será o diálogo senón que o diálogo chamará a un método público na xanela pai que será o encargado de amosar a mensaxe. Primeiro creamos o método público que se encargará de lanzar as mensaxes na xanela pai. Este será o seu código:

```
public void xestionDeMensaxesDeGraoAcadado(int grao)
{
    switch(grao)
    {
        case 1: JOptionPane.showMessageDialog(this, "O máximo grao que acadou vostede e ESO");
            break;
        case 2: JOptionPane.showMessageDialog(this, "O máximo grao que acadou vostede e Bachalerato");
            break;
        case 3: JOptionPane.showMessageDialog(this, "O máximo grao que acadou vostede e FP");
            break;
        case 4: JOptionPane.showMessageDialog(this, "O máximo grao que acadou vostede e Universidade");
            break;
    }
}
```

Agora imos modificar a clase `DlgDatosAcademicos` para que poida recibir unha referencia da súa xanela pai. Primeiro definimos un atributo na clase no cal almacenaremos a referencia á xanela pai:

```
private FrmPrincipal xanelaPai;
```

Cando teñamos que acceder a un método público da xanela pai farémolo a través da referencia `xanelaPai`.

Agora imos modificar o construtor do diálogo para capturar a referencia ao seu pai que recibe a través do construtor cando é creado:

```
public DlgDatosAcademicos(java.awt.Frame parent, boolean modal) {
    super(parent, modal);
    xanelaPai=(FrmPrincipal)parent;
    initComponents();
}
```

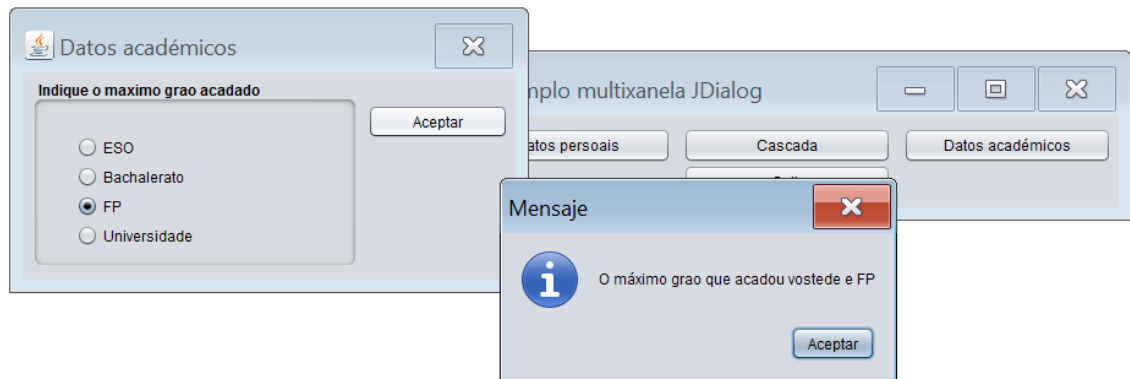
A través do parámetro `parent` do construtor a xanela pai pasa ao diálogo unha referencia de si mesma. No corpo do construtor asignamos esa referencia da xanela pai ao noso atributo de clase `xanelaPai` de xeito que agora a través de `xanelaPai` xa podemos acceder a calquera método público da xanela pai. Fixémonos en que para asignar a referencia da xanela pai recibida como parámetro ao noso atributo de clase `xanelaPai` é necesario casteala, xa que o parámetro recibido no construtor é de clase `java.awt.Frame`, é dicir, un formulario xenérico e non obstante o que nos recibimos realmente é un `FrmPrincipal`.

Agora que temos unha referencia á xanela pai a través do atributo de clase `xanelaPai` unicamente hai que invocar aos métodos públicos deste que necesitamos empregar. No caso que estamos tratando, ao premer sobre o botón Aceptar da xanela de clase `DlgDatosAcademicos` executaremos o seguinte código:

```
private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int grao;
    if(rbtESO.isSelected()) grao=1;
    else if(rbtBachalerato.isSelected()) grao=2;
    else if(rbtFP.isSelected()) grao=3;
    else grao=4;
    xanelaPai.xestionDeMensaxesDeGraoAcadado(graο);
}
```

Como pódese ver no bloque de código anterior, empregando o atributo de clase xanelaPai (o cal referencia á xanela pai) podemos executar o método público da xanela pai xestionDeMensaxesDeGraoAcadado.

Na seguinte imaxe podemos ver a chamada ao método público na xanela pai que se encarga de xestionar as mensaxes de grao acadado:



A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación:

```
FrmPrincipal

package exemploJDialog;

import java.awt.event.ItemEvent;
import java.util.Vector;
import javax.swing.JOptionPane;

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */

/**
 *
 * @author German DR
 */
public class FrmPrincipal extends javax.swing.JFrame {

    /**
     * Creates new form FrmPrincipal
     */
    public FrmPrincipal() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        btnSair = new javax.swing.JButton();
        btnDatosPersonais = new javax.swing.JButton();
        btnDatosAcademicos = new javax.swing.JButton();
        btnCascada = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Exemplo multixanela JDialog");
        getContentPane().setLayout(null);

        btnSair.setText("Sair");
        btnSair.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSairActionPerformed(evt);
            }
        });
        getContentPane().add(btnSair);
        btnSair.setBounds(190, 40, 170, 29);
    }
}
```

```

        btnDatosPersoais.setText("Datos persoais");
        btnDatosPersoais.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnDatosPersoaisActionPerformed(evt);
            }
        });
        getContentPane().add(btnDatosPersoais);
        btnDatosPersoais.setBounds(10, 10, 170, 29);

        btnDatosAcademicos.setText("Datos académicos");
        btnDatosAcademicos.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnDatosAcademicosActionPerformed(evt);
            }
        });
        getContentPane().add(btnDatosAcademicos);
        btnDatosAcademicos.setBounds(370, 10, 170, 29);

        btnCascada.setText("Cascada");
        btnCascada.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnCascadaActionPerformed(evt);
            }
        });
        getContentPane().add(btnCascada);
        btnCascada.setBounds(190, 10, 170, 29);

        java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
        setBounds((screenSize.width-568)/2, (screenSize.height-139)/2, 568, 139);
    } // </editor-fold>

    private void btnDatosPersoaisActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        if (XestorDeXanelas.abrirXanelasDatosPersoais())
        {
            DlgDatosPersoais dlgDatosPersoais=new DlgDatosPersoais(this, false);
            XestorDeXanelas.engadirXanelaDatosPersoais(dlgDatosPersoais);
            dlgDatosPersoais.setVisible(true);
        }
        else
        {
            JOptionPane.showMessageDialog(this, "Non é posible abrir máis xanelas deste tipo");
            return;
        }
    }

    public void xanelasDatosPersoaisEnCascada()
    {
        Vector xanelasDatosPersoais=XestorDeXanelas.recuperarXanelasDatosPersoais();
        int posX=100, posY=300, incremento=50;
        for(int i=0;i<xanelasDatosPersoais.size();i++)
        {
            DlgDatosPersoais xanela=(DlgDatosPersoais)xanelasDatosPersoais.elementAt(i);
            xanela.setLocation(posX, posY);
            posX+=incremento;
            posY+=incremento;
        }
    }

    public void xestionDeMensaxesDeErro(int numErro)
    {
        switch(numErro)
        {
            case 1: JOptionPane.showMessageDialog(this, "O nome non pode estar baleiro");
                    break;
            case 2: JOptionPane.showMessageDialog(this, "Os apelidos non poden estar baleiros");
                    break;
        }
    }

    public void xestionDeMensaxesDeGraoAcadado(int grao)
    {
        switch(grao)
        {
            case 1: JOptionPane.showMessageDialog(this, "O máximo grao que acadou vostede e ESO");
                    break;
            case 2: JOptionPane.showMessageDialog(this, "O máximo grao que acadou vostede e Bachalerato");
                    break;
            case 3: JOptionPane.showMessageDialog(this, "O máximo grao que acadou vostede e FP");
                    break;
            case 4: JOptionPane.showMessageDialog(this, "O máximo grao que acadou vostede e Universidade");
                    break;
        }
    }

    private void btnDatosAcademicosActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        if (XestorDeXanelas.abrirXanelasDatosAcademicos())
        {
            DlgDatosAcademicos dlgDatosAcademicos=new DlgDatosAcademicos(this, false);
            dlgDatosAcademicos.setVisible(true);
        }
        else
        {
            JOptionPane.showMessageDialog(this, "Non é posible abrir máis xanelas deste tipo");
            return;
        }
    }

    private void btnSairActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        System.exit(0);
    }
}

```

```

private void btnCascadaActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    xanelasDatosPersoaisEnCascada();
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    }
    //</editor-fold>

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new FrmPrincipal().setVisible(true);
        }
    });
}
// Variables declaration - do not modify
private javax.swing.JButton btnCascada;
private javax.swing.JButton btnDatosAcademicos;
private javax.swing.JButton btnDatosPersoais;
private javax.swing.JButton btnSair;
// End of variables declaration
}

```

DlgDatosPersoais

```

/**
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

/**
 *
 * @author German DR
 */
public class DlgDatosPersoais extends javax.swing.JDialog {

    /**
     * Creates new form DlgTipoA
     */
    public DlgDatosPersoais(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        lblNome = new javax.swing.JLabel();
        txtNome = new javax.swing.JTextField();
        lblApellidos = new javax.swing.JLabel();
        txtApellidos = new javax.swing.JTextField();
        btnAceptar = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Datos persoais");
        setCursor(new java.awt.Cursor(java.awt.Cursor.DEFAULT_CURSOR));
        setType(java.awt.Window.Type.POPUP);
        addWindowListener(new java.awt.event.WindowAdapter() {
            public void windowClosed(java.awt.event.WindowEvent evt) {
                formWindowClosed(evt);
            }
        });

        lblNome.setText("Nome");

        lblApellidos.setText("Apellidos");
    }

```

```

        btnAceptar.setText("Aceptar");
        btnAceptar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnAceptarActionPerformed(evt);
            }
        });
    }

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(lblNome)
                    .addComponent(lblApellidos))
                .addGap(18, 18, 18)
                .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(txtNome)
                    .addComponent(txtApellidos))
                .addContainerGap())
            .addGroup(layout.createSequentialGroup()
                .addComponent(btnAceptar)
                .addGap(153, 153, 153))
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addComponent(lblNome)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(lblApellidos)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(txtNome)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(txtApellidos)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(btnAceptar)
                .addContainerGap())
    );

    pack();
} // </editor-fold>

private void formWindowClosed(java.awt.event.WindowEvent evt) {
    // TODO add your handling code here:
    XestorDeXanelas.eliminarXanelaDadosPersoais(this);
    XestorDeXanelas.cerrarXanelasDadosPersoais();
}

private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    if (txtNome.getText().trim().compareTo("") == 0) {
        ((FrmPrincipal)getParent()).xestionDeMensaxesDeErro(1);
        return;
    }
    if (txtApellidos.getText().trim().compareTo("") == 0) {
        ((FrmPrincipal)getParent()).xestionDeMensaxesDeErro(2);
        return;
    }
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(DlgDatosPersoais.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(DlgDatosPersoais.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(DlgDatosPersoais.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(DlgDatosPersoais.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    }
} //</editor-fold>

/* Create and display the dialog */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        DlgDatosPersoais dialog = new DlgDatosPersoais(new javax.swing.JFrame(), true);
        dialog.addWindowListener(new java.awt.event.WindowAdapter() {
            @Override
            public void windowClosing(java.awt.event.WindowEvent e) {

```

```

        System.exit(0);
    }
    });
    dialog.setVisible(true);
}
});
}
// Variables declaration - do not modify
private javax.swing.JButton btnAceptar;
private javax.swing.JLabel lblApellidos;
private javax.swing.JLabel lblNome;
private javax.swing.JTextField txtApellidos;
private javax.swing.JTextField txtNome;
// End of variables declaration
}

```

DlgDatosAcademicos

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

/**
 *
 * @author German DR
 */
public class DlgDatosAcademicos extends javax.swing.JDialog {

    /**
     * Creates new form DlgDatosAcademicos
     */
    public DlgDatosAcademicos(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        xanelaPai=(FrmPrincipal)parent;
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        btngrpGrao = new javax.swing.ButtonGroup();
        pnlMaximoGrao = new javax.swing.JPanel();
        rbtESO = new javax.swing.JRadioButton();
        rbtBachalerato = new javax.swing.JRadioButton();
        rbtFP = new javax.swing.JRadioButton();
        rbtUniversidade = new javax.swing.JRadioButton();
        btnAceptar = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Datos académicos");
        addWindowListener(new java.awt.event.WindowAdapter() {
            public void windowClosed(java.awt.event.WindowEvent evt) {
                formWindowClosed(evt);
            }
        });

        pnlMaximoGrao.setBorder(javax.swing.BorderFactory.createTitledBorder("Indique o maximo grao acadado"));

        btngrpGrao.add(rbtESO);
        rbtESO.setSelected(true);
        rbtESO.setText("ESO");

        btngrpGrao.add(rbtBachalerato);
        rbtBachalerato.setText("Bachalerato");

        btngrpGrao.add(rbtFP);
        rbtFP.setText("FP");

        btngrpGrao.add(rbtUniversidade);
        rbtUniversidade.setText("Universidade");

        javax.swing.GroupLayout pnlMaximoGraoLayout = new javax.swing.GroupLayout(pnlMaximoGrao);
        pnlMaximoGrao.setLayout(pnlMaximoGraoLayout);
        pnlMaximoGraoLayout.setHorizontalGroup(
            pnlMaximoGraoLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(pnlMaximoGraoLayout.createSequentialGroup()
                    .add(pnlMaximoGraoLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                        .addGroup(pnlMaximoGraoLayout.createSequentialGroup()
                            .addGap(24, 24, 24)
                            .addGroup(pnlMaximoGraoLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                                .addComponent(rbtESO)
                                .addComponent(rbtUniversidade)
                                .addComponent(rbtFP)
                                .addComponent(rbtBachalerato))
                            .addContainerGap(123, Short.MAX_VALUE))
                        .addGroup(pnlMaximoGraoLayout.createSequentialGroup()
                            .add(pnlMaximoGraoLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                                .addComponent(rbtESO)
                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                                .addComponent(rbtBachalerato)
                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                                .addComponent(rbtFP)
                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                                .addComponent(rbtUniversidade))
                            .addContainerGap(123, Short.MAX_VALUE))
                    )
                )
        );
    }

```

```

        .addGap(7, 7, 7))
    );

    btnAceptar.setText("Aceptar");
    btnAceptar.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnAceptarActionPerformed(evt);
        }
    });

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addComponent(pnlMaximoGrao, javax.swing.GroupLayout.PREFERRED_SIZE,
                    javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(btnAceptar, javax.swing.GroupLayout.PREFERRED_SIZE, Short.MAX_VALUE)
                .addContainerGap())
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addComponent(pnlMaximoGrao, javax.swing.GroupLayout.PREFERRED_SIZE, 157,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(btnAceptar, javax.swing.GroupLayout.PREFERRED_SIZE, Short.MAX_VALUE)
                .addContainerGap())
    );

    pack();
} // </editor-fold>

private void formWindowClosed(java.awt.event.WindowEvent evt) {
    // TODO add your handling code here:
    XestorDeXanelas.cerrarXanelasDatosAcademicos();
}

private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int grao;
    if (rbtESO.isSelected()) grao=1;
    else if (rbtBachalerato.isSelected()) grao=2;
    else if (rbtFP.isSelected()) grao=3;
    else grao=4;
    xanelaPai.xestionDeMensaxesDeGraoAcadado(graο);
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(DlgDatosAcademicos.class.getName()).log(java.util.logging.Level.SEVERE,
            null, ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(DlgDatosAcademicos.class.getName()).log(java.util.logging.Level.SEVERE,
            null, ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(DlgDatosAcademicos.class.getName()).log(java.util.logging.Level.SEVERE,
            null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(DlgDatosAcademicos.class.getName()).log(java.util.logging.Level.SEVERE,
            null, ex);
    }
} //</editor-fold>

/* Create and display the dialog */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        DlgDatosAcademicos dialog = new DlgDatosAcademicos(new javax.swing.JFrame(), true);
        dialog.addWindowListener(new java.awt.event.WindowAdapter() {
            @Override
            public void windowClosing(java.awt.event.WindowEvent e) {
                System.exit(0);
            }
        });
        dialog.setVisible(true);
    }
});

// Variables declaration - do not modify
private javax.swing.JButton btnAceptar;
private javax.swing.ButtonGroup btnGrpGrao;
private javax.swing.JPanel pnlMaximoGrao;
private javax.swing.JRadioButton rbtBachalerato;
private javax.swing.JRadioButton rbtESO;

```



```

private javax.swing.JRadioButton rbtFP;
private javax.swing.JRadioButton rbtUniversidade;
// End of variables declaration
private FrmPrincipal xanelaPai;
}

```

XestorDeXanelas

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

import java.util.Vector;

/**
 *
 * @author German DR
 */
public class XestorDeXanelas {
    static private int numXanelasDatosPersoais=0;
    static private int numXanelasDatosAcademicos=0;
    static private Vector xanelasDatosPersoais=new Vector();

    public static void engadirXanelaDatosPersoais(DlgDatosPersoais xanela)
    {
        xanelasDatosPersoais.add(xanela);
    }

    public static void eliminarXanelaDatosPersoais(DlgDatosPersoais xanela)
    {
        for(int i=0;i<xanelasDatosPersoais.size();i++)
        {
            if(xanelasDatosPersoais.elementAt(i)==xanela)
            {
                xanelasDatosPersoais.removeElementAt(i);
                break;
            }
        }
        System.out.println(xanelasDatosPersoais.size());
    }

    public static Vector recuperarXanelasDatosPersoais()
    {
        return xanelasDatosPersoais;
    }

    public static void cerrarXanelasDatosPersoais()
    {
        numXanelasDatosPersoais--;
    }
    public static boolean abrirXanelasDatosPersoais()
    {
        if(numXanelasDatosPersoais<5)
        {
            numXanelasDatosPersoais++;
            return true;
        }
        else
        {
            return false;
        }
    }

    public static void cerrarXanelasDatosAcademicos()
    {
        numXanelasDatosAcademicos--;
    }

    public static boolean abrirXanelasDatosAcademicos()
    {
        if(numXanelasDatosAcademicos<2)
        {
            numXanelasDatosAcademicos++;
            return true;
        }
        else
        {
            return false;
        }
    }
}

```

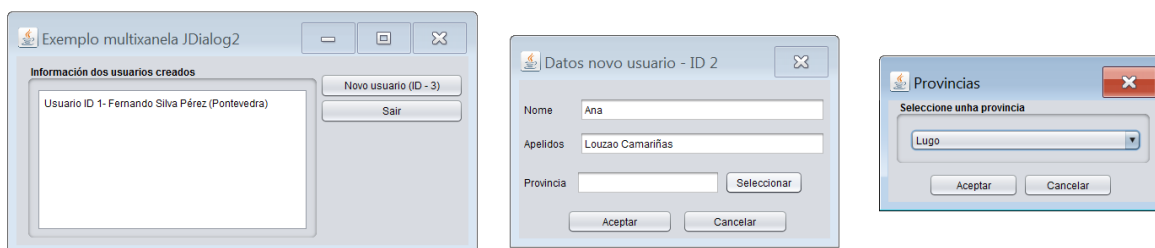
Paso de información entre xanelas.

É habitual que unha aplicación de tipo multixanela teña que pasar información dunha xanela a outra. A resolución deste problema baséase sempre nalgunha destas dúas técnicas:

- Ter unha referencia á xanela á que lle queremos pasar a información e empregar algún método público desta para enviar a información

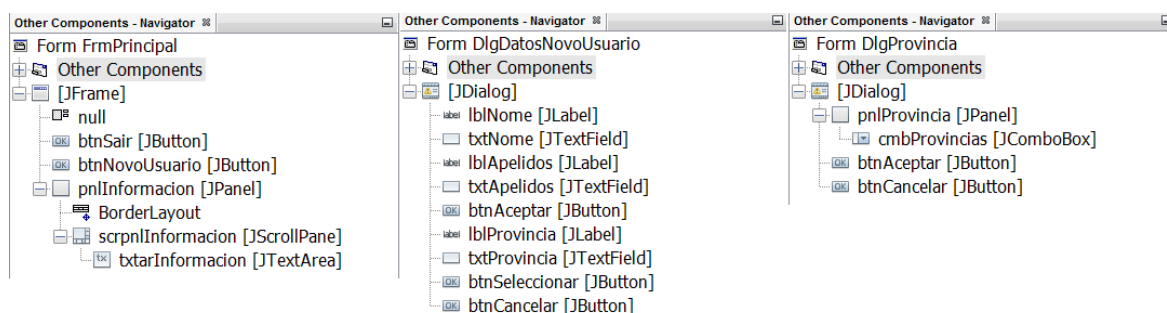
- Pasar a información a través do construtor cando creamos a xanela.

Hai outras solucións como p.e. empregar unha clase intermedia que funcione a modo de buzón entre as xanelas que queren comunicarse, pero en xeral, excepto en casos puntuais, o emprego deste método complica innecesariamente o código da aplicación. Polo tanto, imos ver como pasar información entre xanelas empregando as dúas técnicas enumeradas anteriormente. Para elo imos desenvolver a seguinte aplicación:



A primeira xanela é a xanela principal da aplicación. Por suposto, tal e como estamos vendo, é un JFrame. Emprégase para dar de alta novos usuarios no sistema e para listar os que temos creados. Ao premer sobre o botón Novo usuario crearase un novo usuario cuxo identificador será o amosado entre parénteses no botón Novo usuario. Ao premer sobre este botón Novo usuario créase unha xanela de diálogo (xanela central na imaxe) na cal danse de alta os datos dese novo usuario. ¿Onde está o paso de información entre estas dúas xanelas?. Hai que fixarse no título do diálogo aberto. O seu título é Datos novo usuario concatenado co identificador que amosado no botón da xanela pai (neste caso ID – 2). Na xanela central daranse de alta os datos do novo usuario, non obstante, para indicar a provincia non é posible escribir na caixa de texto adxunta, senón que hai que seleccionar a provincia dun combo de provincias. Este combo con provincias atópase nunha terceira xanela (a da dereita) de tipo diálogo que é amosada cando prememos sobre o botón seleccionar da xanela central. O comportamento da xanela Provincias é modal. Isto como xa explicamos con anterioridade quere dicir que non se pode acceder ao resto de xanelas que compoñen a aplicación ata que a xanela modal sexa pechada. Cando prememos sobre o botón Aceptar do diálogo Provincias, pasarase a provincia seleccionada no combo á caixa de texto provincia do diálogo Datos novo usuario. Por ultimo, cando premamos sobre o botón Aceptar do diálogo Datos novo usuario, no caso de que a información introducida sexa correcta, enviaranse os datos do novo usuario creado ao formulario principal para que os amose na area de texto que contén. Non hai limitación respecto ao número de xanelas fillas que pode ter abertas o formulario principal, polo tanto, neste caso non precisaremos empregar un xestor de xanelas.

Na seguinte imaxe amósase a composición gráfica de cada unha das xanelas que forman parte da aplicación:



Primeiramente imos ver como pasamos a información entre o formulario FrmPrincipal e o diálogo DlgDatosNovoUsuario. Lembremos que a información a enviar será o identificador para a creación do novo usuario, o cal atópase no texto do botón btnNovoUsuario do formulario FrmPrincipal. Neste caso a información vaise pasar ao diálogo no momento da súa creación, de xeito que a forma máis sinxela de facelo é pasar a información a través do construtor do diálogo. Para elo modificamos o seu construtor:

```
public DlgDatosNovoUsuario(java.awt.Frame parent, boolean modal, int id) {  
    super(parent, modal);  
    idUsuario=id;  
    initComponents();  
    setTitle("Datos novo usuario - ID "+idUsuario);  
}
```

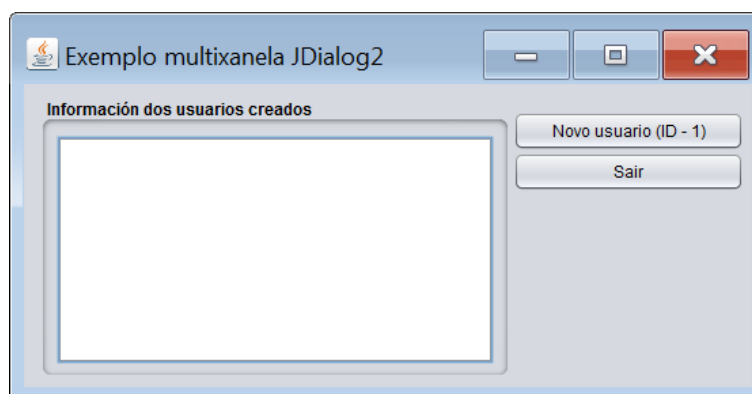
Como podemos observar no código anterior, foi engadido ao construtor xerado polo entorno de programación un terceiro parámetro chamado id. Empregaremos este parámetro para pasar ao diálogo o identificador de usuario que ten que amosar na súa barra de título. Ademais no corpo do construtor podemos ver como é empregado o parámetro pasado para modificar o título da xanela mediante o seu método setTitle.

Respecto á chamada realizada desde o formulario FrmPrincipal ao premer sobre o botón Novo usuario, o único que hai que facer é empregar o novo construtor pasándolle os argumentos axeitados:

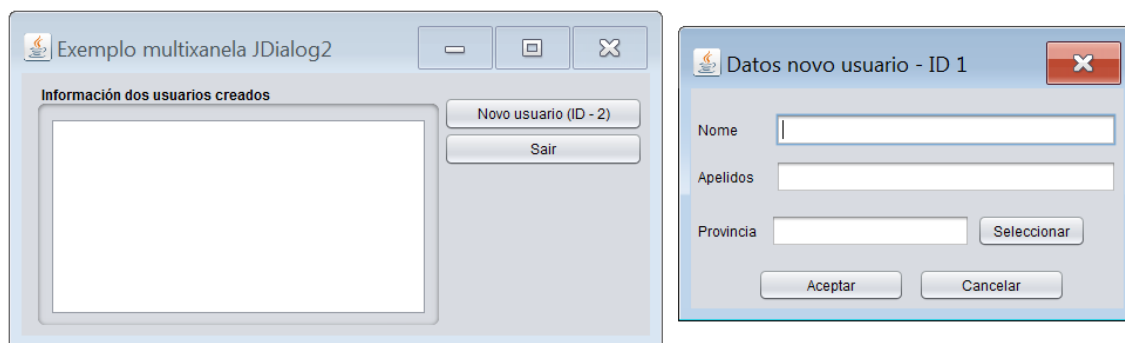
```
DlgDatosNovoUsuario dlgDatosNovoUsuario=new DlgDatosNovoUsuario(this, false, idUsuario);
```

Empregamos a variable idUsuario na chamada ao construtor do diálogo de clase DlgDatosNovoUsuario. Esta variable é un atributo a nivel de clase do formulario FrmPrincipal no cal almacenamos o identificador do seguinte usuario a crear. Este atributo emprégase tamén para xerar o texto do botón Novo usuario.

Nas seguintes imaxes pódese observar o paso de información que acabamos de explicar. Nesta imaxe e na seguinte visualízase o inicio do proceso para crear un usuario con identificador 1:



Partindo da situación inicial da imaxe anterior, Ao pulsar sobre o botón Novo usuario ábrese a xanela para introducir os datos de ese novo usuario. Fixémonos no título do diálogo aberto. Nel pódese observar o paso de información entre as dúas xanelas, xa que na súa barra de título amósase o identificador de usuario pasado desde o formulario pai. Fixémonos tamén que na xanela pai o texto do botón cambiou o número do identificador, de xeito que se é premido abrírase un novo diálogo no cal poderase introducir a información referente ao usuario con identificador 2:



De seguido imos resolver o paso de información entre o diálogo `DlgProvincia` e o diálogo `DlgDatosNovoUsuario`. Lembremos que se debe pasar a provincia seleccionada no combo do diálogo `DlgProvincia` á caixa de texto do diálogo `DlgDatosNovoUsuario`. Antes de ver como pasamos a información entre estes dous diálogos imos facer un pequeno inciso.

Este inciso refírese a como é creado un diálogo desde outro diálogo. Cando sobre o diálogo `DlgDatosNovoUsuario` prememos sobre o botón `Seleccionar` creárase e amosárase o diálogo `DlgProvincias`. A creación do novo diálogo é practicamente igual ao visto ata agora, pero hai un pequeno detalle a considerar. Ao crear a clase `DlgProvincias` o entorno de programación crea o seguinte construtor:

```
public DlgProvincia(java.awt.Frame parent, boolean modal) {
    super(parent, modal);
    initComponents();
}
```

O problema reside en que o primeiro parámetro (xanela pai) é de tipo `java.awt.Frame` e polo tanto unicamente pode recibir obxectos desa clase e dos seus descendentes. Non obstante, neste caso estamos creando un diálogo desde outro diálogo e os diálogos non pertencen á arbore de herdanza de `java.awt.Frame`, de xeito que temos que crear un novo construtor que sexa capaz de recibir obxectos de clase `JDialog`. Ao final, é tan sinxelo como engadir un novo construtor igual que o xerado polo entorno de programación pero coa sinatura modificada:

```
public DlgProvincia(javax.swing.JDialog parent, boolean modal) {
    super(parent, modal);
    initComponents();
}
```

Fixémonos que agora o primeiro parámetro do construtor si está preparado para recibir obxectos da clase `JDialog`.

Ben, unha vez resolto este pequeno problema continuemos co paso de información entre os diálogos `DlgProvincia` e `DlgDatosNovoUsuario`. Neste caso imos facer o seguinte: o diálogo `DlgDatosNovoUsuario` vai ter un método público que recibirá como parámetro un `String` (a provincia) que ao ser invocado colocara ese `String` recibido na caixa de texto `txtProvincia`. Por outra banda o diálogo `DlgProvincia` vai ter unha referencia ao diálogo que a creou (diálogo de clase `DlgDatosNovoUsuario`) a cal vai empregar para invocar os seus métodos públicos. Empregando esta referencia invocará ao método que establece un texto na caixa de texto `txtProvincia` e ao invocalo vaino facer pasándolle a provincia seleccionada no combo `cmbProvincias`.

Imos por partes. Primeiro creamos no diálogo `DlgDatosNovoUsuario` un método público que recibirá como parámetro un `String` (a provincia) que ao ser invocado colocara ese `String` recibido na caixa de texto `txtProvincia`

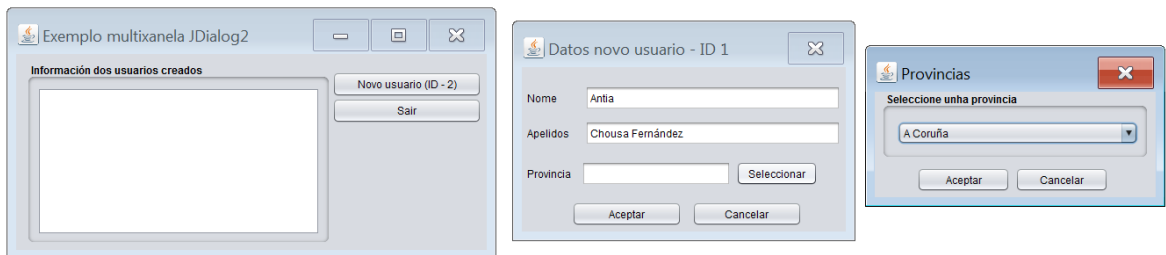
```
public void establecerProvincia(String provincia)
{
    txtProvincia.setText(provincia);
}
```

A continuación no clic do botón `btnAceptar` do diálogo `DlgProvincia` capturamos o valor seleccionado no combo e pasámolo ao seu diálogo pai:

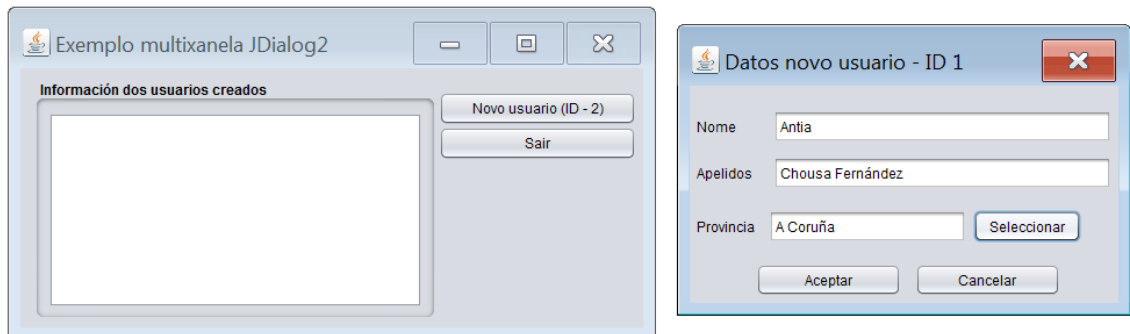
```
private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    String provincia=(String)cmbProvincias.getModel().getElementAt(cmbProvincias.getSelectedIndex());  
    ((DlgDatosNovoUsuario)getParent()).establecerProvincia(provincia);  
    dispose();  
}
```

Unha vez recuperado o elemento seleccionado no combo, facemos emprego do método `getParent` para ter unha referencia á xanela pai e invocamos ao seu método `establecerProvincia` pasándolle a provincia que acabamos de recuperar do combo. Fixémonos na seguinte liña, `dispose()`. O método `dispose` emprégase para pechar a xanela a través de código. É o equivalente a premer na aspa da xanela.

Nas seguintes imaxes pódese observar o paso de información que acabamos de explicar. Nesta imaxe estamos seleccionando a provincia no combo:



Nesta outra imaxe acabamos de premer sobre o botón aceptar e polo tanto a información envórcase sobre o diálogo pai do diálogo `Provincias`, e ademais o diálogo `Provincias` péchase:



Finalmente imos ver como pasamos a información desde o diálogo `DlgDatosNovoUsuario` ao formulario `FrmPrincipal`. Cando xa temos recheos os campos do diálogo `DlgDatosNovoUsuario` premeremos sobre o botón `Aceptar`. No caso de que a validación sexa correcta crearemos un obxecto da clase `Usuario` (ver código) que encapsula tódolos datos do usuario que estamos creando e ese obxecto creado enviáremolo ao formulario `FrmPrincipal` para que este envorque os datos dese obxecto pasado na área de texto que conten. Para resolver o paso de información entre o diálogo `DlgDatosNovoUsuario` e o formulario `FrmPrincipal` imos facer o seguinte: o formulario `FrmPrincipal` vai ter un método público que recibirá como parámetro un obxecto da clase `Usuario` que conterá a información do novo usuario creado. Este método ao ser invocado engadirá a información do obxecto recibido á información existente na área de texto `txtarInformacion` do formulario `FrmPrincipal`. Por outra parte o diálogo `DlgDatosNovoUsuario` vai ter unha referencia ao formulario `FrmPrincipal`, a cal empregará para invocar os seus métodos públicos. Empregando esta referencia vai invocar ao método público do formulario `FrmPrincipal` que

establece a información dun obxecto de clase Usuario na súa área de texto txtarInformacion e ao invocalo farano pasándolle o obxecto de clase Usuario xerado a partir da información recollida mediante os campos do diálogo DlgDatosNovoUsuario.

Primeiramente creamos no formulario FrmPrincipal un método público que recibirá como parámetro un obxecto de clase Usuario que ao ser invocado colocará a información dese obxecto na área de texto txtarInformacion.

```
public void engadirInfoNovoUsuario (Usuario usuario)
{
    txtarInformacion.setText(usuario.toString()+"\n"+txtarInformacion.getText());
}
```

A continuación, no clic do botón btnAceptar do diálogo DlgDatosNovoUsuario validamos os campos e no caso de ser correctos os encapsulamos nun obxecto de clase Usuario. Por último pasamos o obxecto creado á xanela pai do diálogo:

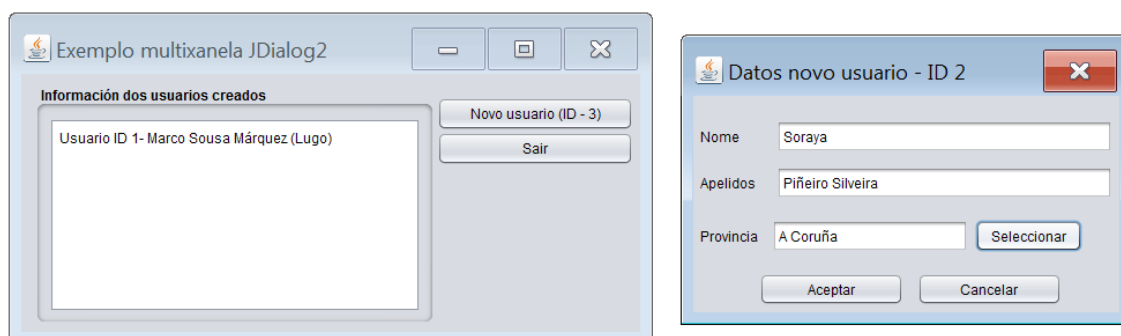
```
private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String nome=txtNome.getText().trim();
    String apelidos=txtApelidos.getText().trim();
    String provincia=txtProvincia.getText().trim();

    if (nome.compareTo("")==0)
    {
        JOptionPane.showMessageDialog(this, "Debe indicar o nome do usuario");
        return;
    }
    if (apelidos.compareTo("")==0)
    {
        JOptionPane.showMessageDialog(this, "Debe indicar os apelidos do usuario");
        return;
    }
    if (provincia.compareTo("")==0)
    {
        JOptionPane.showMessageDialog(this, "Debe indicar a provincia do usuario");
        return;
    }

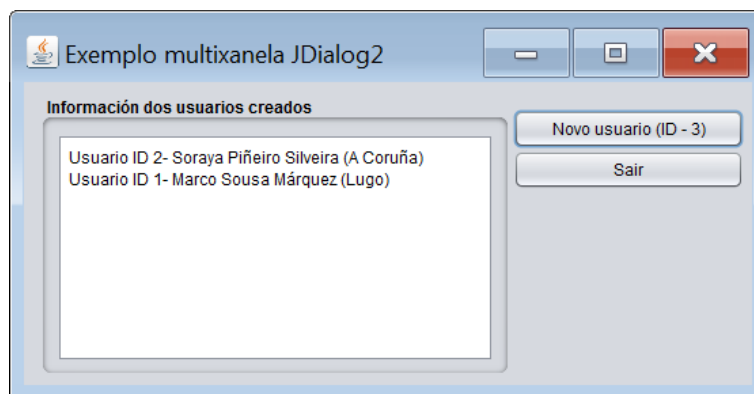
    Usuario usuario=new Usuario(idUsuario, nome, apelidos, provincia);
    ((FrmPrincipal)getParent()).engadirInfoNovoUsuario(usuario);
    dispose();
}
```

Unha vez xerado o obxecto de clase Usuario, facemos emprego do método getParent para ter unha referencia á xanela pai e invocamos ao seu método engadirInfoNovoUsusario pasándolle o obxecto de clase Usuario que acabamos de xerar. Finalmente facemos un dispose para pechar o diálogo.

Nas seguintes imaxes pódese observar o paso de información que acabamos de explicar. Nesta imaxe estamos creando un novo usuario:



Nesta outra imaxe acabamos de premer sobre o botón Aceptar e polo tanto a información do diálogo encapsúlase nun obxecto de clase Usuario que se envía ao formulario FrmPrincipal. Finalmente a información recibida do diálogo é engadida á área de texto do formulario. Ademais o diálogo empregado para dar de alta ao novo usuario é pechado:



A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación:

FrmPrincipal

```
package exemploJDialog;

import java.awt.event.ItemEvent;
import javax.swing.JOptionPane;

/**
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */

/**
 *
 * @author German DR
 */
public class FrmPrincipal extends javax.swing.JFrame {

    /**
     * Creates new form FrmPrincipal
     */
    public FrmPrincipal() {
        initComponents();
        pintarBoton();
    }

    private void pintarBoton()
    {
        idUsuario++;
        btnNovoUsuario.setText("Novo usuario (ID - "+idUsuario+")");
    }

    public void engadirInfoNovoUsuario(Usuario usuario)
    {
        txtarInformacion.setText(usuario.toString()+"\n"+txtarInformacion.getText());
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        btnSair = new javax.swing.JButton();
        btnNovoUsuario = new javax.swing.JButton();
        pnlInformacion = new javax.swing.JPanel();
        scrpnlInformacion = new javax.swing.JScrollPane();
        txtarInformacion = new javax.swing.JTextArea();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Exemplo multixanela JDialog2");
        getContentPane().setLayout(null);

        btnSair.setText("Sair");
        btnSair.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSairActionPerformed(evt);
            }
        });
        getContentPane().add(btnSair);
        btnSair.setBounds(360, 50, 170, 29);

        btnNovoUsuario.setText("Novo usuario");
        btnNovoUsuario.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnNovoUsuarioActionPerformed(evt);
            }
        });
        getContentPane().add(btnNovoUsuario);
        btnNovoUsuario.setBounds(360, 20, 170, 29);
    }
}
```

```

        pnlInformacion.setBorder(javax.swing.BorderFactory.createTitledBorder("Información dos usuarios creados"));
        pnlInformacion.setLayout(new java.awt.BorderLayout());

        txtarInformacion.setColumns(20);
        txtarInformacion.setRows(5);
        scrpnlInformacion.setViewportView(txtarInformacion);

        pnlInformacion.add(scrpnlInformacion, java.awt.BorderLayout.CENTER);

        getContentPane().add(pnlInformacion);
        pnlInformacion.setBounds(10, 10, 350, 210);

        java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
        setBounds((screenSize.width-558)/2, (screenSize.height-287)/2, 558, 287);
    } // </editor-fold>

    private void btnNovoUsuarioActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        DlgDatosNovoUsuario dlgDatosNovoUsuario=new DlgDatosNovoUsuario(this, false, idUsuario);
        dlgDatosNovoUsuario.setVisible(true);
        pintarBoton();
    }

    private void btnSairActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        System.exit(0);
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
         * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (InstantiationException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (IllegalAccessException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        }
        //</editor-fold>

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new FrmPrincipal().setVisible(true);
            }
        });
    }
    // Variables declaration - do not modify
    private javax.swing.JButton btnNovoUsuario;
    private javax.swing.JButton btnSair;
    private javax.swing.JPanel pnlInformacion;
    private javax.swing.JScrollPane scrpnlInformacion;
    private javax.swing.JTextArea txtarInformacion;
    // End of variables declaration
    private int idUsuario=0;
}

```

DlgDatosNovoUsuario

```

/**
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class DlgDatosNovoUsuario extends javax.swing.JDialog {

    /**
     * Creates new form DlgTipoA
     */
    public DlgDatosNovoUsuario(java.awt.Frame parent, boolean modal, int id) {
        super(parent, modal);
    }
}

```



```

        idUsuario=id;
        initComponents();
        setTitle("Datos novo usuario - ID "+idUsuario);
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        lblNome = new javax.swing.JLabel();
        txtNome = new javax.swing.JTextField();
        lblApelidos = new javax.swing.JLabel();
        txtApelidos = new javax.swing.JTextField();
        btnAceptar = new javax.swing.JButton();
        lblProvincia = new javax.swing.JLabel();
        txtProvincia = new javax.swing.JTextField();
        btnSeleccionar = new javax.swing.JButton();
        btnCancelar = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Datos novo usuario");
        setCursor(new java.awt.Cursor(java.awt.Cursor.DEFAULT_CURSOR));
        setType(java.awt.Window.Type.POPUP);

        lblNome.setText("Nome");

        lblApelidos.setText("Apelidos");

        btnAceptar.setText("Aceptar");
        btnAceptar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnAceptarActionPerformed(evt);
            }
        });

        lblProvincia.setText("Provincia");

        txtProvincia.setEditable(false);

        btnSeleccionar.setText("Seleccionar");
        btnSeleccionar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSeleccionarActionPerformed(evt);
            }
        });

        btnCancelar.setText("Cancelar");
        btnCancelar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnCancelarActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(layout.createSequentialGroup()
                    .addGap(18, 18, 18)
                    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                        .addComponent(lblNome)
                        .addComponent(lblApelidos))
                    .addGap(18, 18, 18)
                    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                        .addComponent(txtNome, javax.swing.GroupLayout.DEFAULT_SIZE, 291, Short.MAX_VALUE)
                        .addComponent(txtApelidos))
                    .addGap(18, 18, 18)
                    .addComponent(lblProvincia)
                    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, javax.swing.GroupLayout.PREFERRED_SIZE,
                        javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(txtProvincia, javax.swing.GroupLayout.PREFERRED_SIZE, 170,
                        javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                    .addComponent(btnSeleccionar)
                    .addGap(0, 0, Short.MAX_VALUE))
                .addGroup(layout.createSequentialGroup()
                    .addGap(50, 50, 50)
                    .addComponent(btnAceptar, javax.swing.GroupLayout.PREFERRED_SIZE, 125,
                        javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                    .addComponent(btnCancelar, javax.swing.GroupLayout.PREFERRED_SIZE, 125,
                        javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addGap(50, 50, 50))
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(layout.createSequentialGroup()
                    .addGap(21, 21, 21)
                    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                        .addComponent(txtNome)
                        .addComponent(lblNome)
                        .addComponent(lblApelidos)
                        .addComponent(txtApelidos, javax.swing.GroupLayout.PREFERRED_SIZE,
                            javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(lblProvincia)
                        .addComponent(txtProvincia, javax.swing.GroupLayout.PREFERRED_SIZE,
                            javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(btnAceptar, javax.swing.GroupLayout.PREFERRED_SIZE,
                            javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addComponent(btnCancelar, javax.swing.GroupLayout.PREFERRED_SIZE,
                            javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addGap(18, 18, 18))
                )
        );
    }

```

```

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(lblProvincia)
            .addComponent(txtProvincia, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
            .addComponent(btnSelecionar))
        .addGap(18, 18, 18)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(btnCancelar)
            .addComponent(btnAceptar))
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold>

private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String nome=txtNome.getText().trim();
    String apellidos=txtApellidos.getText().trim();
    String provincia=txtProvincia.getText().trim();

    if(nome.compareTo("")==0)
    {
        JOptionPane.showMessageDialog(this, "Debe indicar o nome do usuario");
        return;
    }
    if(apellidos.compareTo("")==0)
    {
        JOptionPane.showMessageDialog(this, "Debe indicar os apellidos do usuario");
        return;
    }
    if(provincia.compareTo("")==0)
    {
        JOptionPane.showMessageDialog(this, "Debe indicar a provincia do usuario");
        return;
    }

    Usuario usuario=new Usuario(idUsuario, nome, apellidos, provincia);
    ((FrmPrincipal)getParent()).engadirInfoNovoUsuario(usuario);
    dispose();
}

public void establecerProvincia(String provincia)
{
    txtProvincia.setText(provincia);
}

private void btnSelecionarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    DlgProvincia dlgProvincia=new DlgProvincia(this, true);
    dlgProvincia.setVisible(true);
}

private void btnCancelarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    dispose();
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(DlgDatosNovoUsuario.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(DlgDatosNovoUsuario.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(DlgDatosNovoUsuario.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(DlgDatosNovoUsuario.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    }
    //</editor-fold>

    /* Create and display the dialog */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            DlgDatosNovoUsuario dialog = new DlgDatosNovoUsuario(new javax.swing.JFrame(), true, 0);
            dialog.addWindowListener(new java.awt.event.WindowAdapter() {
                @Override
                public void windowClosing(java.awt.event.WindowEvent e) {
                    System.exit(0);
                }
            });
            dialog.setVisible(true);
        }
    });
}
// Variables declaration - do not modify

```

```

private javax.swing.JButton btnAceptar;
private javax.swing.JButton btnCancelar;
private javax.swing.JButton btnSeleccionar;
private javax.swing.JLabel lblApellidos;
private javax.swing.JLabel lblNombre;
private javax.swing.JLabel lblProvincia;
private javax.swing.JTextField txtApellidos;
private javax.swing.JTextField txtNombre;
private javax.swing.JTextField txtProvincia;
// End of variables declaration
private int idUsuario;
}

```

DlgProvincia

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package ejemploJDialog;

/**
 *
 * @author German DR
 */
public class DlgProvincia extends javax.swing.JDialog {

    public DlgProvincia(javax.swing.JDialog parent, boolean modal) {
        super(parent, modal);
        initComponents();
    }

    /**
     * Creates new form DlgDatosAcademicos
     */
    public DlgProvincia(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        pnlProvincia = new javax.swing.JPanel();
        cmbProvincias = new javax.swing.JComboBox();
        btnAceptar = new javax.swing.JButton();
        btnCancelar = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Provincias");

        pnlProvincia.setBorder(javax.swing.BorderFactory.createTitledBorder("Seleccione unha provincia"));

        cmbProvincias.setModel(new javax.swing.DefaultComboBoxModel(new String[] { "A Coruña", "Lugo", "Ourense",
"Pontevedra" }));

        javax.swing.GroupLayout pnlProvinciaLayout = new javax.swing.GroupLayout(pnlProvincia);
        pnlProvincia.setLayout(pnlProvinciaLayout);
        pnlProvinciaLayout.setHorizontalGroup(
            pnlProvinciaLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(pnlProvinciaLayout.createSequentialGroup()
                    .add(pnlProvinciaLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                        .add(cmbProvincias)
                        .add(btnAceptar)
                        .add(btnCancelar)
                    )
                    .add(new javax.swing.GroupLayout.DEFAULT_SIZE(Short.MAX_VALUE), Short.MAX_VALUE))
                .add(new javax.swing.GroupLayout.DEFAULT_SIZE(Short.MAX_VALUE), Short.MAX_VALUE))
        );

        btnAceptar.setText("Aceptar");
        btnAceptar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnAceptarActionPerformed(evt);
            }
        });

        btnCancelar.setText("Cancelar");
        btnCancelar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnCancelarActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .add(pnlProvincia)
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .add(pnlProvincia)
        );
    }

```

```

        .addGroup(layout.createSequentialGroup())
        .addGap(48, 48, 48)
        .addComponent(btnAceptar, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(btnCancelar, javax.swing.GroupLayout.PREFERRED_SIZE, 106,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addContainerGap(54, Short.MAX_VALUE))
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addComponent(pnlProvincia, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(btnAceptar)
                .addComponent(btnCancelar))
            .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
        );

    pack();
} // </editor-fold>

private void btnCancelarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    dispose();
}

private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String provincia=(String)cmbProvincias.getModel().getElementAt(cmbProvincias.getSelectedIndex());
    ((DlgDatosNovoUsuario)getParent()).establecerProvincia(provincia);
    dispose();
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(DlgProvincia.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(DlgProvincia.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(DlgProvincia.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(DlgProvincia.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
    }
} //</editor-fold>

/* Create and display the dialog */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        DlgProvincia dialog = new DlgProvincia(new javax.swing.JFrame(), true);
        dialog.addWindowListener(new java.awt.event.WindowAdapter() {
            @Override
            public void windowClosing(java.awt.event.WindowEvent e) {
                System.exit(0);
            }
        });
        dialog.setVisible(true);
    }
});

// Variables declaration - do not modify
private javax.swing.JButton btnAceptar;
private javax.swing.JButton btnCancelar;
private javax.swing.JComboBox cmbProvincias;
private javax.swing.JPanel pnlProvincia;
// End of variables declaration
private FrmPrincipal xanelaPai;
}

```

Usuario

```

/**
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

/**
 *
 * @author German DR
 */
public class Usuario {
    private int id;
}

```

```

private String nome;
private String apelidos;
private String provincia;

@Override
public String toString() {
    return "Usuario ID " + id + "- " + nome + " " + apelidos + " (" + provincia + ")";
}

public Usuario(int id, String nome, String apelidos, String provincia) {
    this.id = id;
    this.nome = nome;
    this.apelidos = apelidos;
    this.provincia = provincia;
}
}

```

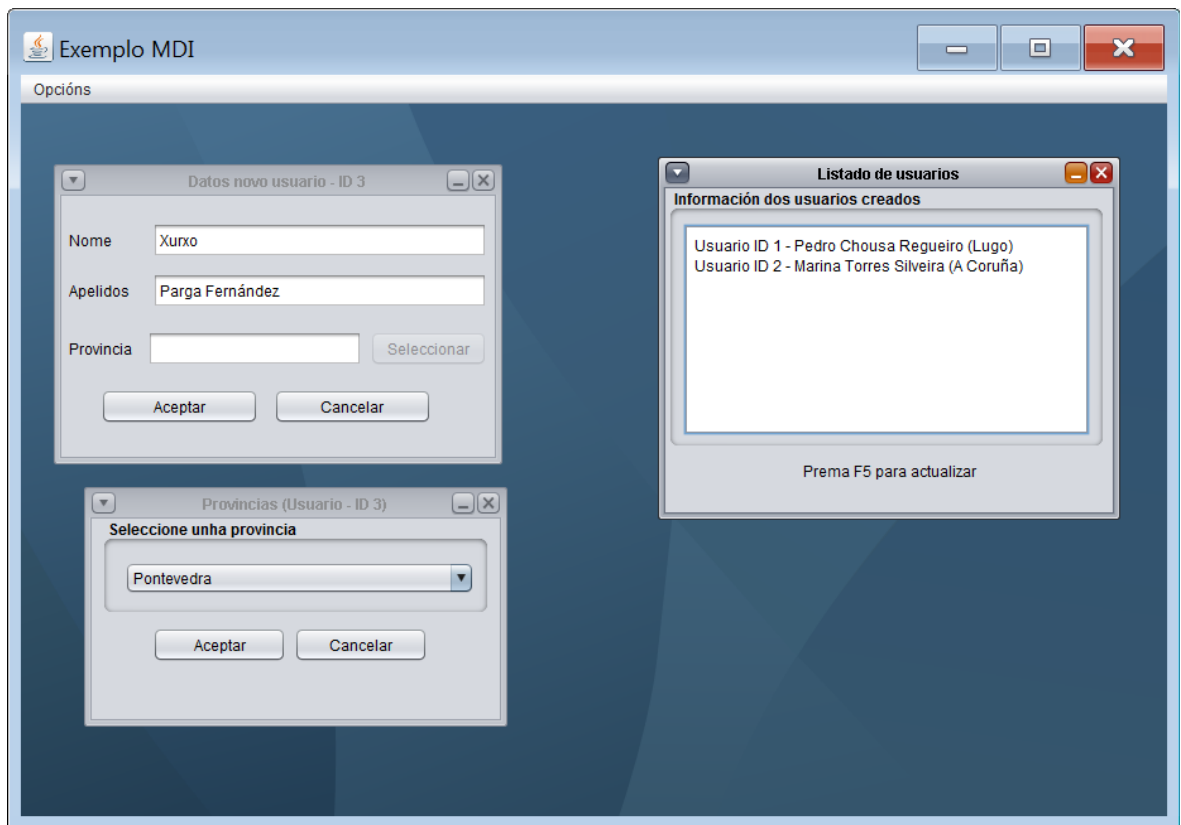


Realización da tarefa Ta1.1 - Aplicación multixanela Dialog. Nesta tarefa crearemos unha aplicación multixanela empregando técnicas multixanela Dialog.

1.2.2.2. Aplicacións multixanela MDI

Cando desenvolvemos aplicacións multixanela empregando un formulario Frame como xanela principal e diálogos para o resto de xanelas, preséntanos un pequeno inconveniente. A medida que medra o número de xanelas que temos abertas simultaneamente na nosa aplicación, o manexo do noso programa vólvese máis caótico xa que en pantalla teremos tódalas xanelas abertas da nosa aplicación sumadas a tódalas xanelas abertas que pertence a outras aplicacións que esteamos empregando. Un xeito de poñer orde dentro deste maremágnum de xanelas é desenvolver as aplicacións multixanela para que sexan aplicacións de tipo MDI. MDI quere dicir interface de múltiples documentos (multiple document interface). Cando desenvolvemos unha aplicación multixanela de tipo MDI o que facemos é reunir tódalas xanelas que a compoñen dentro dun contedor do cal non poden saírse, de xeito que calquera acción que realicemos queda acoutada a ese contedor, sendo o funcionamento das nosas aplicacións multixanela moito máis ordenado. Tipicamente unha aplicación de tipo MDI está composta por un contedor principal que conterá unha barra de menú na súa parte superior e un escritorio que ocupará o resto do contedor. A barra de menú dispón de distintas opcións que nos permiten abrir as diferentes xanelas que compoñen a nosa aplicación. Las xanelas que abrimos a través do emprego das diferentes opcións presentes na barra de menú sitúanse sobre o escritorio do contedor. O contedor encargarase da xestión das xanelas internas, de xeito que se son minimizadas, o contedor amosará unha barra de ferramentas na cal representaranse graficamente as diferentes xanelas abertas en cada momento de xeito que premendo sobre elas as poidamos restaurar ao seu tamaño orixinal. Ademais, no caso de que maximicemos unha xanela interna, o seu tamaño máximo será o do escritorio do contedor.

Graficamente o aspecto dunha aplicación MDI é o seguinte:



Como pódese observar na imaxe anterior, temos un contedor que na súa parte superior contén unha barra de menú e o resto do contedor é o seu escritorio (zona azul), o cal empregárase para a xestión das súas xanelas internas. Neste caso temos abertas tres xanelas internas que no caso de movelas nunca poderanse saír do contedor, co cal o emprego do programa será máis limpo.

Para acadar este resultado empregamos dous clases de obxectos. Por unha parte un obxecto da clase `javax.swing.JDesktopPane` que é empregado para xerar o escritorio da aplicación. Por outro lado un obxecto da clase `javax.swing.JInternalFrame` por cada unha das xanelas que contén a nosa aplicación. Realmente o que faremos será o seguinte: dentro da nosa xanela principal de tipo `JFrame` colocaremos o `JDesktopPane` e os `JInternalFrames` situaranse dentro do `JDesktopPane`.

Propiedades do contedor `JInternalFrame` empregadas máis habitualmente e que pódense establecer a través do entorno de programación:

Propiedade	Función
Closable	Mediante esta propiedade indícase se a xanela presenta un botón para pechala.
DefaultCloseOperation	Mediante esta propiedade indícase cal será o comportamento da xanela ao pechala. Pode tomar 3 valores posibles: <code>DISPOSE</code> , <code>HIDE</code> ou <code>DO_NOTHING</code> . Con <code>DISPOSE</code> péchase a xanela liberando os recursos consumidos pola mesma. Con <code>HIDE</code> a xanela unicamente será escondida. Con <code>DO_NOTHING</code> non se fará nada.
Iconifiable	Mediante esta propiedade indícase se a xanela presenta un botón para minimizala.
Maximizable	Mediante esta propiedade indícase se a xanela presenta un botón para maximizala.
Resizable	Mediante esta propiedade indícase se a xanela é redimensionable.
Title	Título da xanela.
Visible	Mediante esta propiedade indícase se a xanela é ou non visible.

Eventos do contedor JFrame empregados máis habitualmente e que pódense establecer a través do entorno de programación:

Evento	Lanzamento
InternalFrameClosed	Este evento é disparado cando pechamos a xanela xa sexa ao premer sobre a aspa da xanela ou ao pechala a través de código.

Métodos do contedor JFrame empregados máis habitualmente:

Método	Función
public void dispose()	Este método emprégase para pechar a xanela
public void setSelected(boolean selected)	Mediante este método é posible facer que a xanela pase a ser a xanela activa de entre tódalas xanelas existentes ou tamén pódese empregar para deseleccionar a xanela sobre a que se aplica o método
public void show()	Mediante este método, se a xanela non é visible, visualízase e a sitúase ao fronte de tódalas xanelas

Configuración dunha aplicación multixanela MDI empregando NetBeans

Imos desenvolver unha aplicación parecida á aplicación de xestión de usuarios que creamos para explicar o paso de información entre xanelas, pero a xestión multixanela farémola mediante MDI. Esta aplicación será a seguinte:

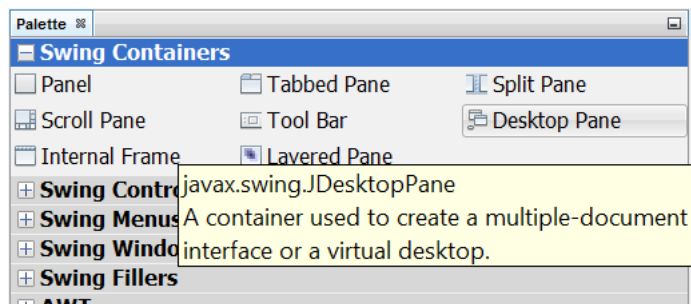


A opción de Listado de usuarios abrirá unha xanela na cal amosaranse os diferentes usuarios dados de alta na aplicación. Só poderemos ter aberta unha xanela deste tipo.

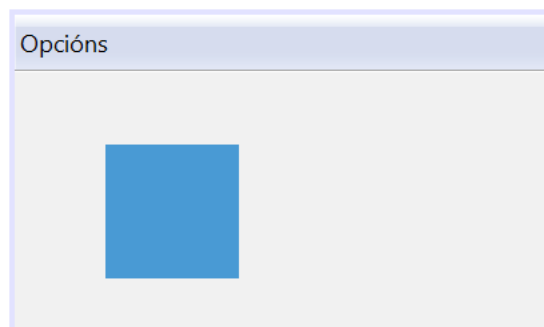
A opción Novo usuario abrirá unha xanela na cal poderemos dar de alta os datos dun novo usuario. Ademais desde esta xanela abriremos unha segunda xanela para elixir dun listado de provincias cal é a provincia do usuario.

A opción Saír finalizará a aplicación.

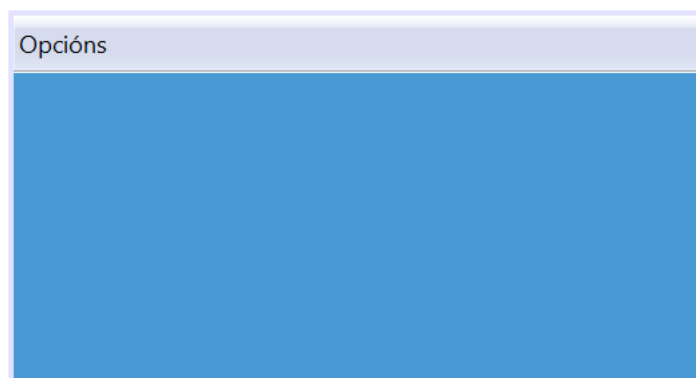
O formulario principal é un formulario de clase JFrame e o menú crearémolo como xa explicamos con anterioridade. A continuación imos ver como crear o escritorio da nosa aplicación MDI. Para elo temos que engadir un compoñente de tipo JDesktopPane no noso formulario, así que primeiramente seleccionaremos da paleta de compoñentes do entorno de programación:



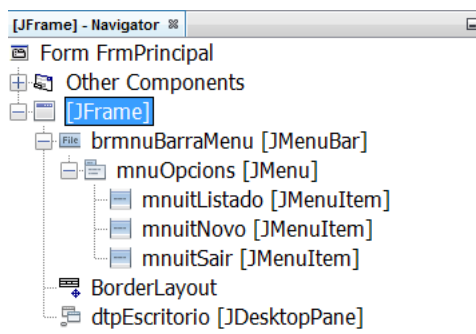
e arrastrámolo ata o formulario:



Unha vez situado dentro do formulario adaptámolo para que teña o aspecto visual que desexamos que teña. O lóxico é que o JDesktopPane ocupe todo o formulario independentemente do tamaño deste último. O xeito máis sinxelo para conseguilo é aplicar un BorderLayout ao formulario:

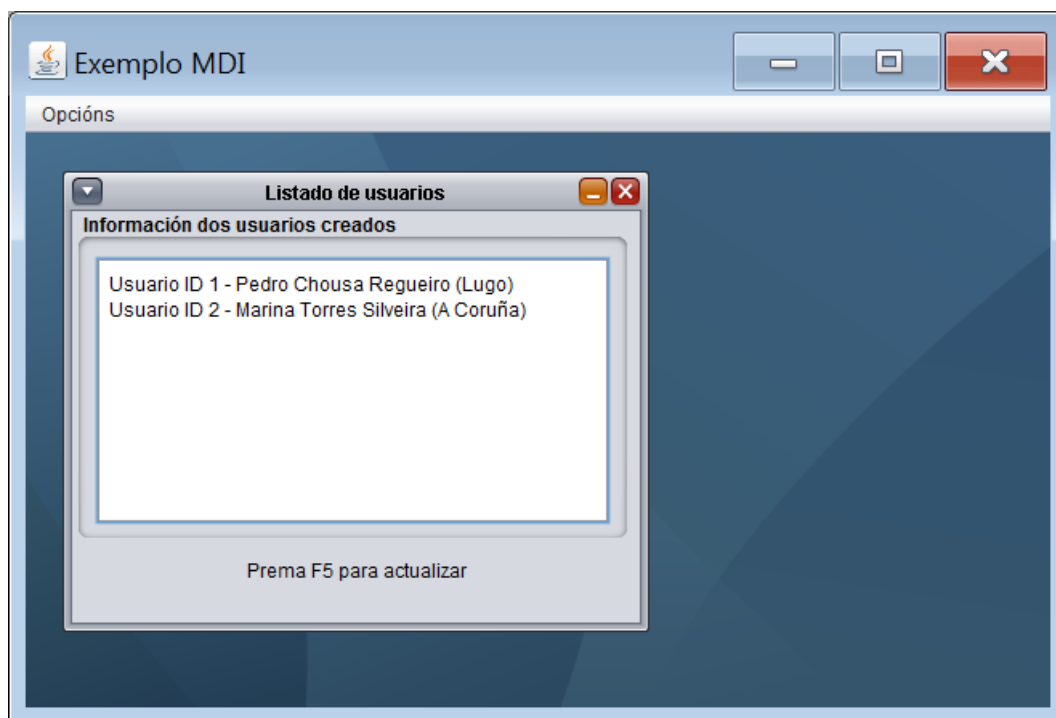


Finalmente o noso formulario principal queda configurado do seguinte xeito:



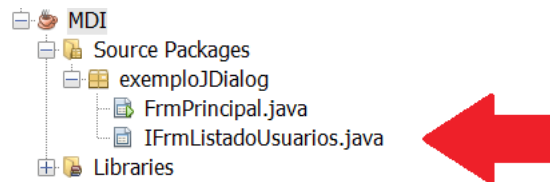
O obxecto `dpEscritorio` de clase `JDesktopPane` é o escritorio sobre o cal situaremos as diferentes xanelas que compoñen a nosa aplicación. A continuación imos ver como colocar as xanelas sobre o escritorio.

Ao premer sobre a opción de menú de Listado de usuarios abrírase unha xanela interna na cal amosarase un resumo dos usuarios que hai dados de alta na aplicación. O aspecto desta xanela interna é o seguinte:

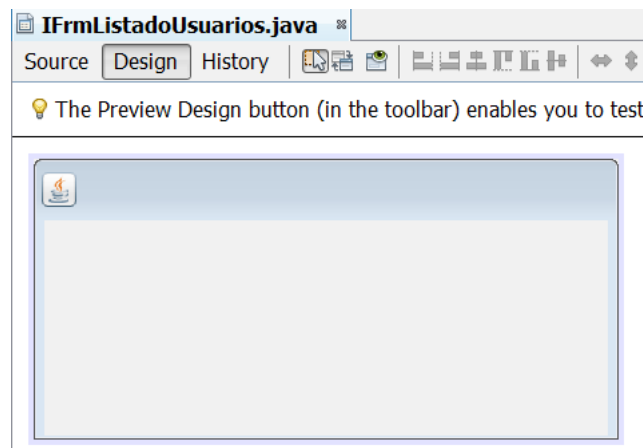


Esta xanela interna é unha xanela de clase `JInternalFrame`. Para crear este `JInternalFrame` debemos facer o seguinte: o primeiro é crear unha clase persoalizada que herde de `JInternalFrame` para definir a nosa xanela interna. O proceso é practicamente idéntico ao explicado para a creación de xanelas de tipo `JDialog`: prememos co botón dereito do rato sobre o paquete no cal queremos crear a nosa clase persoalizada. Sobre o menú despregado eliximos `New` e sobre o novo menú despregado eliximos `JInternalFrame Form`. É posible que a primeira vez que creamos un `JInternalFrame`, ao premer na opción `New` non apareza a opción `JInternalFrame Form`. Nese caso, no mesmo menú deberemos elixir unha opción etiquetada como `Other`. Ao seleccionala abríranos a pantalla que se emprega para crear Clases baseándose nos diferentes modelos que ofrece o entorno de programación. No caso que estamos desenvolvendo elixiremos dentro de `Categories` a opción `Swing GUI Forms` e en `File Types` `JInternalFrame Form`.

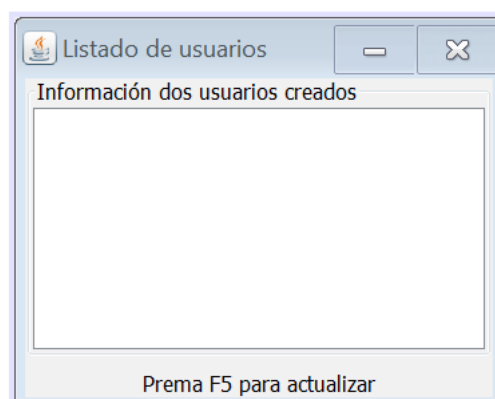
Independentemente de como creamos a xanela interna, ao seleccionar a opción `JInternalFrame Form` saltará un asistente similar ao de creación dun novo formulario para indicar o nome que lle imos dar á nosa nova clase de tipo `JInternalFrame` (chamarémola `IFrmListadoUsuarios`) e opcionalmente para indicar en que paquete queremos gardala. Unha vez que completamos o asistente, na nosa árbore de proxecto teremos unha nova clase que representara ao `JInternalFrame` que acabamos de crear:



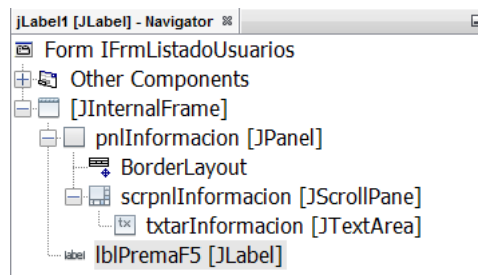
Respecto ao deseño gráfico do JInternalFrame, realízase do mesmo xeito que para un formulario de tipo JFrame. Se seleccionamos a clase que acabamos de crear (IFrmListadoUsuarios), na xanela de deseño do contedor veremos o seguinte:



Como pódese observar é un contedor baleiro. Sobre el estableceremos as súas propiedades, inseriremos os compoñentes, estableceremos os layouts necesarios e escribiremos o código necesario para os diferentes eventos que queiramos xestionar sobre os compoñentes do JInternalFrame. Resumindo, unha vez creado o noso JInternalFrame persoalizado, xestionarémolo tal e como temos feito ata agora cos nosos formularios. Polo tanto, o deseño visual da xanela interna IFrmListadoUsuarios quedará configurado do seguinte xeito:



Na seguinte imaxe amósase a composición gráfica da xanela interna IFrmListadoUsuarios:



A continuación imos ver que é o que temos que facer para que se amose a xanela interna ao premer sobre a opción de menú Listado de usuarios. Para amosar a xanela interna implementamos o `actionPerformed` da opción de menú Listado de usuarios, sendo o código resultante o seguinte:

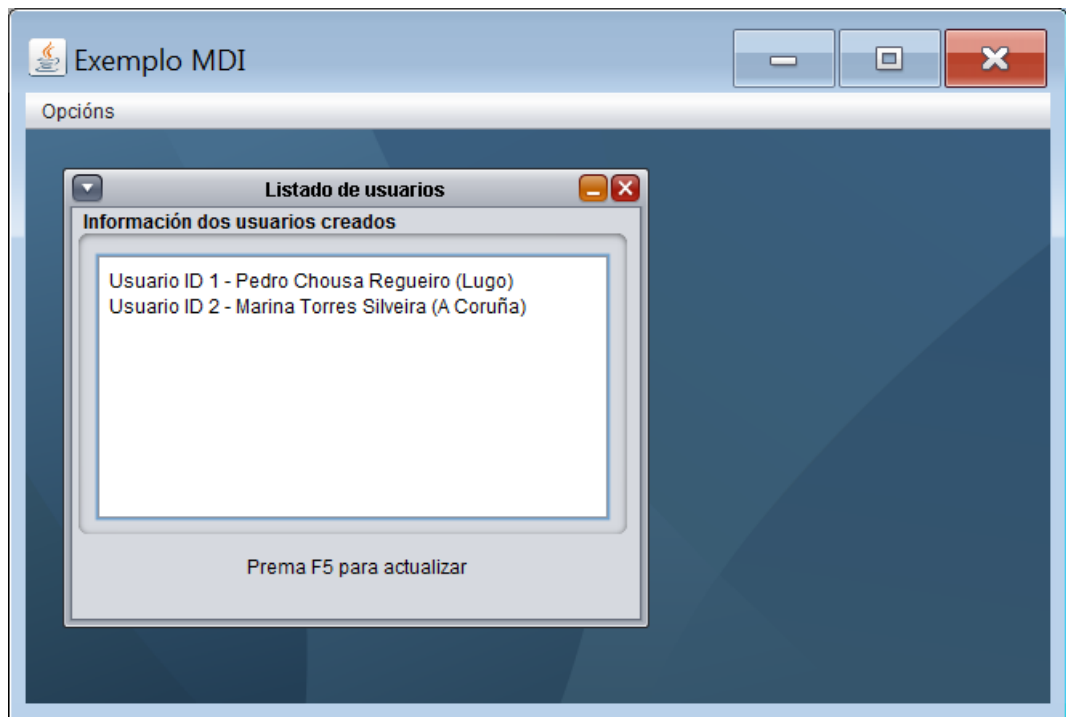
```
private void mnuitListadoActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    switch(XestorDeXanelas.podeseAbrirIFrmListadoUsuarios())
    {
        case 0: IFrmListadoUsuarios iFrmListadoUsuarios=new IFrmListadoUsuarios();
                dtpEscritorio.add(iFrmListadoUsuarios);
                iFrmListadoUsuarios.show();
                XestorDeXanelas.abrirIFrmListadoUsuarios();
                break;
        case 1: break;
        case 2: System.out.println("ddd");
                JOptionPane.showMessageDialog(this, "Non hai usuarios na base de datos");
                return;
    }
}
```

Como dixéramos anteriormente, unicamente podemos ter aberta á vez unha xanela deste tipo, polo tanto realizamos este control a través dun xestor de xanelas (ver clase `XestorDeXanelas`) dun xeito parecido ao explicado para os diálogos. Nas liñas que acabamos de expoñer, unicamente creárase unha nova xanela no caso de que entremos pola opción 0 do switch, polo tanto centrémonos nesas liñas:

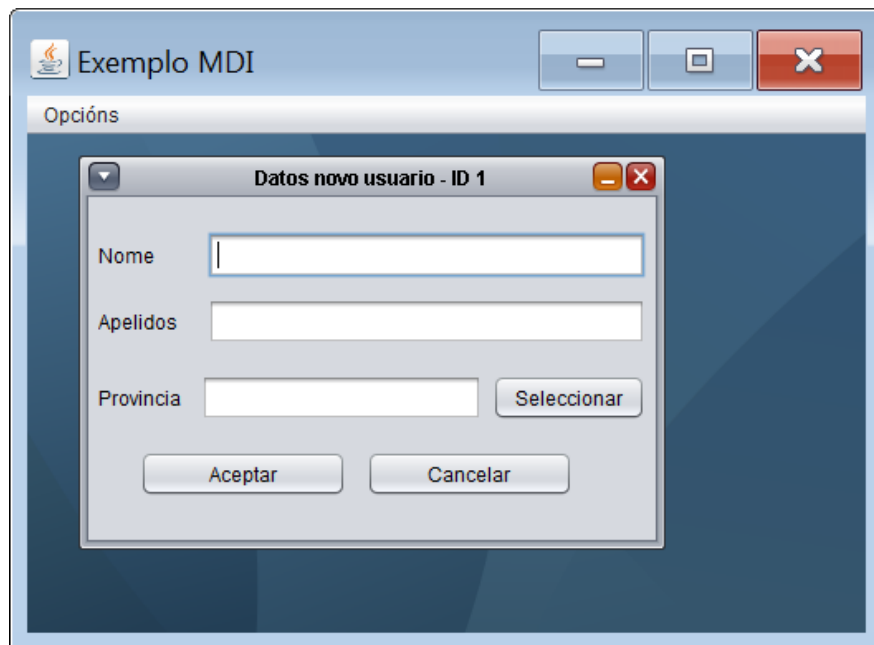
```
IFrmListadoUsuarios iFrmListadoUsuarios=new IFrmListadoUsuarios();
dtpEscritorio.add(iFrmListadoUsuarios);
iFrmListadoUsuarios.show();
XestorDeXanelas.abrirIFrmListadoUsuarios();
```

Na primeira liña, o que facemos é instanciar un obxecto da clase `IFrmListadoUsuarios` empregando o seu construtor. A continuación engadimos o obxecto que acabamos de crear ao noso escritorio mediante o método `add` do `JDesktopPane`. Neste momento a xanela interna está creada e situada sobre o `JDesktopPane`, pero non se visualizará (a menos que teñamos activada a súa propiedade visible con anterioridade). Por ultimo, para que a xanela interna sexa visible empregaremos sobre a xanela interna que acabamos de crear o seu método `show` o cal fará que sexa visualizable dentro do escritorio da aplicación MDI.

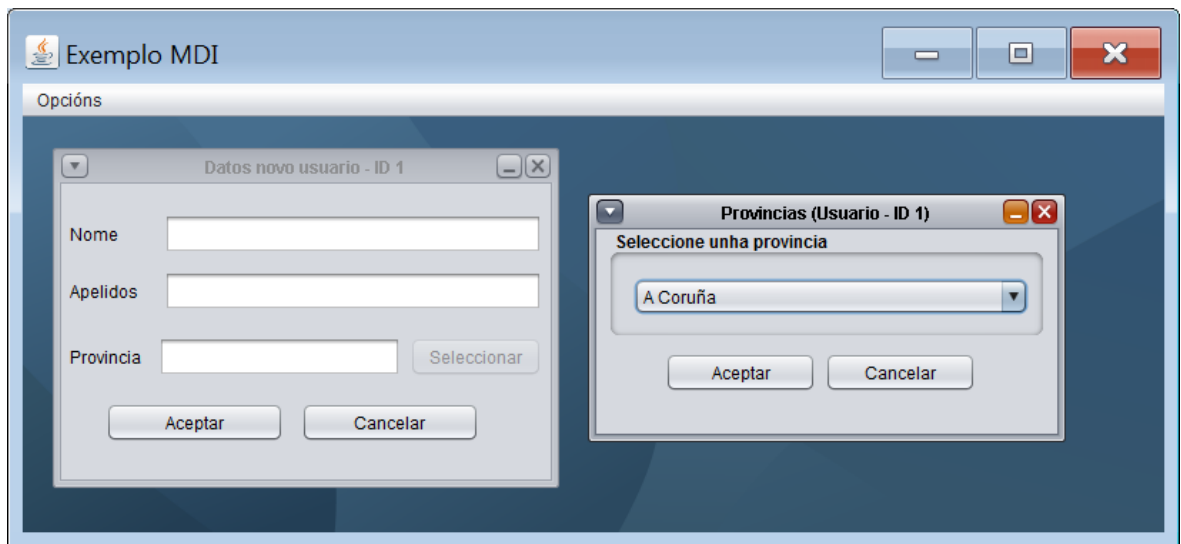
Se executamos a aplicación e pulsamos sobre a opción de menú Listado de usuarios este será o resultado:



A opción de menú Novo usuario abrirá unha xanela interna na cal poderase dar de alta un novo usuario cuxo identificador será o amosado entre paréntese na opción de menú seleccionada. Nel título da xanela interna amosarase o identificador do usuario que imos crear (esta información pásase á xanela interna desde o formulario principal da aplicación). O aspecto da xanela interna será o seguinte:



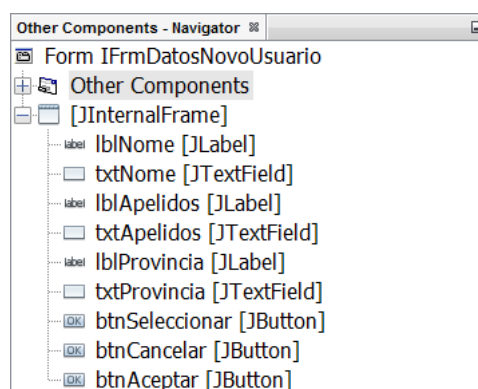
Na xanela interna aberta daranse de alta os datos do novo usuario, non obstante, para indicar a provincia non é posible escribir na caixa de texto adxunta, senón que hai que seleccionar a provincia dun combo de provincias. Este combo con provincias atópase noutra xanela interna que se abrirá ao premer sobre o botón seleccionar. O aspecto da xanela interna na cal seleccionaremos a provincia é o seguinte:



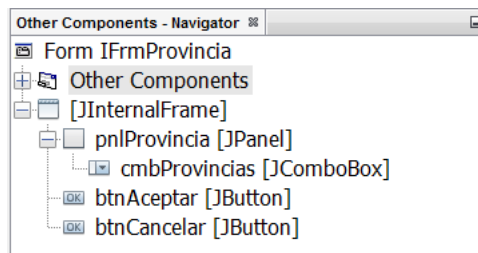
Cando a xanela interna de provincias é aberta desde a xanela interna de Datos novo usuario, esta última pásalle o identificador do usuario que se está creando para que o amose na súa barra de título. Unha vez seleccionada a provincia, ao premer sobre o botón Aceptar da xanela interna de provincias, o valor seleccionado no combo envíase á xanela interna de Datos novo usuario para ser amosado na súa caixa de texto Provincia. Se prememos sobre o botón Aceptar da xanela interna de Datos novo usuario, no caso de que a validación sexa correcta crearase un obxecto de clase Usuario (ver código clase Usuario) que encapsula a información e será enviado á clase AlmacenDeUsuarios (ver código clase AlmacenDeUsuarios). A clase AlmacenDeUsuarios simplemente simula unha pequena base de datos para xestionar a información dos usuarios que imos creando durante a execución do programa (non empregamos unha base de datos real xa que non é parte desta actividade).

As xanelas internas descritas créanse do mesmo xeito que explicamos anteriormente para a xanela interna IFrmListadoUsuarios. Neste caso crearemos as xanelas internas IFrmDatosNovoUsuario (para a alta de usuarios) e IFrmProvincia (para a selección da provincia).

A composición gráfica da xanela interna IFrmDatosNovoUsuario será a seguinte:



mentres que a composición gráfica da xanela interna IFrmProvincia será esta outra:



Imos ver a continuación como creamos a xanela interna IFrmDatosNovoUsuario ao seleccionar a opción de menú Novo usuario. Para elo implementamos o actionPerformed da opción de menú Novo usuario, sendo o código resultante o seguinte:

```
private void mnuitNovoActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    IFrmDatosNovoUsuario iFrmDatosNovoUsuario=new IFrmDatosNovoUsuario(idUsuario);
    dtpEscritorio.add(iFrmDatosNovoUsuario);
    iFrmDatosNovoUsuario.show();
    pintarMenuItemNovo();
}
```

Na primeira liña, o que facemos é instanciar un obxecto da clase IFrmDatosNovoUsuario empregando o seu construtor. É importante salientar que o construtor ofrecido polo entorno de programación foi modificado para pasar á xanela interna de clase IFrmDatosNovoUsuario o identificador do usuario que vai ser creado (empregamos a técnica de paso de información a través do construtor vista nos diálogos). A continuación engadimos o obxecto que acabamos de crear ao noso escritorio mediante o método add do JDesktopPane. Neste momento a xanela interna está creada e situada sobre o JDesktopPane, pero non se visualizará (a menos que teñamos activada a súa propiedade visible). Por último, para que a xanela interna sexa visible empregamos sobre a xanela interna que acabamos de crear o seu método show, o cal a fará visible dentro do escritorio da aplicación MDI.

Para ver como creamos a xanela de clase IFrmProvincia, temos que estudar o código implementado para o actionPerformed do botón btnSeleccionar:

```
private void btnSeleccionarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    filla=new IFrmProvincia(idUsuario, this);
    getParent().add(filla);
    filla.show();
    btnSeleccionar.setEnabled(false);
}
```

O obxecto filla é un atributo a nivel de clase de tipo IFrmProvincia. É empregado polo formulario de clase IFrmDatosNovoUsuario para ter unha referencia sobre a xanela de provincias. Máis adiante veremos para que é necesario o obxecto filla. De momento sigamos analizando o código do actionPerformed do botón btnSeleccionar. Na primeira liña, o que facemos é instanciar un obxecto da clase IFrmProvincia empregando o seu construtor. É importante salientar que o construtor ofrecido polo entorno de programación foi modificado para pasar á xanela interna de clase IFrmProvincia dous informacións. A primeira é o identificador do usuario que vai ser creado. A segunda é unha referencia ao propio formulario que crea a xanela interna de provincias, de xeito que a xanela de provincias teña unha referencia para acceder á xanela que creouna e facer emprego dos seus métodos públicos (para ambas informacións empregamos a técnica de paso de información a través do construtor vista nos diálogos). A continuación engadimos o obxecto que acabamos de crear ao noso escritorio mediante o método add do JDesktopPane. Fixémonos que como desde a xanela interna da clase IFrmDatosNovoUsuario non temos visibilidade sobre o obxecto que referencia ao escritorio da aplicación MDI, o xeito de acadala é empregando o método getParent da clase IFrmDatosNovoUsuario. Este método devolveranos quen é o pai gráfico da xanela interna de clase IFrmDatosNovoUsuario, neste caso o escritorio da aplicación MDI no cal está situada a xanela interna de clase IFrmDatosNovoUsuario. Neste momento a xanela interna está creada e situada sobre o JDesktopPane, pero non se

visualizará (a menos que teñamos activada a súa propiedade visible). Por último, para que a xanela interna sexa visible empregaremos o seu método show o cal a fará visible dentro do escritorio da aplicación MDI.

Unha vez que na xanela interna de clase IFrmProvincia seleccionamos unha provincia e prememos sobre o botón aceptar, envíase a información do elemento seleccionado á xanela interna de clase IFrmDatosNovoUsuario que creou a xanela interna de clase IFrmProvincia. Para elo, o código implementado no botón Aceptar da xanela interna de clase IFrmProvincia é o seguinte:

```
private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    String provincia=(String)cmbProvincias.getModel().getElementAt(cmbProvincias.getSelectedIndex());  
    xanelaChamadora.establecerProvincia(provincia);  
    dispose();  
}
```

Empregamos o obxecto xanelaChamadora para invocar a un método público da xanela interna de clase IFrmDatosNovoUsuario que creou á actual xanela interna de clase IFrmProvincia (empregamos a técnica vista nos diálogos para invocar métodos públicos de outras xanelas). Este método encargarase de amosar na caixa de texto txtProvincia a provincia seleccionada no combo cmbProvincias. O obxecto xanelaChamadora ten unha referencia á xanela interna de clase IFrmDatosNovoUsuario que creou á actual xanela interna de clase IFrmProvincia. Esta referencia, como viuse anteriormente, foi pasada a través do construtor da xanela interna de clase IFrmProvincia.

Unha vez que enchamos tódolos datos da xanela interna da clase IFrmDatosNovoUsuario e premamos sobre o seu botón Aceptar, validase a información, créase un obxecto de clase Usuario con ela e envíase á clase AlmacenDeUsuarios. Por ultimo faise un dispose do formulario para pechalo:

```
private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    String nome=txtNome.getText().trim();  
    String apellidos=txtApellidos.getText().trim();  
    String provincia=txtProvincia.getText().trim();  
  
    if(nome.compareTo("")==0)  
    {  
        JOptionPane.showMessageDialog(this, "Debe indicar o nome do usuario");  
        return;  
    }  
    if(apellidos.compareTo("")==0)  
    {  
        JOptionPane.showMessageDialog(this, "Debe indicar os apellidos do usuario");  
        return;  
    }  
    if(provincia.compareTo("")==0)  
    {  
        JOptionPane.showMessageDialog(this, "Debe indicar a provincia do usuario");  
        return;  
    }  
  
    Usuario usuario=new Usuario(idUsuario, nome, apellidos, provincia);  
    AlmacenDeUsuarios.anhadirUsuario(usuario);  
    dispose();  
}
```

Non obstante, se prememos sobre o botón Cancelar da xanela interna de clase IFrmDatosNovoUsuario unicamente faise un dispose para pechalo:

```
private void btnCancelarActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    dispose();  
}
```

Nos dous casos executamos un dispose. Esta liña ao igual que ocorre nos diálogos e tamén nos formularios é equivalente a premer no botón de pechar da xanela e o mesmo que esta ultima acción, desencadea o evento internalFrameClosed. Neste caso implementamos este evento do seguinte xeito:

```
private void formInternalFrameClosed(javax.swing.event.InternalFrameEvent evt) {  
    // TODO add your handling code here:  
    if(filla!=null) filla.dispose();  
}
```

Tal e como comentamos antes, o obxecto filla é un atributo a nivel de clase de tipo IFrmProvincia empregado polo formulario de clase IFrmDatosNovoUsuario para ter unha referencia sobre a xanela de provincias. Imaxinemos a seguinte situación: abrimos a xanela interna para crear un novo usuario. Prememos no seu botón seleccionar de xeito que tamén abrimos a xanela interna de provincias. Agora, xa sexa premendo sobre o botón Cancelar da xanela de novo usuario ou sobre o seu botón de pechar, pechamos a xanela interna. Se non implementamos algún código adicional pecharíase a xanela interna de novo usuario pero a súa xanela interna de provincias seguiría estando visualizable sobre o JDesktopPane da aplicación MDI. O que facemos mediante as liñas implementadas a través do evento internaFrameClosed da clase IFrmDatosNovoUsuario é precisamente evitar que isto ocorra.

A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación:

```
FrmPrincipal

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class FrmPrincipal extends javax.swing.JFrame {

    /**
     * Creates new form FrmPrincipal
     */
    public FrmPrincipal() {
        initComponents();
        //setExtendedState(MAXIMIZED_BOTH);
        pintarMenuItemNovo();
    }

    private void pintarMenuItemNovo()
    {
        idUsuario++;
        mnuitNovo.setText("Novo usuario (ID - "+idUsuario+")");
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        dtpEscritorio = new javax.swing.JDesktopPane();
        brmnvBarraMenu = new javax.swing.JMenuBar();
        mnuOpciones = new javax.swing.JMenu();
        mnuitListado = new javax.swing.JMenuItem();
        mnuitNovo = new javax.swing.JMenuItem();
        mnuitSair = new javax.swing.JMenuItem();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Exemplo MDI");
        getContentPane().add(dtpEscritorio, java.awt.BorderLayout.CENTER);

        mnuOpciones.setText("Opciones");

        mnuitListado.setText("Listado de usuarios");
        mnuitListado.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                mnuitListadoActionPerformed(evt);
            }
        });
        mnuOpciones.add(mnuitListado);

        mnuitNovo.setText("Novo usuario");
        mnuitNovo.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                mnuitNovoActionPerformed(evt);
            }
        });
        mnuOpciones.add(mnuitNovo);

        mnuitSair.setText("Sair");
        mnuitSair.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                mnuitSairActionPerformed(evt);
            }
        });
        mnuOpciones.add(mnuitSair);

        brmnvBarraMenu.add(mnuOpciones);

        setJMenuBar(brmnvBarraMenu);
    }
}
```



```

        java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
        setBounds((screenSize.width-968)/2, (screenSize.height-531)/2, 968, 531);
    } // </editor-fold>

    private void mnuitSairActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        System.exit(0);
    }

    private void mnuitNovoActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        IFrmDadosNovoUsuario iFrmDadosNovoUsuario=new IFrmDadosNovoUsuario(idUsuario);
        dtpEscritorio.add(iFrmDadosNovoUsuario);
        iFrmDadosNovoUsuario.show();
        pintarMenuItemNovo();
    }

    private void mnuitListadoActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        switch(XestorDeXanelas.podeseAbrirIFrmListadoUsuarios())
        {
            case 0: IFrmListadoUsuarios iFrmListadoUsuarios=new IFrmListadoUsuarios();
                    dtpEscritorio.add(iFrmListadoUsuarios);
                    iFrmListadoUsuarios.show();
                    XestorDeXanelas.abrirIFrmListadoUsuarios();
                    break;
            case 1: break;
            case 2: System.out.println("ddd");
                    JOptionPane.showMessageDialog(this, "Non hai usuarios na base de datos");
                    return;
        }
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
         * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (InstantiationException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (IllegalAccessException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {
            java.util.logging.Logger.getLogger(FrmPrincipal.class.getName()).log(java.util.logging.Level.SEVERE, null,
ex);
        }
        //</editor-fold>

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new FrmPrincipal().setVisible(true);
            }
        });
    }
    // Variables declaration - do not modify
    private javax.swing.JMenuBar brmnvBarraMenu;
    private javax.swing.JDesktopPane dtpEscritorio;
    private javax.swing.JMenu mnuOpciones;
    private javax.swing.JMenuItem mnuitListado;
    private javax.swing.JMenuItem mnuitNovo;
    private javax.swing.JMenuItem mnuitSair;
    // End of variables declaration
    private int idUsuario=0;
}

```

IFrmListadoUsuarios

```

/**
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

import java.awt.event.KeyEvent;
import java.util.Vector;

/**
 *
 * @author German DR
 */
public class IFrmListadoUsuarios extends javax.swing.JInternalFrame {

    /**
     * Creates new form IFrmListadoUsuarios
     */
}

```

```

public IFrmListadoUsuarios() {
    initComponents();
    actualizarInformacion();
}

private void actualizarInformacion()
{
    Vector usuarios=AlmacenDeUsuarios.recuperarUsuarios();
    String resultado="";
    for(int i=0;i<usuarios.size();i++)
    {
        resultado+=(Usuario)usuarios.elementAt(i)+"\n";
    }

    txtarInformacion.setText(resultado);
}

/**
 * This method is called from within the constructor to initialize the form.
 * WARNING: Do NOT modify this code. The content of this method is always
 * regenerated by the Form Editor.
 */
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    pnlInformacion = new javax.swing.JPanel();
    scrpnlInformacion = new javax.swing.JScrollPane();
    txtarInformacion = new javax.swing.JTextArea();
    lblPremaF5 = new javax.swing.JLabel();

    setClosable(true);
    setIconifiable(true);
    setTitle("Listado de usuarios");
    addInternalFrameListener(new javax.swing.event.InternalFrameListener() {
        public void internalFrameClosing(javax.swing.event.InternalFrameEvent evt) {
        }
        public void internalFrameActivated(javax.swing.event.InternalFrameEvent evt) {
        }
        public void internalFrameClosed(javax.swing.event.InternalFrameEvent evt) {
            formInternalFrameClosed(evt);
        }
        public void internalFrameDeactivated(javax.swing.event.InternalFrameEvent evt) {
        }
        public void internalFrameDeiconified(javax.swing.event.InternalFrameEvent evt) {
        }
        public void internalFrameIconified(javax.swing.event.InternalFrameEvent evt) {
        }
        public void internalFrameOpened(javax.swing.event.InternalFrameEvent evt) {
        }
    });

    pnlInformacion.setBorder(javax.swing.BorderFactory.createTitledBorder("Información dos usuarios creados"));
    pnlInformacion.setLayout(new java.awt.BorderLayout());

    txtarInformacion.setEditable(false);
    txtarInformacion.setColumns(20);
    txtarInformacion.setRows(5);
    txtarInformacion.addKeyListener(new java.awt.event.KeyAdapter() {
        public void keyPressed(java.awt.event.KeyEvent evt) {
            txtarInformacionKeyPressed(evt);
        }
    });
    scrpnlInformacion.setViewportView(txtarInformacion);

    pnlInformacion.add(scrpnlInformacion, java.awt.BorderLayout.CENTER);

    lblPremaF5.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
    lblPremaF5.setText("Prema F5 para actualizar");

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addComponent(pnlInformacion, javax.swing.GroupLayout.PREFERRED_SIZE, 350,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(0, 0, Short.MAX_VALUE)
                .addComponent(lblPremaF5, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
                    Short.MAX_VALUE)
            );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addComponent(pnlInformacion, javax.swing.GroupLayout.PREFERRED_SIZE, 210,
                    javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                .addComponent(lblPremaF5)
            );

    java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
    setBounds((screenSize.width-366)/2, (screenSize.height-288)/2, 366, 288);
}

private void txtarInformacionKeyPressed(java.awt.event.KeyEvent evt) {
    // TODO add your handling code here:
    if(evt.getKeyCode()==KeyEvent.VK_F5) actualizarInformacion();
}

private void formInternalFrameClosed(javax.swing.event.InternalFrameEvent evt) {
    // TODO add your handling code here:
    XestorDeXanelas.cerrarIFrmListadoUsuarios();
}

// Variables declaration - do not modify

```

```

private javax.swing.JLabel lblPrensaF5;
private javax.swing.JPanel pnlInformacion;
private javax.swing.JScrollPane scrpnlInformacion;
private javax.swing.JTextArea txtarInformacion;
// End of variables declaration
}

```

IFrmDatosNovoUsuario

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialo;

import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class IFrmDatosNovoUsuario extends javax.swing.JInternalFrame {

    /**
     * Creates new form IFrmDatosNovoUsuario
     */
    public IFrmDatosNovoUsuario(int id) {
        initComponents();
        idUsuario=id;
        setTitle("Datos novo usuario - ID "+idUsuario);
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        lblNome = new javax.swing.JLabel();
        txtNome = new javax.swing.JTextField();
        lblApellidos = new javax.swing.JLabel();
        txtApellidos = new javax.swing.JTextField();
        lblProvincia = new javax.swing.JLabel();
        txtProvincia = new javax.swing.JTextField();
        btnSeleccionar = new javax.swing.JButton();
        btnCancelar = new javax.swing.JButton();
        btnAceptar = new javax.swing.JButton();

        setClosable(true);
        setIconifiable(true);
        setVisible(false);
        addInternalFrameListener(new javax.swing.event.InternalFrameListener() {
            public void internalFrameClosing(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameActivated(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameClosed(javax.swing.event.InternalFrameEvent evt) {
                formInternalFrameClosed(evt);
            }
            public void internalFrameDeactivated(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameDeiconified(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameIconified(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameOpened(javax.swing.event.InternalFrameEvent evt) {
            }
        });

        lblNome.setText("Nome");

        lblApellidos.setText("Apellidos");

        lblProvincia.setText("Provincia");

        txtProvincia.setEditable(false);

        btnSeleccionar.setText("Seleccionar");
        btnSeleccionar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSeleccionarActionPerformed(evt);
            }
        });

        btnCancelar.setText("Cancelar");
        btnCancelar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnCancelarActionPerformed(evt);
            }
        });

        btnAceptar.setText("Aceptar");
        btnAceptar.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnAceptarActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
    }

```

```

        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(layout.createSequentialGroup())
                    .addContainerGap()
                    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                        .addGroup(layout.createSequentialGroup())
                            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                                .addComponent(lblNome)
                                .addComponent(lblApellidos))
                            .addGap(18, 18, 18)
                            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                                .addComponent(txtNome)
                                .addComponent(txtApellidos)))
                        .addGroup(layout.createSequentialGroup())
                            .addComponent(lblProvincia)
                            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                                .addComponent(txtProvincia, javax.swing.GroupLayout.PREFERRED_SIZE, 170,
javax.swing.GroupLayout.PREFERRED_SIZE)
                                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                                    .addComponent(btnSeleccionar)
                                    .addGap(0, 0, Short.MAX_VALUE)))
                    .addContainerGap()
                .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, layout.createSequentialGroup())
                    .addGap(0, 0, Short.MAX_VALUE)
                        .addComponent(btnAceptar, javax.swing.GroupLayout.PREFERRED_SIZE, 125,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                            .addComponent(btnCancelar, javax.swing.GroupLayout.PREFERRED_SIZE, 125,
javax.swing.GroupLayout.PREFERRED_SIZE)
                            .addGap(50, 50, 50))
        );
        layout.setVerticalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(layout.createSequentialGroup())
                    .addGap(21, 21, 21)
                    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                        .addComponent(lblNome)
                        .addComponent(txtNome, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
                    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                    .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                        .addComponent(lblApellidos)
                        .addComponent(txtApellidos, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))
                        .addGap(18, 18, 18)
                        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                            .addComponent(lblProvincia)
                            .addComponent(txtProvincia, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                            .addComponent(btnSeleccionar)
                            .addGap(18, 18, 18)
                            .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
                                .addComponent(btnCancelar)
                                .addComponent(btnAceptar))
                            .addContainerGap(36, Short.MAX_VALUE))
                    );
        pack();
    } // </editor-fold>

    private void btnSeleccionarActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        filla=new IFrmProvincia(idUsuario, this);
        getParent().add(filla);
        filla.show();
        btnSeleccionar.setEnabled(false);
    }

    public void establecerProvincia(String provincia)
    {
        if(provincia!=null) txtProvincia.setText(provincia);
        btnSeleccionar.setEnabled(true);
    }

    private void btnCancelarActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        dispose();
    }

    private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        String nome=txtNome.getText().trim();
        String apellidos=txtApellidos.getText().trim();
        String provincia=txtProvincia.getText().trim();

        if(nome.compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this, "Debe indicar o nome do usuario");
            return;
        }
        if(apellidos.compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this, "Debe indicar os apellidos do usuario");
            return;
        }
        if(provincia.compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this, "Debe indicar a provincia do usuario");
            return;
        }

        Usuario usuario=new Usuario(idUsuario, nome, apellidos, provincia);
        AlmacenDeUsuarios.anhadirUsuario(usuario);
    }

```

```

        dispose();
    }

    private void formInternalFrameClosed(javax.swing.event.InternalFrameEvent evt) {
        // TODO add your handling code here:
        if(filla!=null) filla.dispose();
    }

    // Variables declaration - do not modify
    private javax.swing.JButton btnAceptar;
    private javax.swing.JButton btnCancelar;
    private javax.swing.JButton btnSeleccionar;
    private javax.swing.JLabel lblApellidos;
    private javax.swing.JLabel lblNome;
    private javax.swing.JLabel lblProvincia;
    private javax.swing.JTextField txtApellidos;
    private javax.swing.JTextField txtNome;
    private javax.swing.JTextField txtProvincia;
    // End of variables declaration
    private int idUsuario;
    private IFrmProvincia filla;
}

```

IFrmProvincia

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

/**
 *
 * @author German DR
 */
public class IFrmProvincia extends javax.swing.JInternalFrame {

    /**
     * Creates new form IFrmProvincia
     */
    public IFrmProvincia(int id, IFrmDatosNovoUsuario chamador) {
        initComponents();
        idUsuario=id;
        xanelaChamadora=chamador;
        setTitle("Provincias (Usuario - ID "+idUsuario+")");
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        pnlProvincia = new javax.swing.JPanel();
        cmbProvincias = new javax.swing.JComboBox();
        btnAceptar = new javax.swing.JButton();
        btnCancelar = new javax.swing.JButton();

        setClosable(true);
        setIconifiable(true);
        setTitle("Provincias");
        addInternalFrameListener(new javax.swing.event.InternalFrameListener() {
            public void internalFrameClosing(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameActivated(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameClosed(javax.swing.event.InternalFrameEvent evt) {
                formInternalFrameClosed(evt);
            }
            public void internalFrameDeactivated(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameDeiconified(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameIconified(javax.swing.event.InternalFrameEvent evt) {
            }
            public void internalFrameOpened(javax.swing.event.InternalFrameEvent evt) {
            }
        });

        pnlProvincia.setBorder(javax.swing.BorderFactory.createTitledBorder("Seleccione unha provincia"));

        cmbProvincias.setModel(new javax.swing.DefaultComboBoxModel(new String[] { "A Coruña", "Lugo", "Ourense", "Pontevedra" }));

        javax.swing.GroupLayout pnlProvinciaLayout = new javax.swing.GroupLayout(pnlProvincia);
        pnlProvincia.setLayout(pnlProvinciaLayout);
        pnlProvinciaLayout.setHorizontalGroup(
            pnlProvinciaLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(pnlProvinciaLayout.createSequentialGroup()
                    .addGroup(pnlProvinciaLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                        .add(cmbProvincias)
                        .add(btnAceptar)
                        .add(btnCancelar)
                    )
                    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
        );
        pnlProvinciaLayout.setVerticalGroup(
            pnlProvinciaLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(pnlProvinciaLayout.createSequentialGroup()
                    .add(cmbProvincias)
                    .add(btnAceptar)
                    .add(btnCancelar)
                    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
        );
    }
}

```

```

    );

    btnAceptar.setText("Aceptar");
    btnAceptar.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnAceptarActionPerformed(evt);
        }
    });

    btnCancelar.setText("Cancelar");
    btnCancelar.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnCancelarActionPerformed(evt);
        }
    });

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addComponent(pnlProvincia, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addContainerGap()
                .addComponent(btnAceptar, javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE)
                .addContainerGap()
                .addComponent(btnCancelar, javax.swing.GroupLayout.PREFERRED_SIZE, 106, javax.swing.GroupLayout.PREFERRED_SIZE)
            )
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addComponent(pnlProvincia, javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE)
                .addContainerGap()
                .addComponent(btnAceptar)
                .addContainerGap()
                .addComponent(btnCancelar)
            )
    );

    java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
    setBounds((screenSize.width-335)/2, (screenSize.height-174)/2, 335, 174);
}

private void btnAceptarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String provincia=(String)cmbProvincias.getModel().getElementAt(cmbProvincias.getSelectedIndex());
    xanelaChamadora.establecerProvincia(provincia);
    dispose();
}

private void btnCancelarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    dispose();
}

private void formInternalFrameClosed(javax.swing.event.InternalFrameEvent evt) {
    // TODO add your handling code here:
    xanelaChamadora.establecerProvincia(null);
}

// Variables declaration - do not modify
private javax.swing.JButton btnAceptar;
private javax.swing.JButton btnCancelar;
private javax.swing.JComboBox cmbProvincias;
private javax.swing.JPanel pnlProvincia;
// End of variables declaration
private int idUsuario;
private IFrmDatosNovoUsuario xanelaChamadora;
}

```

XestorDeXanelas

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

/**
 *
 * @author German DR
 */
public class XestorDeXanelas {
    static boolean iFrmListadoUsuariosAberta=false;

    public static int podeseAbrirIFrmListadoUsuarios()
    {
        if(iFrmListadoUsuariosAberta)
        {
            return 1;
        }
        else
        {
            if(AlmacenDeUsuarios.isBaleiroAlmacenDeDatos())
            {
                return 2;
            }
        }
    }
}

```

```

        else
        {
            return 0;
        }
    }

    public static void abrirIFrmListadoUsuarios()
    {
        iFrmListadoUsuariosAberta=true;
    }

    public static void cerrarIFrmListadoUsuarios()
    {
        iFrmListadoUsuariosAberta=false;
    }
}

```

```

Usuario

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

/**
 *
 * @author German DR
 */
public class Usuario {
    private int id;
    private String nome;
    private String apelidos;
    private String provincia;

    @Override
    public String toString() {
        return "Usuario ID " + id + " - "+nome + " " + apelidos + " (" + provincia + ")";
    }

    public Usuario(int id, String nome, String apelidos, String provincia) {
        this.id = id;
        this.nome = nome;
        this.apelidos = apelidos;
        this.provincia = provincia;
    }

}

AlmacenDeUsuarios
/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemploJDialog;

import java.util.Vector;

/**
 *
 * @author German DR
 */
public class AlmacenDeUsuarios {
    private static Vector usuarios=new Vector();

    public static boolean isBaleiroAlmacenDeDatos()
    {
        if(usuarios.size()==0) return true;
        else return false;
    }

    public static void anhadirUsuario(Usuario usuario)
    {
        usuarios.add(usuario);
    }

    public static Vector recuperarUsuarios()
    {
        return usuarios;
    }

}

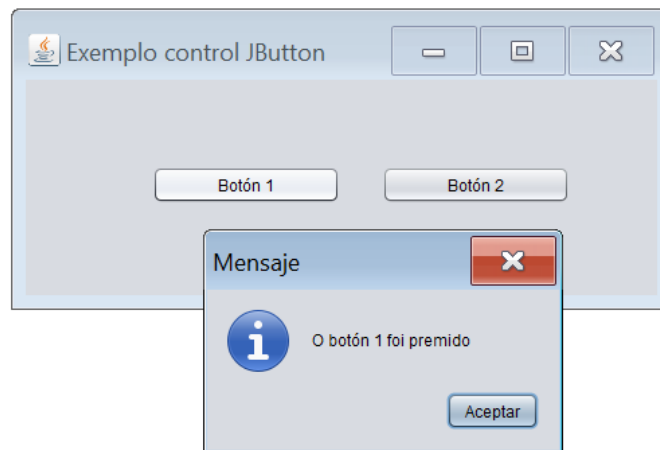
```



Realización da tarefa Ta1.2 - Aplicación multixanela MDI. Nesta tarefa crearemos unha aplicación multixanela empregando técnicas multixanela MDI.

1.2.3. Diálogos predefinidos

Con anterioridade vimos como desenvolver xanelas de diálogo. Os diálogos creados eran xanelas cuxos compoñentes eran establecidos polo programador co fin de cumprir cos requisitos das aplicacións. Non obstante, a linguaxe java proporcionáanos unha serie de xanelas de diálogo cun formato predefinido que se adecúan as situacións máis habituais. Unha destas xanelas de diálogo predefinidas témola empregado con gran asiduidade ao longo do desenvolvemento das diferentes actividades realizadas ata agora. Estamos a falar da seguinte xanela:



A xanela cuxo título é Mensaje (tamén veremos como persoalizar este texto para que sexa amosado en galego) é un diálogo modal que se xera escribindo a seguinte liña de código:

```
JOptionPane.showMessageDialog(this, "O botón 1 foi premido");
```

Esta liña afórranos o ter que crear unha xanela de diálogo, deseñar os seus compoñentes e xestionar os seus eventos. Simplifica a tarefa ao invocar un método da clase `JOptionPane` pasándolle a mensaxe que queremos amosarlle ao usuario.

A continuación imos estudar os diferentes diálogos predefinidos proporcionados pola linguaxe java e como facer emprego deles.

1.2.3.1. Diálogo `showMessageDialog`

Este tipo de diálogo emprégase para amosar un diálogo modal que vai conter unha mensaxe. Para xerar este tipo de diálogo emprégase a clase `JOptionPane`. Esta clase proporcionáanos tres métodos diferentes para crear diálogos de tipo `showMessageDialog`. En función do método empregado para crear a xanela de diálogo o seu aspecto variará. Os métodos que podemos empregar para crear unha xanela de diálogo de tipo `showMessageDialog` son os seguintes:

- `public static void showMessageDialog(Component parentComponent, Object message)`
- `public static void showMessageDialog(Component parentComponent, Object message, String title, int messageType)`
- `public static void showMessageDialog(Component parentComponent, Object message, String title, int messageType, Icon icon)`

De seguido explícase o significado de cada parámetro:

- `parentComponent` fai referencia a quen é a xanela pai (graficamente) do diálogo.
- `message`: mensaxe que se amosara na xanela.

- title: título da xanela.
- messageType: en función do valor indicado amosaranse distintas iconas predefinidas xunto á mensaxe. Os valores posibles son os seguintes: ERROR_MESSAGE, INFORMATION_MESSAGE, WARNING_MESSAGE, QUESTION_MESSAGE, PLAIN_MESSAGE (Todas son constantes da clase JOptionPane).
- icon: se en lugar dunha icona predefinida queremos amosar unha personalizada, indicáremolo mediante este parámetro. Se empregamos este parámetro o valor dado ao parámetro messageType é ignorado.

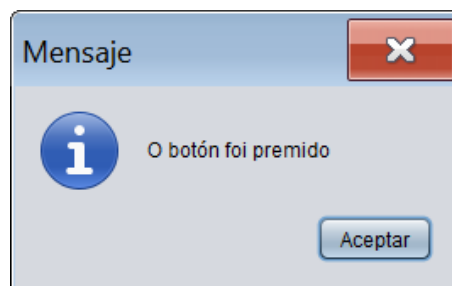
A continuación imos desenvolver unha pequena aplicación que amosa o emprego de cada un dos métodos que acabamos de comentar. O seu aspecto será o seguinte:



Ao premer sobre o primeiro botón, xeraremos un showMessageDialog empregando o primeiro método. Para elo, na implementación do actionPerformed do botón escribimos o seguinte código:

```
private void btnMetodo1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "O botón foi premido");
}
```

O resultado de premer o primeiro botón é o seguinte:

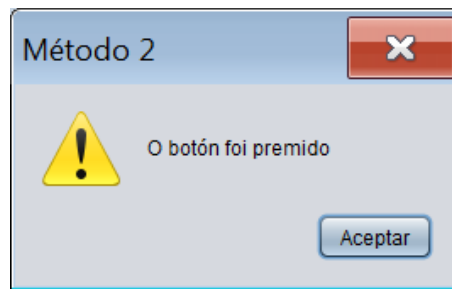


Tal e como era de esperar unicamente persoalizamos a mensaxe da xanela. Fixémonos que o título da xanela ten un texto predefinido (dependente dos locais da nosa máquina virtual, neste caso configuración española).

Ao premer sobre o segundo botón, xeraremos un showMessageDialog empregando o segundo método. Para elo, na implementación do actionPerformed do botón escribimos o seguinte código:

```
private void btnMetodo2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "O botón foi premido", "Método 2", JOptionPane.WARNING_MESSAGE);
}
```

O resultado de premer no segundo botón é o seguinte:

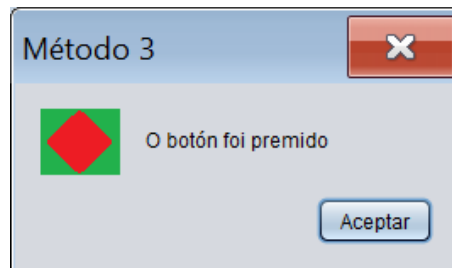


Neste caso, ademais de persoalizar a mensaxe da xanela, tamén persoalizamos o título da xanela e a icona.

Por ultimo, ao premer sobre o terceiro botón, xeraremos un `showMessageDialog` empregando o terceiro método. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

```
private void btnMetodo3ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    ImageIcon icona=new ImageIcon(getClass().getResource("/imaxes/icona.png"));
    JOptionPane.showMessageDialog(this, "O botón foi premido", "Método 3",JOptionPane.WARNING_MESSAGE, icona);
}
```

O resultado de premer o terceiro botón é o seguinte:



Neste caso, persoalizamos a mensaxe da xanela, o título da xanela e a icona, pero coa diferenza de que esta vez a icona non é unha entre as que nos proporciona o sistema senón que é unha icona persoalizada.

A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación :

```
/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemplosDialogosPredefinidos;

import java.awt.event.ItemEvent;
import javax.swing.ImageIcon;
import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class FrmShowMessageDialog extends javax.swing.JFrame {

    /**
     * Creates new form FrmPrincipal
     */
    public FrmShowMessageDialog() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        btnMetodo1 = new javax.swing.JButton();
        btnMetodo2 = new javax.swing.JButton();
        btnMetodo3 = new javax.swing.JButton();
    }
}
```

```

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
setTitle("Exemplo showMessageDialog");
getContentPane().setLayout(null);

btnMetodo1.setText("Mostrar showMessageDialog (método 1)");
btnMetodo1.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnMetodo1ActionPerformed(evt);
    }
});
getContentPane().add(btnMetodo1);
btnMetodo1.setBounds(15, 16, 612, 29);

btnMetodo2.setText("Mostrar showMessageDialog (método 2)");
btnMetodo2.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnMetodo2ActionPerformed(evt);
    }
});
getContentPane().add(btnMetodo2);
btnMetodo2.setBounds(15, 49, 612, 29);

btnMetodo3.setText("Mostrar showMessageDialog (método 3)");
btnMetodo3.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnMetodo3ActionPerformed(evt);
    }
});
getContentPane().add(btnMetodo3);
btnMetodo3.setBounds(15, 87, 612, 29);

java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
setBounds((screenSize.width-664)/2, (screenSize.height-188)/2, 664, 188);
//</editor-fold>

private void btnMetodo1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "O botón foi premido");
}

private void btnMetodo2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    JOptionPane.showMessageDialog(this, "O botón foi premido", "Método 2", JOptionPane.WARNING_MESSAGE);
}

private void btnMetodo3ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    ImageIcon icona=new ImageIcon(getClass().getResource("/imagenes/icona.png"));
    JOptionPane.showMessageDialog(this, "O botón foi premido", "Método 3",JOptionPane.WARNING_MESSAGE, icona);
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(FrmShowMessageDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(FrmShowMessageDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(FrmShowMessageDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(FrmShowMessageDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    }
    //</editor-fold>

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new FrmShowMessageDialog().setVisible(true);
        }
    });
}
// Variables declaration - do not modify
private javax.swing.JButton btnMetodo1;
private javax.swing.JButton btnMetodo2;
private javax.swing.JButton btnMetodo3;
// End of variables declaration
}

```

1.2.3.2. Diálogo showConfirmDialog

Este tipo de diálogo emprégase para pedir confirmación ao usuario. Amosa un diálogo modal que contén unha mensaxe e dous ou máis botóns, sendo posible detectar cal foi o

botón premido polo usuario. Para xerar este tipo de diálogo emprégase a clase `JOptionPane`. Esta clase proporciónanos catro métodos diferentes para crear diálogos de tipo `showConfirmDialog`. En función do método empregado para crear a xanela de diálogo o seu aspecto variará. Os métodos que podemos empregar para crear unha xanela de diálogo de tipo `showConfirmDialog` son os seguintes:

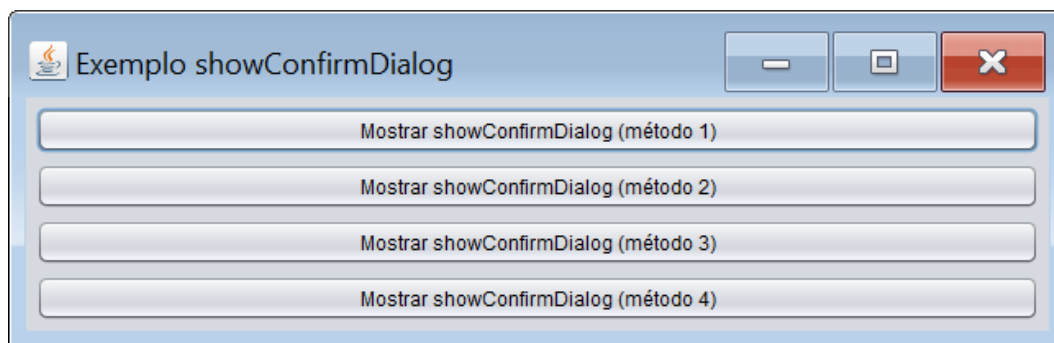
- `public static int showConfirmDialog(Component parentComponent, Object message)`
- `public static int showConfirmDialog(Component parentComponent, Object message, String title, int optionType)`
- `public static int showConfirmDialog(Component parentComponent, Object message, String title, int optionType, int messageType)`
- `public static int showConfirmDialog(Component parentComponent, Object message, String title, int optionType, int messageType, Icon icon)`

De seguido explícase o significado de cada parámetro:

- `parentComponent` fai referencia a quen é a xanela pai (graficamente) do diálogo.
- `message`: mensaxe que se amosará na xanela
- `title`: título da xanela
- `optionType`: en función do valor indicado amosaranse distintas configuracións de botóns. Os valores posibles son os seguintes: `DEFAULT_OPTION`, `YES_NO_OPTION`, `YES_NO_CANCEL_OPTION`, `OK_CANCEL_OPTION` (Todas son constantes da clase `JOptionPane`). P.e., `YES_NO_CANCEL_OPTION` presentará tres botóns (botón YES, botón NO e botón CANCEL).
- `messageType`: en función do valor indicado amosaranse distintas iconas predefinidas xunto á mensaxe. Os valores posibles son os seguintes: `ERROR_MESSAGE`, `INFORMATION_MESSAGE`, `WARNING_MESSAGE`, `QUESTION_MESSAGE`, `PLAIN_MESSAGE` (Todas son constantes da clase `JOptionPane`).
- `icon`: se en lugar dunha icona predefinida queremos amosar unha personalizada, indicáremolo mediante este parámetro. Se empregamos este parámetro o valor dado ao parámetro `messageType` é ignorado.

O método devolve un valor enteiro que indica o botón elixido polo usuario ou `JOptionPane.CLOSED_OPTION` no caso de que o usuario peche o diálogo sin premer ningún botón.

A continuación imos desenvolver unha pequena aplicación que amose o emprego de cada un dos métodos que acabamos de comentar. O seu aspecto será o seguinte:

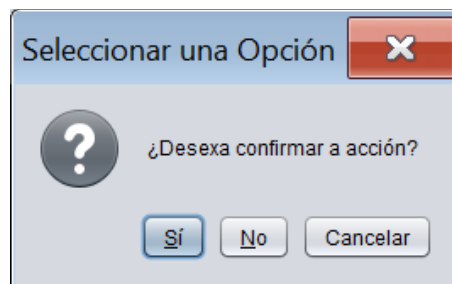


Ao premer sobre o primeiro botón, xeraremos un `showConfirmDialog` que amosa tres botóns (sí, no e cancelar). Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

```
private void btnMetodo1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showConfirmDialog(this, "¿Desexa confirmar a acción?");
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
        case JOptionPane.CANCEL_OPTION: mensaxe="Pulsado o botón Cancelar";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}
```

O resultado de premer o primeiro botón é o seguinte:



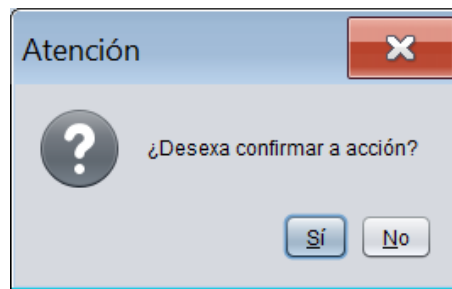
Fixémonos que o texto dos botóns está en español. Isto é debido a que cando indicamos a configuración de botóns que queremos (`YES_NO_OPTION`), o compilador facendo emprego do seu sistema de internacionalización aplica as cadeas correspondentes para a configuración actual da máquina virtual (configuración española). Outro tema destacable é ver como recollemos a acción realizada polo usuario sobre o diálogo. Neste caso o usuario pode facer unha das seguintes accións posibles: pechar a xanela, premer o botón sí, premer o botón no ou premer o botón cancelar. O método `showConfirmDialog` vainsos devolver un valor enteiro en función da acción realizada. Para cada botón predefinido a clase `JOptionPane` ten unhas constantes co valor devolto ao premer ese botón. P.e., para o botón sí é `JOptionPane.YES_OPTION`, para o botón no é `JOptionPane.NO_OPTION` e para o botón cancelar é `JOptionPane.CANCEL_OPTION` (en xeral, as constantes de `JOptionPane` de tipo `TEXTOBOTON_OPTION` representan valores devoltos polos botóns predefinidos da clase). No caso de que o valor devolto non sexa ningún deles querrá dicir que o usuario pechou a xanela.

Ao premer sobre o segundo botón, xeraremos un `showConfirmDialog` que amosa dous botóns (sí e non) e ademais un texto personalizado no título da xanela. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

```
private void btnMetodo2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showConfirmDialog(this, "¿Desexa confirmar a acción?", "Atención",JOptionPane.YES_NO_OPTION);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}
```

O resultado de premer o segundo botón é o seguinte:

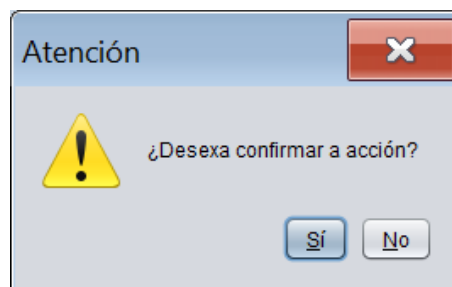


Ao premer sobre o terceiro botón, xeraremos un `showConfirmDialog` que amose dous botóns (sí e non), un texto persoalizado no título da xanela e unha icona seleccionada entre as predefinidas no sistema. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

```
private void btnMetodo3ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showConfirmDialog(this, "¿Desexa confirmar a acción?", "Atención", JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:      mensaxe="Pulsado o botón Non";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}
```

O resultado de premer o terceiro botón é o seguinte:

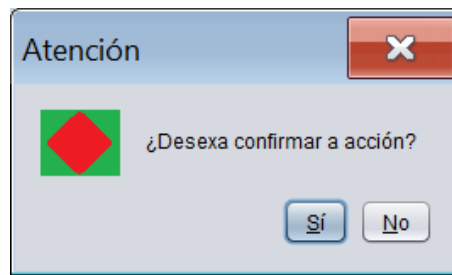


Por último, ao premer sobre o cuarto botón, xeraremos un `showConfirmDialog` que amose dous botóns (sí e non), un texto persoalizado no título da xanela e unha icona personalizada. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

```
private void btnMetodo4ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    ImageIcon icona=new ImageIcon(getClass().getResource("/imaxes/icona.png"));
    int seleccion=JOptionPane.showConfirmDialog(this, "¿Desexa confirmar a acción?", "Atención",JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE,icona);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:      mensaxe="Pulsado o botón Non";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}
```

O resultado de premer o cuarto botón é o seguinte:



A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación:

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemplosDialogosPredefinidos;

import javax.swing.ImageIcon;
import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class FrmShowConfirmDialog extends javax.swing.JFrame {

    /**
     * Creates new form FrmShowConfirmDialog
     */
    public FrmShowConfirmDialog() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        btnMetodo1 = new javax.swing.JButton();
        btnMetodo2 = new javax.swing.JButton();
        btnMetodo3 = new javax.swing.JButton();
        btnMetodo4 = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Exemplo showConfirmDialog");

        btnMetodo1.setText("Mostrar showConfirmDialog (método 1)");
        btnMetodo1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnMetodo1ActionPerformed(evt);
            }
        });

        btnMetodo2.setText("Mostrar showConfirmDialog (método 2)");
        btnMetodo2.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnMetodo2ActionPerformed(evt);
            }
        });

        btnMetodo3.setText("Mostrar showConfirmDialog (método 3)");
        btnMetodo3.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnMetodo3ActionPerformed(evt);
            }
        });

        btnMetodo4.setText("Mostrar showConfirmDialog (método 4)");
        btnMetodo4.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnMetodo4ActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
        getContentPane().setLayout(layout);
        layout.setHorizontalGroup(
            layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(layout.createSequentialGroup()
                    .add(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                        .add(btnMetodo1)
                        .add(btnMetodo2)
                        .add(btnMetodo3)
                        .add(btnMetodo4))
                    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
        );
    }
}

```

```

    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
            .addContainerGap()
            .addComponent(btnMetodo1)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(btnMetodo2)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(btnMetodo3)
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
            .addComponent(btnMetodo4)
            .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
    );

    pack();
} // </editor-fold>

private void btnMetodo1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showConfirmDialog(this, "¿Desexa confirmar a acción?");
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
        case JOptionPane.CANCEL_OPTION:  mensaxe="Pulsado o botón Cancelar";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

private void btnMetodo2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showConfirmDialog(this, "¿Desexa confirmar a acción?", "Atención",JOptionPane.YES_NO_OPTION);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

private void btnMetodo3ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showConfirmDialog(this, "¿Desexa confirmar a acción?", "Atención",JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

private void btnMetodo4ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    ImageIcon icona=new ImageIcon(getClass().getResource("/imaxes/icona.png"));
    int seleccion=JOptionPane.showConfirmDialog(this, "¿Desexa confirmar a acción?", "Atención",JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE,icona);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ClassNotFoundException ex) {
        java.util.logging.Logger.getLogger(FrmShowConfirmDialog.class.getName()).log(java.util.logging.Level.SEVERE,
        null, ex);
    }
}

```



```

    } catch (InstantiationException ex) {
        java.util.logging.Logger.getLogger(FrmShowConfirmDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (IllegalAccessException ex) {
        java.util.logging.Logger.getLogger(FrmShowConfirmDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    } catch (javax.swing.UnsupportedLookAndFeelException ex) {
        java.util.logging.Logger.getLogger(FrmShowConfirmDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
    }
}
//</editor-fold>

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new FrmShowConfirmDialog().setVisible(true);
    }
});
}
// Variables declaration - do not modify
private javax.swing.JButton btnMetodo1;
private javax.swing.JButton btnMetodo2;
private javax.swing.JButton btnMetodo3;
private javax.swing.JButton btnMetodo4;
// End of variables declaration
}

```

1.2.3.3. Diálogo showInputDialog

Este tipo de diálogo emprégase para amosar un diálogo modal para solicitar información ao usuario. O usuario poderá introducir a información ben a través dunha caixa de texto ou ben seleccionar un valor posible entre varios ofertados. Para xerar este tipo de diálogo emprégase a clase JOptionPane. Esta clase vainos proporcionar seis métodos diferentes para crear diálogos de tipo showInputDialog. En función do método empregado para crear a xanela de diálogo o seu aspecto variará. Os métodos que podemos empregar para crear unha xanela de diálogo de tipo showInputDialog son os seguintes:

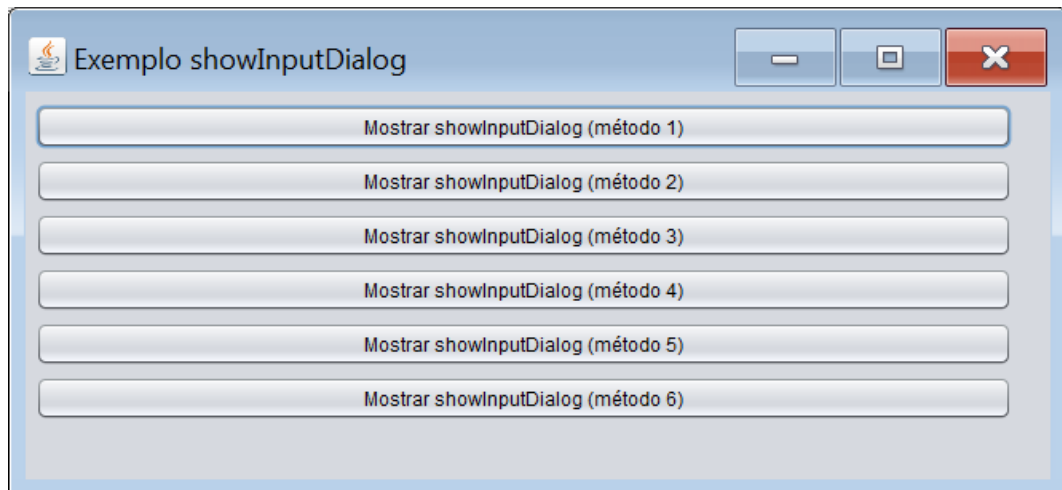
- public static String showInputDialog(Object message)
- public static String showInputDialog(Object message, Object initialSelectionValue)
-
- public static String showInputDialog(Component parentComponent, Object message)
- public static String showInputDialog(Component parentComponent, Object message, Object initialSelectionValue)
- public static String showInputDialog(Component parentComponent, Object message, String title, int messageType)
- public static Object showInputDialog(Component parentComponent, Object message, String title, int messageType, Icon icon, Object[] selectionValues, Object initialSelectionValue)

De seguido explícase o significado de cada parámetro:

- parentComponent fai referencia a quen é a xanela pai (graficamente) do diálogo.
- message: mensaxe que se amosa na xanela
- title: título da xanela
- messageType: en función do valor indicado amosaranse distintas iconas predefinidas xunto á mensaxe. Os valores posibles son os seguintes: ERROR_MESSAGE, INFORMATION_MESSAGE, WARNING_MESSAGE, QUESTION_MESSAGE, PLAIN_MESSAGE (Todas son constantes da clase JOptionPane).
- icon: se en lugar dunha icona predefinida queremos amosar unha persoalizada, indicáremolo mediante este parámetro. Se empregamos este parámetro o valor dado ao parámetro messageType é ignorado.

- **SelectionValues:** no caso de que este parámetro sexa distinto de null, o diálogo recibirá un array con diferentes valores que se ofertarán ao usuario podendo este elixir un deles. Os posibles valores serán visualizados nun combo. No caso de que este parámetro sexa null, o compoñente amosado para recoller a resposta do usuario será unha caixa de texto.
- **InitialSelection:** valor inicial ofertado ao usuario. No caso de que a xanela amose unha caixa de texto amosarase este valor nela. No caso de que na xanela amósesse un combo deberase indicar un dos valores do combo e ese será o preseleccionado.

A continuación imos desenvolver unha pequena aplicación que amosa o emprego de cada un dos métodos que acabamos de comentar. O seu aspecto será o seguinte:



Ao premer sobre o primeiro botón ou sobre o terceiro botón, xeraremos un `showInputDialog` que amosa dous botóns e unha caixa de texto na cal o usuario pode introducir un valor. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

Para o primeiro botón:

```
private void btnMetodo1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog("Indique o seu domicilio");
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }
    System.out.println(domicilio);
}
```

Para o terceiro botón:

```
private void btnMetodo3ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog(this, "Indique o seu domicilio");
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {

```

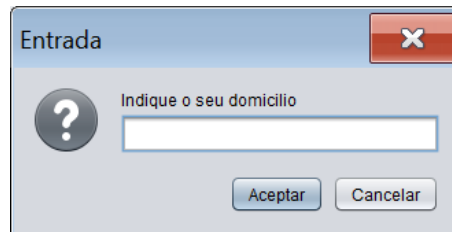
```

JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
    }

    }
    System.out.println(domicilio);
}

```

O resultado de premer o primeiro ou o terceiro botón é o seguinte:



O usuario poderá introducir un valor na caixa de texto do diálogo. Cando o diálogo é pechado, o método devolve un obxecto que contén o valor introducido polo usuario. No caso de que o usuario peche a xanela ben ca aspa o ben co botón Cancelar, o valor devolto será null. A única diferencia entre a creación do diálogo empregando o primeiro método e empregando o terceiro consiste en que se empregamos o terceiro, mediante o parámetro `parentComponent` estámoslle pasando ao diálogo quen é a súa xanela pai. Isto finalmente tradúcese no seguinte: empregando o primeiro método ao abrir o diálogo, este ábrese centrado na pantalla. Non obstante, se empregamos o terceiro método, ao abrir o diálogo, este ábrese no centro da súa xanela pai.

Ao premer sobre o segundo botón ou sobre o cuarto botón, xeraremos un `showInputDialog` que amosa dous botóns e unha caixa de texto na cal o usuario pode introducir un valor. Adicionalmente a caixa de texto ofrece un valor preseleccionado ao usuario. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

Para o segundo botón:

```

private void btnMetodo2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog("Indique o seu domicilio","Rúa Descoñecida s/n, 15001, A Coruña");
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }
    System.out.println(domicilio);
}

```

Para o cuarto botón:

```

private void btnMetodo4ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog(this, "Indique o seu domicilio","Rúa Descoñecida s/n, 15001, A
Coruña");
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else

```

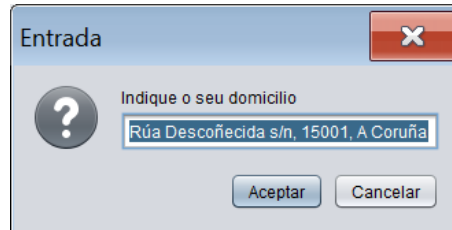
```

        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }

    }
    System.out.println(domicilio);
}

```

O resultado de premer o segundo ou o cuarto botón é o seguinte:



Como pódese observar na imaxe a caixa de texto ten un valor inicial preseleccionado que o usuario poderá cambiar. A única diferenza entre a creación do diálogo empregando o segundo método e empregando o cuarto consiste en que se empregamos o cuarto, mediante o parámetro `parentComponent` estamoslle pasando ao diálogo quen é a súa xanela pai. Ao igual que ocorría anteriormente, empregando o segundo método ao abrir o diálogo, este ábrese centrado na pantalla, mentres que se empregamos o cuarto método, ao abrir o diálogo, este ábrese no centro da súa xanela pai.

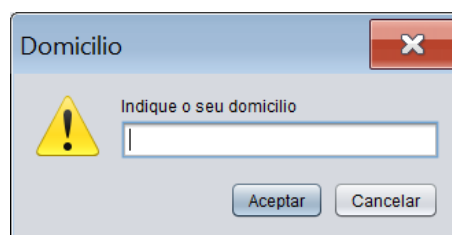
Ao premer sobre o quinto botón, xeraremos un `showInputDialog` que amosa dous botóns e unha caixa de texto na cal o usuario pode introducir un valor. Ademais podemos persoalizar o título da xanela e amosar unha icona predefinida. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

```

private void btnMetodo5ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog(this, "Indique o seu
domicilio", "Domicilio",JOptionPane.WARNING_MESSAGE);
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }
    System.out.println(domicilio);
}

```

O resultado de premer o quinto botón é o seguinte:

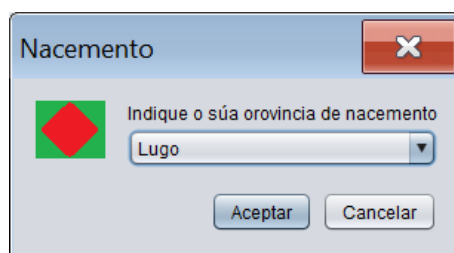


Por último, ao premer sobre o sexto botón, xeraremos un `showInputDialog` que amosa dous botóns e un combo con varios valores posibles entre os cales o usuario pode elixir un. Adicionalmente preseleccionaremos un valor concreto do combo. Ademais podemos

personalizar o título da xanela e amosar unha icona predefinida ou persoalizada. Para elo, na implementación do actionPerformed do botón escribimos o seguinte código:

```
private void btnMetodo6ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    ImageIcon icona=new ImageIcon(getClass().getResource("/imaxes/icona.png"));
    String provincias[]={ "A Coruña","Lugo","Ourense","Pontevedra"};
    Object seleccion=JOptionPane.showInputDialog(this, "Indique o súa orovincia de nacemento","Nacemento",
    JOptionPane.QUESTION_MESSAGE, icona, provincias, "Lugo");
    if(seleccion==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        JOptionPane.showMessageDialog(this,"A provincia seleccionada é "+seleccion, "Atención",
        JOptionPane.INFORMATION_MESSAGE);
    }
}
```

O resultado de premer o sexto botón é o seguinte:



Como pódese observar preséntase ao usuario un combo cunha serie de valores entre os cales pódese elixir un. Para personalizar os contidos do combo creamos un arrai de obxectos (neste caso aplicando herdanza é de Strings). Cada elemento deste arrai mapearase sobre un elemento do combo. Ademais mediante o parámetro initialSelectionValue podemos indicar un obxecto dese arrai que debe ser preseleccionado no combo. No caso de que o parámetro selectionValues sexa null a xanela de diálogo amosará unha caixa de texto en lugar dun combo.

A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación:

```
/**
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemplosDialogosPredefinidos;

import javax.swing.ImageIcon;
import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class FrmShowInputDialog extends javax.swing.JFrame {

    /**
     * Creates new form FrmShowInputDialog
     */
    public FrmShowInputDialog() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        btnMetodo1 = new javax.swing.JButton();
        btnMetodo2 = new javax.swing.JButton();
        btnMetodo3 = new javax.swing.JButton();
        btnMetodo4 = new javax.swing.JButton();
        btnMetodo5 = new javax.swing.JButton();
        btnMetodo6 = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Exemplo showInputDialog");

        btnMetodo1.setText("Mostrar showInputDialog (método 1)");
        btnMetodo1.addActionListener(new java.awt.event.ActionListener() {
```

```

        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnMetodo1ActionPerformed(evt);
        }
    });

    btnMetodo2.setText("Mostrar showInputDialog (método 2)");
    btnMetodo2.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnMetodo2ActionPerformed(evt);
        }
    });

    btnMetodo3.setText("Mostrar showInputDialog (método 3)");
    btnMetodo3.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnMetodo3ActionPerformed(evt);
        }
    });

    btnMetodo4.setText("Mostrar showInputDialog (método 4)");
    btnMetodo4.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnMetodo4ActionPerformed(evt);
        }
    });

    btnMetodo5.setText("Mostrar showInputDialog (método 5)");
    btnMetodo5.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnMetodo5ActionPerformed(evt);
        }
    });

    btnMetodo6.setText("Mostrar showInputDialog (método 6)");
    btnMetodo6.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnMetodo6ActionPerformed(evt);
        }
    });

    javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(btnMetodo1, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                    .addComponent(btnMetodo2, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                    .addComponent(btnMetodo3, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                    .addComponent(btnMetodo4, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                    .addComponent(btnMetodo5, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                    .addComponent(btnMetodo6, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
                .addContainerGap())
    );

    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(layout.createSequentialGroup()
                .addContainerGap()
                .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(btnMetodo1)
                    .addComponent(btnMetodo2)
                    .addComponent(btnMetodo3)
                    .addComponent(btnMetodo4)
                    .addComponent(btnMetodo5)
                    .addComponent(btnMetodo6))
                .addContainerGap())
    );

    java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
    setBounds((screenSize.width-664)/2, (screenSize.height-307)/2, 664, 307);
} // </editor-fold>

private void btnMetodo1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog("Indique o seu domicilio");
    if (domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if (domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }
    System.out.println(domicilio);
}

```

```

private void btnMetodo2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog("Indique o seu domicilio","Rúa Descoñecida s/n, 15001, A Coruña");
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }
    System.out.println(domicilio);
}

private void btnMetodo3ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog(this, "Indique o seu domicilio");
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }
    System.out.println(domicilio);
}

private void btnMetodo4ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog(this, "Indique o seu domicilio","Rúa Descoñecida s/n, 15001, A
Coruña");
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }
    System.out.println(domicilio);
}

private void btnMetodo5ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String domicilio=JOptionPane.showInputDialog(this, "Indique o seu
domicilio","Domicilio",JOptionPane.WARNING_MESSAGE);
    if(domicilio==null)
    {
        JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
    }
    else
    {
        if(domicilio.trim().compareTo("")==0)
        {
            JOptionPane.showMessageDialog(this,"A caixa de texto está baleira", "Atención",
JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"O seu domicilio é "+domicilio, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }
    System.out.println(domicilio);
}

private void btnMetodo6ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    ImageIcon icona=new ImageIcon(getClass().getResource("/imaxes/icona.png"));
    String provincias[]={ "A Coruña", "Lugo", "Ourense", "Pontevedra" };

```

```

        Object seleccion=JOptionPane.showInputDialog(this, "Indique o súa orovincia de nacemento","Nacemento",
JOptionPane.QUESTION_MESSAGE, icona, provincias, "Lugo");
        if(seleccion==null)
        {
            JOptionPane.showMessageDialog(this,"Acción anulada polo usuario", "Atención", JOptionPane.ERROR_MESSAGE);
        }
        else
        {
            JOptionPane.showMessageDialog(this,"A provincia seleccionada é "+seleccion, "Atención",
JOptionPane.INFORMATION_MESSAGE);
        }
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
         * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {
            java.util.logging.Logger.getLogger(FrmShowInputDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
        } catch (InstantiationException ex) {
            java.util.logging.Logger.getLogger(FrmShowInputDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
        } catch (IllegalAccessException ex) {
            java.util.logging.Logger.getLogger(FrmShowInputDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {
            java.util.logging.Logger.getLogger(FrmShowInputDialog.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
        }
        //</editor-fold>

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new FrmShowInputDialog().setVisible(true);
            }
        });
    }
    // Variables declaration - do not modify
    private javax.swing.JButton btnMetodo1;
    private javax.swing.JButton btnMetodo2;
    private javax.swing.JButton btnMetodo3;
    private javax.swing.JButton btnMetodo4;
    private javax.swing.JButton btnMetodo5;
    private javax.swing.JButton btnMetodo6;
    // End of variables declaration
}

```

1.2.3.4. Diálogo showOptionDialog

Este tipo de diálogo é o máis persoalizable de todos. Con el podemos crear diálogos de tipo showMessageDialog e diálogos de tipo showConfirmDialog nos que é posible configurar calquera elemento do diálogo. O diálogo de tipo showOptionDialog emprégase para amosar un diálogo modal que contén unha mensaxe e un ou máis botóns, sendo posible detectar cal foi o botón premido polo usuario. Para xerar este tipo de diálogo emprégase a clase JOptionPane. Esta clase proporciónanos un único método, pero en función dos parámetros que lle pasemos, a xanela de diálogo variará o seu aspecto. O método que empregamos para crear unha xanela de diálogo de tipo showOptionDialog é o seguinte:

- public static int showOptionDialog(Component parentComponent, Object message,String title, int optionType, int messageType, Icon icon, Object [] options, Object initialValue)

De seguido explícase o significado de cada parámetro:

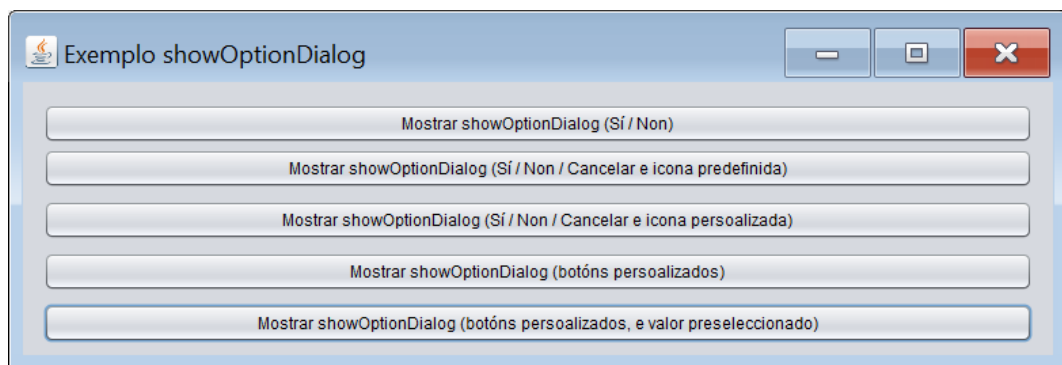
- parentComponent fai referencia a quen é a xanela pai (graficamente) do diálogo.
- message: mensaxe que se amosa na xanela
- title: título da xanela
- optionType: en función do valor indicado amosaranse distintas configuracións de botóns. Os valores posibles son os seguintes: DEFAULT_OPTION, YES_NO_OPTION,

YES_NO_CANCEL_OPTION, OK_CANCEL_OPTION (Todas son constantes da clase JOptionPane). P.e., YES_NO_CANCEL_OPTION presentará tres botóns (botón YES, botón NO e botón CANCEL).

- messageType: en función do valor indicado amosaranse distintas iconas predefinidas xunto á mensaxe. Os valores posibles son os seguintes: ERROR_MESSAGE, INFORMATION_MESSAGE, WARNING_MESSAGE, QUESTION_MESSAGE, PLAIN_MESSAGE (Todas son constantes da clase JOptionPane).
- icon: se en lugar dunha icona predefinida queremos amosar unha persoalizada, o indicaremos mediante este parámetro. Se empregamos este parámetro, o valor dado ao parámetro messageType é ignorado.
- Options: mediante este parámetro defínense os textos dos botóns no caso de que queiramos empregar botóns persoalizados.
- initialValue: en caso de que empreguemos botóns persoalizados, mediante este parámetro podemos indicar cal é o botón preseleccionado por defecto.

O método devolve un valor enteiro que indica o botón elixido polo usuario o CLOSED_OPTION no caso de que o usuario pechase o diálogo sen premer ningún botón.

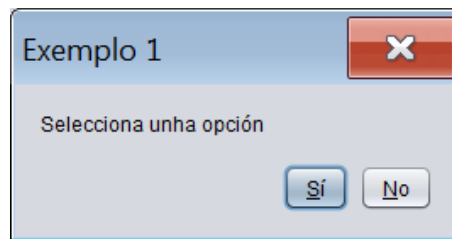
De seguido imos desenvolver unha pequena aplicación que amosa o emprego deste método para distintas configuracións de xanela. O seu aspecto será o seguinte:



Al pulsar sobre o primeiro botón, xeraremos un showOptionDialog que amose dous botóns (sí e no). Para elo, na implementación do actionPerformed do botón escribimos o seguinte código:

```
private void btnSiNonActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showOptionDialog(this, "Selecciona unha opción", "Exemplo 1",
    JOptionPane.YES_NO_OPTION, JOptionPane.PLAIN_MESSAGE, null, null, null);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
    }
    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}
```

O resultado de premer o primeiro botón é o seguinte:



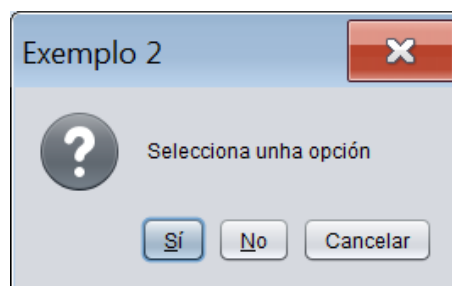
Fixémonos que o texto dos botóns está en español. Isto é debido a que cando indicamos a configuración de botóns que queremos (YES_NO_OPTION), o compilador facendo emprego do seu sistema de internacionalización aplica as cadeas correspondentes para a configuración actual da máquina virtual (configuración española). Outro tema destacable é ver como recolleemos a acción realizada polo usuario sobre o diálogo. Neste caso o usuario pode facer unha das seguintes accións: pechar a xanela, premer o botón sí ou premer o botón no. O método `showOptionDialog` devolveranos un valor enteiro en función da acción realizada. Para cada botón predefinido, a clase `JOptionPane` ten unha constante co valor devolto ao premer nese botón. P.e., para o botón sí é `JOptionPane.YES_OPTION` e para o botón no é `JOptionPane.NO_OPTION` (en xeral, as constantes de `JOptionPane` de tipo `TEXTOBOTON_OPTION` representan valores devoltos polos botóns predefinidos da clase). No caso de que o valor devolto non sexa ningún deles significará que o usuario pechou a xanela.

Ao premer sobre o segundo botón, xeraremos un `showOptionDialog` que amose tres botóns (sí, no e cancelar) e unha icona predefinida. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

```
private void btnSiNonCancelarIconaPredefinidaActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showOptionDialog(this, "Selecciona unha opción", "Exemplo 2",
    JOptionPane.YES_NO_CANCEL_OPTION, JOptionPane.QUESTION_MESSAGE, null, null, null);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
        case JOptionPane.CANCEL_OPTION:  mensaxe="Pulsado o botón Cancelar";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}
```

O resultado de premer o segundo botón é o seguinte:



Ao premer sobre o terceiro botón, xeraremos un `showOptionDialog` que amose tres botóns (sí, no e cancelar) e unha icona persoalizada. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

```
private void btnSiNonCancelarIconaPersoalizadaActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    ImageIcon icona=new ImageIcon(getClass().getResource("/imaxes/icona.png"));
    int seleccion=JOptionPane.showOptionDialog(this, "Selecciona unha opción", "Exemplo 3",
    JOptionPane.YES_NO_CANCEL_OPTION, JOptionPane.QUESTION_MESSAGE, icona, null, null);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
        case JOptionPane.CANCEL_OPTION:  mensaxe="Pulsado o botón Cancelar";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}
```

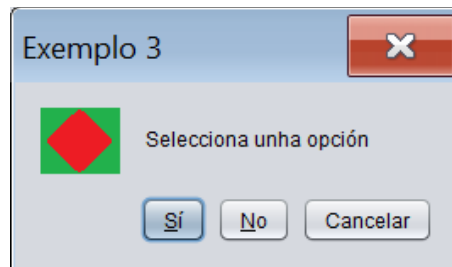
```

    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
        case JOptionPane.CANCEL_OPTION: mensaxe="Pulsado o botón Cancelar";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

```

O resultado de premer o terceiro botón é o seguinte:



Ao premer sobre o cuarto botón, xeraremos un `showOptionDialog` que amose tres botóns persoalizados. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

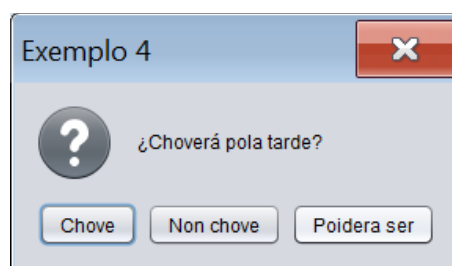
```

private void btnBotonsPersoalizadosActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String botons[]={"Chove","Non chove","Poidera ser"};
    int seleccion=JOptionPane.showOptionDialog(this,    "¿Choverá pola tarde?",    "Exemplo 4",
    JOptionPane.YES_NO_CANCEL_OPTION,    JOptionPane.QUESTION_MESSAGE,    null,    botons,    null);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case 0:    mensaxe="Vai chover";
                   break;
        case 1:    mensaxe="Non vai chover";
                   break;
        case 2: mensaxe="Poidera ser que chova. Xa veremos";
                   break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

```

O resultado de premer o cuarto botón é o seguinte:



Para persoalizar os textos dos botóns creamos un array de Strings. Cada elemento deste array mapearase sobre un botón do diálogo. A orde dos botóns no diálogo será a orde dos Strings no array de Strings. Para recuperar o botón premido farémolo pola súa posición dentro do array de Strings, de xeito que se prememos o primeiro botón, o diálogo devolverá un 0, se prememos sobre o segundo botón o diálogo devolverá un 1, etc.

Por último, ao premer sobre o quinto botón, xeraremos un `showOptionDialog` que amose tres botóns persoalizados e ademais preseleccione un deles. Por defecto se non se indica nada sempre preseleccionarase o primeiro botón. A preselección persoalizada de botóns unicamente pódese aplicar cando os botóns son persoalizados. Para elo, na implementación do `actionPerformed` do botón escribimos o seguinte código:

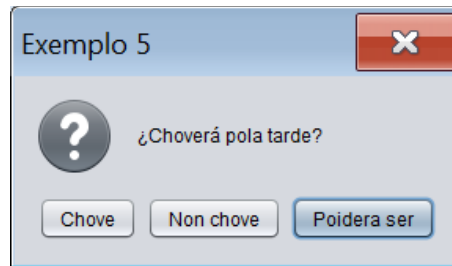
```

private void btnBotonsPersoalizadosEValorPreseleccionadoActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String botons[]={"Chove","Non chove","Poidera ser"};
    int seleccion=JOptionPane.showOptionDialog(this, "¿Choverá pola tarde?", "Exemplo 4",
JOptionPane.YES_NO_CANCEL_OPTION, JOptionPane.QUESTION_MESSAGE, null, botons, botons[2]);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case 0: mensaxe="Vai chover"; break;
        case 1: mensaxe="Non vai chover"; break;
        case 2: mensaxe="Poidera ser que chova. Xa veremos"; break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

```

O resultado de premer o quinto botón é o seguinte:



Como pódese observar na imaxe anterior, o terceiro botón está preseleccionado. Para facelo indicamos no último parámetro do método `cal` de tódolos elementos do array de Strings que empregamos para xerar os botóns é o que queremos preseleccionar.

A continuación amósase o listado do código fonte desenvolvido para a realización da aplicación:

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package exemplosDialogosPredefinidos;

import javax.swing.ImageIcon;
import javax.swing.JOptionPane;

/**
 *
 * @author German DR
 */
public class FrmShowOptionDialog extends javax.swing.JFrame {

    /**
     * Creates new form FrmShowOptionDialog
     */
    public FrmShowOptionDialog() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        btnSiNon = new javax.swing.JButton();
        btnSiNonCancelarIconaPredefinida = new javax.swing.JButton();
        btnSiNonCancelarIconaPersoalizada = new javax.swing.JButton();
        btnBotonsPersoalizados = new javax.swing.JButton();
        btnBotonsPersoalizadosEValorPreseleccionado = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Exemplo showOptionDialog");
        getContentPane().setLayout(null);

        btnSiNon.setText("Mostrar showOptionDialog (Si / Non)");
        btnSiNon.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSiNonActionPerformed(evt);
            }
        });
        getContentPane().add(btnSiNon);
        btnSiNon.setBounds(15, 16, 729, 29);

        btnSiNonCancelarIconaPredefinida.setText("Mostrar showOptionDialog (Si / Non / Cancelar e icona predefinida)");
        btnSiNonCancelarIconaPredefinida.addActionListener(new java.awt.event.ActionListener() {

```

```

        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnSiNonCancelarIconaPredefinidaActionPerformed(evt);
        }
    });
    getContentPane().add(btnSiNonCancelarIconaPredefinida);
    btnSiNonCancelarIconaPredefinida.setBounds(15, 49, 729, 29);

    btnSiNonCancelarIconaPersoalizada.setText("Mostrar showOptionDialog (Si / Non / Cancelar e icona persoalizada)");
    btnSiNonCancelarIconaPersoalizada.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnSiNonCancelarIconaPersoalizadaActionPerformed(evt);
        }
    });
    getContentPane().add(btnSiNonCancelarIconaPersoalizada);
    btnSiNonCancelarIconaPersoalizada.setBounds(15, 87, 729, 29);

    btnBotonsPersoalizados.setText("Mostrar showOptionDialog (botóns persoalizados)");
    btnBotonsPersoalizados.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnBotonsPersoalizadosActionPerformed(evt);
        }
    });
    getContentPane().add(btnBotonsPersoalizados);
    btnBotonsPersoalizados.setBounds(15, 125, 729, 29);

    btnBotonsPersoalizadosEValorPreseleccionado.setText("Mostrar showOptionDialog (botóns persoalizados, e valor preseleccionado)");
    btnBotonsPersoalizadosEValorPreseleccionado.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnBotonsPersoalizadosEValorPreseleccionadoActionPerformed(evt);
        }
    });
    getContentPane().add(btnBotonsPersoalizadosEValorPreseleccionado);
    btnBotonsPersoalizadosEValorPreseleccionado.setBounds(15, 163, 729, 29);

    java.awt.Dimension screenSize = java.awt.Toolkit.getDefaultToolkit().getScreenSize();
    setBounds((screenSize.width-781)/2, (screenSize.height-265)/2, 781, 265);
} // </editor-fold>

private void btnSiNonActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showOptionDialog(this, "Selecciona unha opción", "Exemplo 1",
    JOptionPane.YES_NO_OPTION, JOptionPane.PLAIN_MESSAGE, null, null, null);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

private void btnSiNonCancelarIconaPredefinidaActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int seleccion=JOptionPane.showOptionDialog(this, "Selecciona unha opción", "Exemplo 2",
    JOptionPane.YES_NO_CANCEL_OPTION, JOptionPane.QUESTION_MESSAGE, null, null, null);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
        case JOptionPane.CANCEL_OPTION: mensaxe="Pulsado o botón Cancelar";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

private void btnSiNonCancelarIconaPersoalizadaActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    ImageIcon icona=new ImageIcon(getClass().getResource("/imaxes/icona.png"));
    int seleccion=JOptionPane.showOptionDialog(this, "Selecciona unha opción", "Exemplo 3",
    JOptionPane.YES_NO_CANCEL_OPTION, JOptionPane.QUESTION_MESSAGE, icona, null, null);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case JOptionPane.YES_OPTION:    mensaxe="Pulsado o botón Si";
                                         break;
        case JOptionPane.NO_OPTION:     mensaxe="Pulsado o botón Non";
                                         break;
        case JOptionPane.CANCEL_OPTION: mensaxe="Pulsado o botón Cancelar";
                                         break;
    }

    JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
}

private void btnBotonsPersoalizadosActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String botons[]{"Chove","Non choke","Poidera ser"};
    int seleccion=JOptionPane.showOptionDialog(this, "¿Choverá pola tarde?", "Exemplo 4",
    JOptionPane.YES_NO_CANCEL_OPTION, JOptionPane.QUESTION_MESSAGE, null, botons, null);
    String mensaxe="Xanela pechada polo usuario";
    switch(seleccion)
    {
        case 0:    mensaxe="Vai chover";
                  break;
        case 1:    mensaxe="Non vai chover";
    }
}

```

```

                break;
            case 2: mensaxe="Poidera ser que chova. Xa veremos";
                    break;
        }

        JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
    }

    private void btnBotonsPersoalizadosEValorPreseleccionadoActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        String botons[]={"Chove","Non chove","Poidera ser"};
        int seleccion=JOptionPane.showOptionDialog(this, ";Choverá pola tarde?", "Exemplo 4",
        JOptionPane.YES_NO_CANCEL_OPTION, JOptionPane.QUESTION_MESSAGE, null, botons, botons[2]);
        String mensaxe="Xanela pechada polo usuario";
        switch(seleccion)
        {
            case 0:    mensaxe="Vai chover";
                        break;
            case 1:    mensaxe="Non vai chover";
                        break;
            case 2: mensaxe="Poidera ser que chova. Xa veremos";
                        break;
        }

        JOptionPane.showMessageDialog(this, mensaxe, "Resultado",JOptionPane.INFORMATION_MESSAGE);
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
         * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException ex) {
            java.util.logging.Logger.getLogger(FrmShowOptionDialog.class.getName()).log(java.util.logging.Level.SEVERE,
            null, ex);
        } catch (InstantiationException ex) {
            java.util.logging.Logger.getLogger(FrmShowOptionDialog.class.getName()).log(java.util.logging.Level.SEVERE,
            null, ex);
        } catch (IllegalAccessException ex) {
            java.util.logging.Logger.getLogger(FrmShowOptionDialog.class.getName()).log(java.util.logging.Level.SEVERE,
            null, ex);
        } catch (javax.swing.UnsupportedLookAndFeelException ex) {
            java.util.logging.Logger.getLogger(FrmShowOptionDialog.class.getName()).log(java.util.logging.Level.SEVERE,
            null, ex);
        }
        //</editor-fold>

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new FrmShowOptionDialog().setVisible(true);
            }
        });
    }
    // Variables declaration - do not modify
    private javax.swing.JButton btnBotonsPersoalizados;
    private javax.swing.JButton btnBotonsPersoalizadosEValorPreseleccionado;
    private javax.swing.JButton btnSiNon;
    private javax.swing.JButton btnSiNonCancelarIconaPersoalizada;
    private javax.swing.JButton btnSiNonCancelarIconaPredefinida;
    // End of variables declaration
}

```

1.2.3.5. Consideracións finais

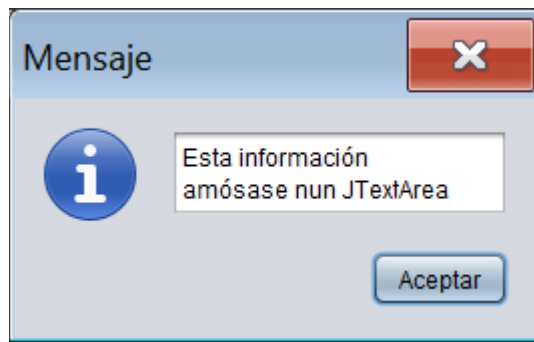
Se prestamos atención aos construtores dalgúns dos diálogos predefinidos que acabamos de explicar, veremos que en algunhas ocasións algúns dos parámetros destes construtores son de clase Object. Non obstante, nos, para eses parámetros, empregamos obxectos de clase String, xa que habitualmente nunha xanela de diálogo amósanse cadeas de texto ao usuario. Se para eses parámetros en lugar de empregar obxectos String empregamos obxectos doutra clase, amosarase a información que devolvan os seus métodos toString. Se ademais os obxectos empregados son dalgunha clase de compoñente gráfico podemos obter resultados curiosos. P.e.: para as seguintes liñas de código:

```

JTextArea txtarInfo=new JTextArea("Esta información\namósase nun JTextArea");
JOptionPane.showMessageDialog(this, txtarInfo);

```

amósase a seguinte xanela:



O máis habitual é traballar con Strings, pero é bo lembrar que sempre hai outras opcións.